

The Mathematics of Machine Learning

From First Principles to a Neural Network in C++

Riccardo di Silvio

Contents

1	Linear Algebra	1
1.1	Vector Spaces and Linear Operators	1
1.1.1	Vector Spaces	1
1.1.2	Vector Subspaces	2
1.1.3	Linear Independence	3
1.1.4	Bases of Vector Spaces	4
1.1.5	Linear Operators	5
1.2	The LU Decomposition	7
1.2.1	Gaussian Elimination	7
1.2.2	The LU Factorisation	8
1.2.3	Solving the Linear System	9
1.2.4	Partial Pivoting	10
1.3	Inner Product Spaces and QR Decompositions	12
1.3.1	Inner Product Spaces	12
1.3.2	Orthogonality	15
1.3.3	QR Decompositions	17
1.4	Eigenvalues and the Spectral Theorem	19
1.4.1	Eigenvalues and Eigenvectors	19
1.4.2	The Spectral Theorem	22
1.5	The Singular Value Decomposition	25
2	Calculus and Optimisation	30
2.1	Multivariable Calculus	30
2.1.1	Differentiability and the Jacobian	30
2.1.2	The Chain Rule	32
2.1.3	Gradient as Steepest Ascent	33
2.2	Gradient Descent	34
2.2.1	Convexity and L -Smoothness	34
2.2.2	The Descent Lemma	35
2.2.3	Convergence of Gradient Descent	36
2.3	Lagrange Multipliers	37
3	Probability and Statistics	40
3.1	Probability Foundations	40
3.1.1	Probability Spaces and Kolmogorov's Axioms	40
3.1.2	Conditional Probability and Independence	41
3.1.3	Bernoulli and Categorical Distributions	42
3.1.4	Random Variables and Expectation	42

3.1.5	Jensen's Inequality and Kullback-Leibler Divergence	43
3.2	Maximum Likelihood Estimation and Cross-Entropy	45
3.2.1	Maximum Likelihood Estimation	45
3.2.2	Cross-Entropy	47
4	Neural Networks	49
4.1	Network Architecture	49
4.1.1	Affine Maps	49
4.1.2	Activation Functions	50
4.1.3	Feedforward Networks	51
4.1.4	The Softmax Output Layer	53
4.2	Backpropagation	54
4.2.1	The Loss Function	54
4.2.2	The Backward Pass	55
4.3	Universal Approximation	56
4.3.1	Tent Functions	57
4.3.2	Piecewise-Linear Interpolation	58
4.3.3	Uniform Continuity and the Approximation Bound	58
4.3.4	The Universal Approximation Theorem	59
5	MNIST	61
5.1	Data and Network Setup	61
5.2	Mini-Batch Stochastic Gradient Descent	62
5.3	Results	63
A	Implementation	65
A.1	Core Matrix Class	65
A.2	LU Decomposition	67
A.3	QR via Gram-Schmidt	70
A.4	Jacobi Eigensolver	72
A.5	SVD	75
A.6	Numerical Gradient Checker	78
A.7	Gradient Descent	79
A.8	KKT Solver	81
A.9	Categorical MLE	84
A.10	Forward Pass	87
A.11	Backward Pass	93
A.12	MNIST Loader	95
A.13	MNIST Training	98
	Bibliography	100

Preface

This expository report develops the mathematics of machine learning from first principles, building the theory and algorithms underlying neural networks from the ground up, implementing the resulting network in C++, and training it on the MNIST handwritten digit dataset to confirm the mathematics produces a working classifier, not limiting itself to theory. Each chapter depends only on results established earlier in the paper; nothing is assumed without proof.

Every algorithm presented is implemented from scratch in C++, with no external libraries, so that the mathematics developed here can be seen to work in practice, not proved to. Where a result has a corresponding implementation, this is noted by a short Computational Remark connecting the two and explaining any design decisions the implementation required.

This report was created relying on material from many different sources and is certainly not intended to be original work, but an exposition on what machine learning is and how it works. The paper is organised into five chapters, followed by an appendix containing the full C++ implementation and a bibliography. It is strongly encouraged to visit the Github repository ([Math of ML](#)) to view the full extent of the implementation.

Chapter 1

Linear Algebra

1.1 Vector Spaces and Linear Operators

This section introduces vector spaces, subspaces, linear independence and linear operators: the basic algebraic structures on which every subsequent chapter depends.

Beyond their intrinsic role in linear algebra, these structures are the language in which the rest of this paper is written, as they are essential for the development of the mathematics of neural networks.

These structures are represented computationally throughout the project by a C++ `Matrix` class implementing construction, basic operations and transposition, given in Appendix A.1. Using a header `.hpp` format this `Matrix` object will be imported and used as a structure to help implement the computational algorithms necessary to achieve this project's goal.

1.1.1 Vector Spaces

Let V be a non-empty set with operation '+', called vector addition. Assume that the pair $(V, +)$ forms an Abelian group i.e. it satisfies the following Axioms:

- (A₀) For all $u, v \in V$, $u + v \in V$, i.e. V is closed under vector addition.
- (A₁) For all $u, v, w \in V$, $u + (v + w) = (u + v) + w$, that is, vector addition is associative.
- (A₂) For all $u, v \in V$, $u + v = v + u$, i.e. vector addition is commutative.
- (A₃) There is a zero element $0_V \in V$ such that $v + 0_V = v$ for all $v \in V$.
- (A₄) Any $v \in V$ has a negative (or additive inverse) $-v \in V$ such that $-v + v = 0_V$.

In addition, let $\mathbb{F}$ be a field (e.g. $\mathbb{F} = \mathbb{R}$) and assume that $\mathbb{F}$ acts on V via a mapping $\cdot$ called scalar multiplication and is required to satisfy the following properties.

- (M₁) For all $\mu \in \mathbb{F}$ and $u, v \in V$, $\mu \cdot (u + v) = \mu \cdot u + \mu \cdot v$, that is, scalar multiplication is distributive over vector addition.
- (M₂) For all $\mu, \nu \in \mathbb{F}$ and $v \in V$, $(\mu + \nu) \cdot v = \mu \cdot v + \nu \cdot v$, that is, scalar multiplication is distributive over scalar addition.

(M₃) For all $\mu, \nu \in \mathbb{F}$ and $v \in V$, $\mu \cdot (\nu \cdot v) = (\mu\nu) \cdot v$, that is, scalar multiplication is associative.

(M₄) For $1 \in \mathbb{F}$, $1 \cdot v = v$ for all $v \in V$.

Definition 1.1 (Vector Space). A vector space over a field $\mathbb{F}$ is a set V along with addition and scalar multiplication on V , i.e. $(V, +, \cdot)$ that obeys all axioms above. Elements of V are vectors and elements of $\mathbb{F}$ are scalars.

The element $0_v \in V$ is called the zero-vector, while the element $0 \in \mathbb{F}$ is called the zero-scalar. Further, subtraction in a vector space is defined as $u + (-v) \forall u, v \in V$. That is, the sum of an element with an inverse element yields a subtraction.

Proposition 1.1. Let V be a $\mathbb{F}$ -vector space

- i. For all $\mu \in \mathbb{F}$, $\mu \cdot 0_v = 0_v$ and for all $v \in V$, $0 \cdot v = 0_v$
- ii. If $\mu \cdot v = 0_v$ and $\mathbb{F} \ni \mu \neq 0$, then $V \ni v = 0_v$.
- iii. For all $v \in V$, $(-1) \cdot v = -v$

Proof. This can be proven using the axioms (A₀)-(A₄) and (M₁)-(M₄).

- i. From axioms (A₃) and (M₁), $\mu \cdot 0_v = \mu \cdot (0_v + 0_v) = \mu \cdot 0_v + \mu \cdot 0_v$, and hence, by (A₃), $\mu \cdot 0_v = 0_v$. Likewise, from (M₂), $0 \cdot v = (0 + 0) \cdot v = 0 \cdot v + 0 \cdot v$, and hence, by (A₃), $0 \cdot v = 0_v$.
- ii. As $\mathbb{F} \ni \mu \neq 0$, there exists $\mu^{-1} \in \mathbb{F}$ with $\mu^{-1}\mu = 1$. From axioms (M₄) and (M₃), $v = 1 \cdot v = (\mu^{-1}\mu) \cdot v = \mu^{-1} \cdot (\mu \cdot v)$. Thus, if $\mu \cdot v = 0_v$ we find that $v = \mu^{-1} \cdot 0_v$, and by (i), $v = 0_v$.
- iii. From (i) and axioms (M₂) and (M₄), $0_v = 0 \cdot v = (1 + (-1)) \cdot v = 1 \cdot v + (-1) \cdot v = v + (-1) \cdot v$, and hence $(-1) \cdot v = -v$.

Q.E.D.

Example 1.1 (Matrices). The set of all $(m \times n)$ -matrices with entries in $\mathbb{F}$, $\text{Mat}(m, n, \mathbb{F})$, is a $\mathbb{F}$ -vector space with the usual definitions of adding matrices and multiplying by constants. For $m = n$ we write $\text{Mat}(n, \mathbb{F})$.

1.1.2 Vector Subspaces

A non-empty subset $U \subseteq V$ of a vector space need not be a vector space itself, since in general it is not closed under vector addition/scalar multiplication. For example, for any $u_1, u_2 \in U$ it is not necessarily true that $u_1 + u_2 \in U$.

Definition 1.2 (Vector Subspace). Let V be a $\mathbb{F}$ -vector space. A vector subspace of V is a non-empty subset U of V such that U is a $\mathbb{F}$ -vector space under the same operations as in V .

If U is a subset of a vector space V , then axioms (A₁), (A₂) and (M₁)-(M₄) automatically hold in U because they hold throughout V .

Definition 1.3 (Linear Combination). Let V be a $\mathbb{F}$ -vector space. A linear combination of the vectors $v_1, \dots, v_m \in V$ is a vector of the form

$$v = \sum_{i=1}^m \mu_i v_i \in V \quad \text{with} \quad \mu_1, \dots, \mu_m \in \mathbb{F}$$

Proposition 1.2 (Subspace Test). Let V be a $\mathbb{F}$ -vector space and let U be a non-empty subset of V . Then U is a vector subspace of V if and only if

$$\mu_1 u_1 + \mu_2 u_2 \in U \quad \forall u_1, u_2 \in U \text{ and scalars } \mu_1, \mu_2 \in \mathbb{F}$$

Thus, to decide whether U is a vector subspace we do not have to check all the axioms. It is enough to show that U is closed under linear combinations and that it is non-empty.

Non-emptiness can be shown by demonstrating that the zero-vector is contained in U (note also (A_3)).

1.1.3 Linear Independence

Definition 1.4 (Span). Let V be a $\mathbb{F}$ -vector space and $v_1, \dots, v_m \in V$. The span of the set $\{v_1, \dots, v_m\}$ is the set of all possible linear combinations of $v_1, \dots, v_m$, that is,

$$\text{span}_{\mathbb{F}}\{v_1, \dots, v_m\} := \left\{ \sum_{i=1}^m \mu_i v_i \mid \mu_1, \dots, \mu_m \in \mathbb{F} \right\}$$

If $V = \text{span}_{\mathbb{F}}\{v_1, \dots, v_m\}$, then $\{v_1, \dots, v_m\}$ is said to be a spanning set for the vector space V .

We also note that the zero-vector $0 \in V$ is always contained in the span of any vectors $v_1, \dots, v_m \in V$ since by Proposition 1.1, we have $0 = 0v_1 + \dots + 0v_m \in \text{span}_{\mathbb{F}}\{v_1, \dots, v_m\}$

Example 1.2 (Span in $\mathbb{R}^3$). In $\mathbb{R}^3$,

$$\text{span}_{\mathbb{R}}\{(1, 0, 0)^T, (0, 1, 0)^T\} = \{(x, y, 0)^T \in \mathbb{R}^3 \mid x, y \in \mathbb{R}\}$$

describes the (x, y) -plane, which is a special instance of a two-dimensional plane in $\mathbb{R}^3$ through the “origin” $0 \in \mathbb{R}^3$.

Now, let V be a $\mathbb{F}$ -vector space and let $v_1, \dots, v_m \in V$.

Definition 1.5 (Linear Dependence). The vectors $v_1, \dots, v_m$ are said to be linearly dependent if and only if at least one of the vectors is a linear combination of the others.

Definition 1.6 (Linear Independence). The vectors $v_1, \dots, v_m$ are said to be linearly independent if and only if they are not linearly dependent.

Remark 1.1. If a set $\{v_1, \dots, v_m\}$ contains the zero-vector $0_V \in V$, then the set is linearly dependent since $\mu \cdot 0_V = 0_V$ for all $\mu \in \mathbb{F}$ by Proposition 1.1.

Proposition 1.3 (Extension Lemma). Let $\{v_1, \dots, v_m\}$ be a linearly independent set of vectors in a $\mathbb{F}$ -vector space V . Then, for any $v \in V$, $v \in \text{span}_{\mathbb{F}}\{v_1, \dots, v_m\} \iff \{v, v_1, \dots, v_m\}$ is a linearly dependent set of vectors.

Proof. ($\Rightarrow$) Suppose $v \in \text{span}_{\mathbb{F}}\{v_1, \dots, v_m\}$. Then $v = \sum_{i=1}^m \mu_i v_i$. By Definition 1.5, v is linearly dependent.

($\Leftarrow$) Suppose $\{v, v_1, \dots, v_m\}$ is linearly dependent.

Case 1: v is a linear combination of $v_1, \dots, v_m$. Then by the definition of span , $v \in \text{span}_{\mathbb{F}}\{v_1, \dots, v_m\}$.

Case 2: some v_i is a linear combination of the other vectors in $\{v, v_1, \dots, v_m\}$, i.e. $v_i = av + \sum_{j \neq i} b_j v_j$. If the coefficient on v were zero, then v_i becomes just a combination of the other v_j 's, contradicting linear independence. Hence that coefficient is non-zero, and solving for v gives $v = a^{-1}v_i - \sum_{j \neq i} a^{-1}b_j v_j$, hence $v \in \text{span}_{\mathbb{F}}\{v_1, \dots, v_m\}$. Q.E.D.

By contraposition, we now have that $V \ni v \notin \text{span}_{\mathbb{F}}\{v_1, \dots, v_m\}$ if and only if $\{v, v_1, \dots, v_m\}$ forms a linearly independent set of vectors.

Definition 1.7 (Dimension). A $\mathbb{F}$ -vector space V is said to be n -dimensional if and only if there is a linearly independent set of n vectors and every set consisting of $n + 1$ vectors is linearly dependent. $\dim_{\mathbb{F}}(V) = n$ expresses the dimension of V . Furthermore, a vector space is said to be of infinite dimension if and only if there is a linearly independent set of vectors of arbitrarily large order.

1.1.4 Bases of Vector Spaces

We now seek to represent any vector $v \in V$ as a linear combination of a "minimal" set of vectors, also called a basis.

Definition 1.8 (Basis). Let V be an n -dimensional $\mathbb{F}$ -vector space. Any linear independent set of n vectors in V is called a *basis* of V .

Corollary 1.1 (Basis Extension). *Let V be an n -dimensional $\mathbb{F}$ -vector space, and let $\{v_1, \dots, v_k\}$ be a linearly independent set in V with $k \leq n$. Then there exist vectors $v_{k+1}, \dots, v_n \in V$ such that $\{v_1, \dots, v_k, v_{k+1}, \dots, v_n\}$ is a basis of V .*

Proof. We proceed by induction on j , for $j = k, k + 1, \dots, n$, constructing at each stage a linearly independent set $\{v_1, \dots, v_j\}$.

Base case ($j = k$). The set $\{v_1, \dots, v_k\}$ is linearly independent by hypothesis.

Inductive step. Suppose $\{v_1, \dots, v_j\}$ is linearly independent for some $k \leq j < n$. If $\{v_1, \dots, v_j\}$ spans V , then by Proposition 1.4 it is a basis, and since $\dim_{\mathbb{F}}(V) = n$, this forces $j = n$ — contradicting $j < n$. So $\{v_1, \dots, v_j\}$ does not span V , and there exists $v_{j+1} \in V \setminus \text{span}_{\mathbb{F}}\{v_1, \dots, v_j\}$. By the contrapositive of Proposition 1.3, $\{v_1, \dots, v_j, v_{j+1}\}$ is linearly independent, completing the inductive step.

By induction, $\{v_1, \dots, v_n\}$ is linearly independent. Since $\dim_{\mathbb{F}}(V) = n$, this set of n linearly independent vectors is a basis of V by Definition 1.8. This completes the induction. Q.E.D.

Proposition 1.4. *Let V be a $\mathbb{F}$ -vector space. Any linearly independent spanning set for V is a basis of V and vice versa.*

Proof. ($\Rightarrow$) Let $\{v_1, \dots, v_n\}$ be a spanning set for V . Hence, any $v \in V$ lies in the span of $\{v_1, \dots, v_n\}$, that is, $v \in \text{span}_{\mathbb{F}}\{v_1, \dots, v_n\}$. If, in addition, $\{v_1, \dots, v_n\}$ is linearly independent, then Proposition 1.3 implies that $\{v, v_1, \dots, v_n\}$ is a linearly dependent set. Thus, $\dim_{\mathbb{F}}(V) = n$ and so $\{v_1, \dots, v_n\}$ is a basis.

($\Leftarrow$) Conversely, let $n = \dim_{\mathbb{F}}(V)$ and let $\{v_1, \dots, v_n\}$ be a basis of V . Then, for any $v \in V$, the set $\{v, v_1, \dots, v_n\}$ is linearly dependent by the definition of a basis. Finally, Proposition 1.3 implies that $v \in \text{span}_{\mathbb{F}}\{v_1, \dots, v_n\}$, which concludes the proof. Q.E.D.

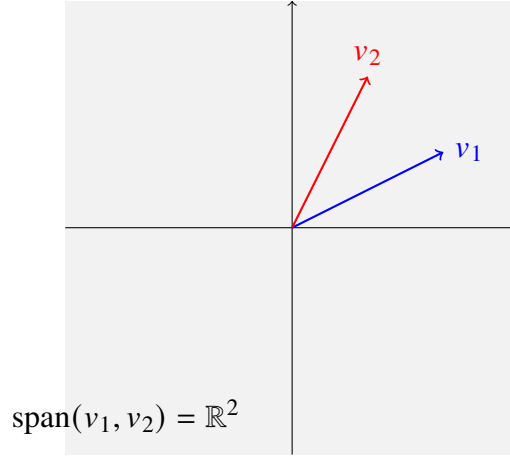


Figure 1.1: Two linearly independent vectors $v_1, v_2 \in \mathbb{R}^2$ span the entire plane: every point in $\mathbb{R}^2$ is a linear combination of v_1 and v_2 .

Example 1.3 (Standard Basis for $\text{Mat}(m, n, \mathbb{F})$). The vector space $\text{Mat}(m, n, \mathbb{F})$ is spanned by the matrices E_{ij} having 1 in the (i, j) position and zero elsewhere. The set of all such matrices is easily shown to be linearly independent. It is the standard basis for $\text{Mat}(m, n, \mathbb{F})$ and $\dim_{\mathbb{F}}(\text{Mat}(m, n, \mathbb{F})) = mn$.

Theorem 1.1 (Uniqueness of Representation). *Let V be a $\mathbb{F}$ -vector space equipped with a basis. Any vector $v \in V$ can be represented uniquely as a linear combination of the given basis vectors with coefficients in $\mathbb{F}$.*

Proof. Let $n = \dim_{\mathbb{F}}(V)$ and let $\{v_1, \dots, v_n\}$ be a basis of V . By virtue of Proposition 1.4, $V = \text{span}_{\mathbb{F}}\{v_1, \dots, v_n\}$, and thus any $v \in V$ can be written as $v = \sum_{i=1}^n \mu_i v_i$ for some $\mu_1, \dots, \mu_n \in \mathbb{F}$. To show uniqueness, let $v = \sum_{i=1}^n \mu_i v_i$ and $v = \sum_{i=1}^n \nu_i v_i$ be any two representations of $v \in V$ in terms of the basis vectors. Then, equating both expressions, we find that $\sum_{i=1}^n (\mu_i - \nu_i) v_i = 0$. Since the basis vectors are linearly independent, all the coefficients must vanish, that is, $\mu_i = \nu_i$ for all $i \in \{1, \dots, n\}$. Q.E.D.

1.1.5 Linear Operators

Definition 1.9 (Linear Operator). Let V and W be two $\mathbb{F}$ -vector spaces. A linear operator is a mapping $T : V \rightarrow W$ with $T(\mu u + \nu v) = \mu T(u) + \nu T(v)$ for all $\mu, \nu \in \mathbb{F}$ and $u, v \in V$. Here, V is called the domain of T , W the codomain, and $T(v)$ the image of $v \in V$ under T .

Linear operators are also called linear transformations, linear maps or homomorphisms. The set of all linear operators from V to W is denoted by $\text{Hom}_{\mathbb{F}}(V, W)$. For any $T_1, T_2 \in \text{Hom}_{\mathbb{F}}(V, W)$, we define

$$(\mu_1 T_1 + \mu_2 T_2)(v) := \mu_1 T_1(v) + \mu_2 T_2(v) \quad \text{for all } v \in V \text{ and } \mu_1, \mu_2 \in \mathbb{F}$$

Example 1.4 (Transposition as a Linear Operator). Let $T : \text{Mat}(m, n, \mathbb{F}) \rightarrow \text{Mat}(n, m, \mathbb{F})$ be given by transposition $T : A \mapsto A^T$. For any $A, B \in \text{Mat}(m, n, \mathbb{F})$ and $a, b \in \mathbb{F}$, we have $T(aA + bB) = (aA + bB)^T = aA^T + bB^T = aT(A) + bT(B)$, so T is a linear operator.

Proposition 1.5 (Matrix Representation of Linear Maps). *Let $T : \mathbb{F}^n \rightarrow \mathbb{F}^m$ be a linear map. Then there exists a unique matrix $A \in \text{Mat}(m, n, \mathbb{F})$ such that $T(x) = Ax$ for all $x \in \mathbb{F}^n$, where the j -th column of A is $T(e_j)$ and $\{e_1, \dots, e_n\}$ is the standard basis of $\mathbb{F}^n$.*

Proof. For any $x \in \mathbb{F}^n$, we write $x = \sum_{j=1}^n x_j e_j$. By the linearity of T , we have $T(x) = T(\sum_{j=1}^n x_j e_j) = \sum_{j=1}^n x_j T(e_j)$. Since matrix-vector multiplication Ax is defined as the linear combination of the columns of A weighted by the components of x , and the j -th column of A is defined as $T(e_j)$, it follows that $T(x) = Ax$.

For uniqueness, suppose $A' \in \text{Mat}(m, n, \mathbb{F})$ also satisfies $T(x) = A'x$ for all $x \in \mathbb{F}^n$. For each j , taking $x = e_j$ gives $Ae_j = T(e_j) = A'e_j$; since Me_j is the j -th column of M , A and A' have the same columns, so $A = A'$. Q.E.D.

Corollary 1.2 (Composition and Matrix Multiplication). *Let $T : \mathbb{F}^n \rightarrow \mathbb{F}^m$ have matrix representation A and $S : \mathbb{F}^m \rightarrow \mathbb{F}^k$ have matrix representation B . Then the composition $S \circ T : \mathbb{F}^n \rightarrow \mathbb{F}^k$ has the matrix representation BA .*

Proof. For any $x \in \mathbb{F}^n$, applying Proposition 1.5 directly yields $(S \circ T)(x) = S(T(x)) = S(Ax)$. Applying the proposition a second time to the action of S on the vector $Ax \in \mathbb{F}^m$ gives $S(Ax) = B(Ax)$. By the associativity of matrix multiplication, $B(Ax) = (BA)x$. Thus, the composition $(S \circ T)$ is represented by the matrix product BA . Q.E.D.

Definition 1.10 (Kernel and Image). Let $T \in \text{Hom}_{\mathbb{F}}(V, W)$. The **kernel** of T is the set $\ker(T) := \{v \in V \mid T(v) = 0\} \subseteq V$. The **image** of T is the set $\text{im}(T) := \{T(v) \in W \mid v \in V\} \subseteq W$.

Remark 1.2. Let $T : V \rightarrow W$ be a linear map between $\mathbb{F}$ -vector spaces. Then:

- i. $\ker(T) = \{v \in V : T(v) = 0_W\}$ is a subspace of V .
- ii. $\text{im}(T) = \{T(v) : v \in V\} \subseteq W$ is a subspace of W .

Theorem 1.2 (Rank-Nullity Theorem). *For $T \in \text{Hom}_{\mathbb{F}}(V, W)$, we have*

$$\dim_{\mathbb{F}}(\text{im}(T)) + \dim_{\mathbb{F}}(\ker(T)) = \dim_{\mathbb{F}}(V)$$

Proof. Set $n := \dim_{\mathbb{F}}(V)$ and $d := \dim_{\mathbb{F}}(\ker(T))$. Let $\{v_1, \dots, v_d\}$ be a basis of $\ker(T)$. Extend this to a basis (Corollary 1.1) $\{v_1, \dots, v_d, u_{d+1}, \dots, u_n\}$ of V .

We show that $\{T(u_{d+1}), \dots, T(u_n)\}$ is a basis of $\text{im}(T)$. This suffices, since it gives

$$\dim_{\mathbb{F}}(\text{im}(T)) = n - d = \dim_{\mathbb{F}}(V) - \dim_{\mathbb{F}}(\ker(T))$$

Linear independence. Suppose

$$0 = \sum_{i=1}^{n-d} \mu_i T(u_{d+i}) = T\left(\sum_{i=1}^{n-d} \mu_i u_{d+i}\right)$$

Then $\sum_{i=1}^{n-d} \mu_i u_{d+i} \in \ker(T)$, so it can be written in terms of the basis $\{v_1, \dots, v_d\}$:

$$\sum_{i=1}^{n-d} \mu_i u_{d+i} = \sum_{j=1}^d v_j v_j \iff \sum_{i=1}^{n-d} \mu_i u_{d+i} - \sum_{j=1}^d v_j v_j = 0$$

Since $\{v_1, \dots, v_d, u_{d+1}, \dots, u_n\}$ is a basis of V , these vectors are linearly independent. Hence all coefficients μ_i and v_j vanish. In particular $\mu_i = 0$ for all i , so $\{T(u_{d+1}), \dots, T(u_n)\}$ is linearly independent.

Spanning. Let $v \in V$. Write

$$v = \sum_{j=1}^d \alpha_j v_j + \sum_{i=1}^{n-d} \beta_i u_{d+i}$$

Applying T and using $T(v_j) = 0$ for all j (since $v_j \in \ker(T)$):

$$T(v) = \sum_{i=1}^{n-d} \beta_i T(u_{d+i})$$

So every element of $\text{im}(T)$ is a linear combination of $\{T(u_{d+1}), \dots, T(u_n)\}$; these vectors span $\text{im}(T)$.

Hence $\{T(u_{d+1}), \dots, T(u_n)\}$ is a basis of $\text{im}(T)$, completing the proof. Q.E.D.

1.2 The LU Decomposition

This section develops the LU decomposition, a factorisation of a matrix into lower and upper-triangular components, used to solve linear systems $Ax = b$ efficiently once computed.

Beyond its role in solving linear systems directly, this factorisation demonstrates a decompose once, then solve cheaply and repeatedly pattern. This is the same pattern that motivates the QR decomposition.

The algorithm is implemented from scratch in C++ as `lu_solver.cpp` (Appendix A.2), including partial pivoting for numerical stability.

1.2.1 Gaussian Elimination

Consider a system of m linear equations in n unknowns $x_1, x_2, \dots, x_n$, written in matrix form $Ax = b$, or explicitly, as:

$$\begin{bmatrix} A_{11} & A_{12} & \dots & A_{1n} \\ A_{21} & A_{22} & \dots & A_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ A_{m1} & A_{m2} & \dots & A_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

The key idea of Gaussian elimination is to transform a system of linear equations into a simpler system, while keeping fixed the set of solutions. This leads to the following definition.

Definition 1.11 (Elementary Row Operations). The following operations are called elementary row operations on a matrix A :

- i. Interchanging two rows.
- ii. Multiplying a row by a non-zero scalar.
- iii. Adding a multiple of one row to another row.

A matrix A is row-equivalent to another matrix A' if and only if it can be transformed to A' via elementary row operations.

Gaussian elimination is the repeated application of elementary row operations, mainly used to zero out entries below each pivot.

Definition 1.12 (Pivot). The leading/first non-zero entry in a row.

1.2.2 The LU Factorisation

It is often convenient to decompose a matrix A into a product of two other matrices. This is called a LU decomposition. We restrict our attention to square, invertible matrices $A \in \text{Mat}(n, \mathbb{F})$, since this is the setting in which the system $Ax = b$ has a unique solution.

Definition 1.13 (LU Decomposition). Let $A \in \text{Mat}(n, \mathbb{F})$. An LU decomposition of A is a factorisation

$$A = LU$$

where $L \in \text{Mat}(n, \mathbb{F})$ is unit lower triangular (that is, lower triangular with 1's on the diagonal) and $U \in \text{Mat}(n, \mathbb{F})$ is upper triangular.

Definition 1.14 (Leading Principal Minor). Let $A \in \text{Mat}(n, \mathbb{F})$. For $k = 1, \dots, n$, the k -th leading principal minor of A , denoted Δ_k , is the determinant of the $k \times k$ submatrix formed by the first k rows and first k columns of A , that is,

$$\Delta_k := \det \begin{bmatrix} A_{11} & \dots & A_{1k} \\ \vdots & \ddots & \vdots \\ A_{k1} & \dots & A_{kk} \end{bmatrix}$$

Theorem 1.3 (Existence and Uniqueness of the LU Decomposition). Let $A \in \text{Mat}(n, \mathbb{F})$ be invertible. Then A admits a unique LU decomposition if and only if every leading principal minor of A is non-zero, that is, $\Delta_k \neq 0$ for all $k = 1, \dots, n$.

Proof. A full treatment of this proof is given in (Golub & Van Loan 2013, §3.2). Q.E.D.

Example 1.5 (A Zero Pivot). Consider the matrix

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

This matrix is invertible, since $\det(A) = -1 \neq 0$. However, its leading principal minors are

$$\Delta_1 = A_{11} = 0, \quad \Delta_2 = \det(A) = -1,$$

and since $\Delta_1 = 0$, by Theorem 1.3 there does not exist a unique LU decomposition for A .

Definition 1.15 (Permutation Matrix). A matrix $P \in \text{Mat}(n, \mathbb{F})$ is called a *permutation matrix* if it is obtained from the identity matrix $\mathbb{I}$ by permuting its rows. That is, there exists a permutation $\sigma \in S_n$ such that

$$P_{ij} = \begin{cases} 1 & \text{if } j = \sigma(i) \\ 0 & \text{otherwise} \end{cases}$$

Equivalently, each row and each column of P contains exactly one entry equal to 1, with all other entries 0.

To extend the definition of a LU decomposition to any invertible square matrix, we use the permutation matrix P that acts on A . This gives the following extension of Theorem 1.3.

Theorem 1.4 (Existence of the $PA = LU$ Decomposition). Let $A \in \text{Mat}(n, \mathbb{F})$ be invertible. Then there exists a permutation matrix $P \in \text{Mat}(n, \mathbb{F})$ such that PA has all leading principal minors non-zero, and hence, by Theorem 1.3, a unique decomposition

$$PA = LU,$$

where L is unit lower triangular and U is upper triangular.

Proof. A full treatment of this proof is given in (Golub & Van Loan 2013, §3.4). Q.E.D.

1.2.3 Solving the Linear System

We now seek to solve the system of linear equations $Ax = b$ via a LU decomposition.

Proposition 1.6 (Solving $Ax = b$ via $PA = LU$). *Let $A \in \text{Mat}(n, \mathbb{F})$ be invertible, let P be a permutation matrix and L, U unit-lower and upper triangular matrices respectively such that $PA = LU$ (Theorem 1.4), and let $b \in \mathbb{F}^n$. Then the system*

$$Ax = b$$

has a unique solution x , obtained by first solving $Ly = Pb$ for y by forward substitution, then solving $Ux = y$ for x by back substitution.

Proof. $Ax = b$ is equivalent to

$$PAx = Pb$$

Substituting $PA = LU$,

$$LUx = Pb$$

Set $y := Ux$. Then the system splits into two triangular systems:

$$Ly = Pb \quad Ux = y$$

Since L is unit lower triangular, $L_{kk} = 1 \neq 0$ for all k , so $Ly = Pb$ has a unique solution obtained by forward substitution:

$$y_k = (Pb)_k - \sum_{j=1}^{k-1} L_{kj}y_j \quad k = 1, \dots, n$$

Since A is invertible and P is invertible, $U = L^{-1}PA$ is invertible, hence $U_{kk} \neq 0$ for all k (an upper triangular matrix is invertible if and only if all diagonal entries are non-zero). So $Ux = y$ has a unique solution obtained by back substitution:

$$x_k = \frac{1}{U_{kk}} \left(y_k - \sum_{j=k+1}^n U_{kj}x_j \right) \quad k = n, n-1, \dots, 1$$

Existence and uniqueness of y and then x give existence and uniqueness of a solution to $Ax = b$. Q.E.D.

Remark 1.3 (The $PA = LU$ Solution Method). Theorem 1.4 and Proposition 1.6 combine into a single procedure for solving $Ax = b$:

- i. **Factor once:** compute P, L, U such that $PA = LU$ (Theorem 1.4). This step costs $O(n^3)$ operations.
- ii. **Solve twice:** for any $b \in \mathbb{F}^n$, solve $Ly = Pb$ by forward substitution, then $Ux = y$ by back substitution (Proposition 1.6). Each substitution costs $O(n^2)$.

The point of separating factorisation from substitution is that the $O(n^3)$ cost is paid only once per matrix A , while each additional right-hand side b costs only $O(n^2)$. This is the practical motivation for the $PA = LU$ approach over recomputing a full solve (e.g. via Cramer's rule or a fresh elimination) for every new b .

Example 1.6 (Solving $Ax = b$ via $PA = LU$). Let

$$A = \begin{bmatrix} 1 & 1 \\ 2 & 3 \end{bmatrix} \quad b = \begin{bmatrix} 2 \\ 5 \end{bmatrix}$$

Here $\Delta_1 = 1 \neq 0$, so A admits an LU decomposition directly by Theorem 1.3, with $P = \mathbb{I}$. But applying partial pivoting (Definition 1.16) at step $k = 1$: the active column is $(A_{11}, A_{21}) = (1, 2)$, and $|A_{21}| = 2 > |A_{11}| = 1$, so the pivot row is $r = 2$, and rows 1 and 2 are swapped before elimination proceeds.

Permutation. Swapping rows 1 and 2 gives $\text{perm} = [2, 1]$ (in the 1-indexed convention of §1.2) and

$$P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \quad PA = \begin{bmatrix} 2 & 3 \\ 1 & 1 \end{bmatrix}$$

Factorisation. Eliminate the $(2, 1)$ entry of PA using multiplier $m_{21} = 1/2$:

$$R_2 \leftarrow R_2 - \frac{1}{2}R_1 = \begin{bmatrix} 1 & 1 \end{bmatrix} - \frac{1}{2} \begin{bmatrix} 2 & 3 \end{bmatrix} = \begin{bmatrix} 0 & -\frac{1}{2} \end{bmatrix}$$

This gives

$$L = \begin{bmatrix} 1 & 0 \\ \frac{1}{2} & 1 \end{bmatrix} \quad U = \begin{bmatrix} 2 & 3 \\ 0 & -\frac{1}{2} \end{bmatrix}$$

Check: $LU = \begin{bmatrix} 1 & 0 \\ \frac{1}{2} & 1 \end{bmatrix} \begin{bmatrix} 2 & 3 \\ 0 & -\frac{1}{2} \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 1 & 1 \end{bmatrix} = PA$. Correct.

Step 1 (forward substitution): solve $Ly = Pb$. Since $\text{perm} = [2, 1]$, $Pb = \begin{bmatrix} 5 \\ 2 \end{bmatrix}$.

$$y_1 = 5, \quad y_2 = 2 - \frac{1}{2}(5) = -\frac{1}{2}$$

So $y = \begin{bmatrix} 5 \\ -\frac{1}{2} \end{bmatrix}$.

Step 2 (back substitution): solve $Ux = y$.

$$x_2 = \frac{-\frac{1}{2}}{-\frac{1}{2}} = 1, \quad x_1 = \frac{1}{2}(5 - 3(1)) = 1$$

So $x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

Check: $Ax = \begin{bmatrix} 1 & 1 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 5 \end{bmatrix} = b$. Correct.

1.2.4 Partial Pivoting

The example above shows that the leading-principal-minor condition in Theorem 1.3 can fail in two ways: exactly (Example 1.5), or nearly, via a small pivot that leaves the algorithm formally intact but numerically unreliable. *Partial pivoting* is the standard modification to Gaussian elimination that addresses both.

Definition 1.16 (Partial Pivoting). Let $A \in \text{Mat}(n, \mathbb{F})$. Set $A^{(1)} := A$, and for $k = 2, \dots, n$, let $A^{(k)}$ denote the matrix obtained after the eliminations and row swaps of steps $1, \dots, k - 1$

have been applied. At step k of Gaussian elimination, define the *active column* to be the entries $A_{kk}^{(k)}, A_{k+1,k}^{(k)}, \dots, A_{nk}^{(k)}$, i.e. column k restricted to rows $k, \dots, n$. Let

$$r := \arg \max_{k \leq i \leq n} |A_{ik}^{(k)}|$$

be a row index attaining the largest absolute value in the active column. Partial pivoting swaps row r with row k before proceeding with elimination as normal. Each such swap is a transposition; the composition of these transpositions, in the order performed across steps $k = 1, \dots, n-1$, defines the permutation $\sigma \in S_n$ of Definition 1.15, and hence the permutation matrix P .

With this modification, the substantive question is whether the procedure can always be carried out. That is, whether a valid pivot always exists at every step, for any invertible A .

Proposition 1.7 (Partial Pivoting on an Invertible Matrix). *Let $A \in \text{Mat}(n, \mathbb{F})$ be invertible. Then at every step $k = 1, \dots, n-1$ of Gaussian elimination with partial pivoting, the active column contains at least one non-zero entry, so a valid pivot row r always exists.*

Proof. A full treatment of this proof is given in (Golub & Van Loan 2013, §3.4). Q.E.D.

But why should one bother with partial pivoting? After all, Theorem 1.4 already guarantees some valid P exists for any invertible A , so why does the specific pivoting strategy matter?

Proposition 1.8 (Bounded Multipliers Under Partial Pivoting). *Suppose partial pivoting is used at every step of Gaussian elimination. Then every multiplier satisfies*

$$|m_{ik}| \leq 1 \quad \text{for all } i > k$$

Proof. Fix step k . By construction, the pivot $A_{kk}^{(k)}$ (after the swap) satisfies

$$|A_{kk}^{(k)}| \geq |A_{ik}^{(k)}| \quad \text{for all } i = k, \dots, n$$

since $A_{kk}^{(k)}$ was chosen to be the entry of largest absolute value among the active column. In particular, for every $i > k$,

$$|m_{ik}| = \left| \frac{A_{ik}^{(k)}}{A_{kk}^{(k)}} \right| \leq 1$$

Q.E.D.

Computational Remark 1.1. `factoriseLU()` (`lu_solver.cpp`) makes two representational choices worth justifying against the mathematics of this section.

- i. **In-place storage of the multipliers.** After eliminating column k , every entry below the diagonal in that column is exactly zero. Since this value is already known, it carries no information worth keeping, so the position is free to reuse. The code writes the multiplier m_{ik} there instead: L 's diagonal is always 1 (Definition 1.13), so nothing about L is lost by leaving it implicit, and the strictly-lower entries of L are exactly these multipliers. So A is overwritten in place to hold both U and the non-trivial part of L , with no extra storage needed.

- ii. **The permutation as a vector, not a matrix.** The code applies P to b via `bPrime(i) = b(perm[i], 0)`, a direct reindexing. This matches how P is actually used in the proof of Proposition 1.6: the argument never needs more from P than reordering b 's rows. So representing P as the vector `perm`, rather than an $n \times n$ matrix, is a faithful, more efficient match for what the proof uses.

Both choices can be checked directly against the worked example (Example 1.6): running `lu_solver.cpp` on that example's A and b reproduces $x = (1, 1)^\top$.

1.3 Inner Product Spaces and QR Decompositions

This section develops inner product spaces, orthogonality and the QR decomposition, providing the tools to construct orthonormal bases and to solve $Ax = b$ via orthogonal factorisation.

Beyond providing a second, more numerically stable route to solving linear systems, orthogonality is the structural idea that the spectral theorem in the next section depends on directly.

The algorithm is implemented from scratch in C++ as `qr_gram_schmidt.cpp` (Appendix A.3), computing the QR decomposition via the Gram-Schmidt process.

1.3.1 Inner Product Spaces

In Section 1.1, we defined a vector space V , but a vector space alone carries no notion of length or angle between vectors. Equipping V with such a notion is the first step toward orthogonality, which in turn is the basis of the QR decomposition developed later in this section.

Recall the dot product for $\mathbb{R}^n$:

$$x \cdot y = \sum_{i=1}^n x_i y_i$$

This immediately gives a notion of length via $\|x\| := \sqrt{x \cdot x}$.

The dot product on $\mathbb{R}^n$ has three properties worth isolating: it is symmetric in x and y , linear in each argument and positive-definite ($x \cdot x \geq 0$, with equality iff $x = 0$). Abstracting exactly these properties to an arbitrary vector space yields the definition of an inner product.

Throughout this section, V will denote a real vector space, i.e. $\mathbb{F} = \mathbb{R}$.

Definition 1.17 (Inner Product / Inner Product Space). An inner product on a real vector space V is a map $\langle \cdot, \cdot \rangle : V \times V \rightarrow \mathbb{R}$ that satisfies the following properties for all $u, v, w \in V$ and all $\lambda \in \mathbb{R}$:

(IP1) **Symmetry:** $\langle u, v \rangle = \langle v, u \rangle$

(IP2) **Linearity in the first argument:** $\langle u + v, w \rangle = \langle u, w \rangle + \langle v, w \rangle$ and $\langle \lambda u, v \rangle = \lambda \langle u, v \rangle$

(IP3) **Positive-definiteness:** $\langle v, v \rangle \geq 0$ for all $v \in V$, with equality $\langle v, v \rangle = 0$ if and only if $v = 0_V$

The pair $(V, \langle \cdot, \cdot \rangle)$ is called an **inner product space**.

Example 1.7 (Trace Inner Product). Consider $\text{Mat}(m, n, \mathbb{R})$. Equipped with the trace inner product $\langle A|B \rangle := \text{tr}(A^\top B)$ for all $A, B \in \text{Mat}(m, n, \mathbb{R})$ the pair $(\text{Mat}(m, n, \mathbb{R}), \langle \cdot, \cdot \rangle)$ is an inner product space.

Further, by Definition 1.18, this inner product's induced norm is $\|A\|_F := \sqrt{\langle A, A \rangle} = \sqrt{\text{tr}(A^\top A)}$, called the Frobenius norm.

Remark 1.4. In a real vector space, the properties of symmetry (IP1) and linearity in the first argument (IP2) together imply linearity in the second argument, meaning the inner product is fully bilinear.

Definition 1.18 (Induced Norm). Let $(V, \langle \cdot, \cdot \rangle)$ be an inner product space. For each vector $v \in V$, the **norm** of v induced by the inner product, denoted by $\|v\|$, is defined as:

$$\|v\| := \sqrt{\langle v, v \rangle}$$

Remark 1.5. Note that the quantity $\langle v, v \rangle$ is non-negative from positive-definiteness (IP3), hence the induced norm is always well-defined in $\mathbb{R}$.

Theorem 1.5 (Cauchy-Schwarz Inequality). Let V be an inner product space. For all vectors $u, v \in V$, the following inequality holds:

$$|\langle u, v \rangle| \leq \|u\| \|v\|$$

This inequality is an equality if and only if u, v are linearly dependent (Definition 1.5).

Proof. Let V be a real inner product space. We wish to show that for all $u, v \in V$, $|\langle u, v \rangle| \leq \|u\| \|v\|$.

Suppose $v = 0_V$. $0_V = 0 \cdot u$ (Proposition 1.1); combined with symmetry (IP1) and linearity in the first argument (IP2), $\langle u, 0_V \rangle = \langle u, 0 \cdot u \rangle = \langle 0 \cdot u, u \rangle = 0 \langle u, u \rangle = 0$. The inequality then becomes:

$$|\langle u, 0_V \rangle| = 0 \leq \|u\| \cdot 0 = 0$$

which holds as an equality. Thus, the theorem is true for $v = 0_V$.

Now assume $v \neq 0_V$. For any real number $t \in \mathbb{R}$, consider the function:

$$f(t) = \|u - tv\|^2$$

By the definition of the induced norm, $f(t) = \langle u - tv, u - tv \rangle$. By positive-definiteness (IP3), the inner product of any vector with itself is non-negative, and is zero iff the vector is the zero vector. Therefore, $f(t) \geq 0$ for all $t \in \mathbb{R}$.

Using bilinearity (Remark 1.5), we expand the inner product:

$$\begin{aligned} f(t) &= \langle u, u - tv \rangle - t \langle v, u - tv \rangle \\ &= \langle u, u \rangle - t \langle u, v \rangle - t \langle v, u \rangle + t^2 \langle v, v \rangle \\ &= \|u\|^2 - 2t \langle u, v \rangle + t^2 \|v\|^2 \end{aligned}$$

This is a quadratic in t of the form $at^2 + bt + c$, where $a = \|v\|^2$, $b = -2\langle u, v \rangle$, and $c = \|u\|^2$.

Since $v \neq 0_V$, positive-definiteness (IP3) gives $\|v\|^2 = \langle v, v \rangle > 0$, so $\|v\|^2 \neq 0$ and division by it below is valid.

Following the minimisation route, we choose the specific value of t that minimises $f(t)$. For a quadratic $at^2 + bt + c$, the minimum occurs at $t = -b/(2a)$. Here, we set:

$$t_0 = \frac{2\langle u, v \rangle}{2\|v\|^2} = \frac{\langle u, v \rangle}{\|v\|^2}$$

Plugging t_0 into our inequality $f(t) \geq 0$:

$$\begin{aligned} 0 &\leq \|u\|^2 - 2 \left(\frac{\langle u, v \rangle}{\|v\|^2} \right) \langle u, v \rangle + \left(\frac{\langle u, v \rangle}{\|v\|^2} \right)^2 \|v\|^2 \\ 0 &\leq \|u\|^2 - \frac{2\langle u, v \rangle^2}{\|v\|^2} + \frac{\langle u, v \rangle^2}{\|v\|^2} \\ 0 &\leq \|u\|^2 - \frac{\langle u, v \rangle^2}{\|v\|^2} \end{aligned}$$

Rearranging terms, we find $\frac{\langle u, v \rangle^2}{\|v\|^2} \leq \|u\|^2$, which implies $\langle u, v \rangle^2 \leq \|u\|^2 \|v\|^2$. Taking the square root of both sides yields the desired inequality:

$$|\langle u, v \rangle| \leq \|u\| \|v\|$$

Equality $|\langle u, v \rangle| = \|u\| \|v\|$ holds iff $f(t_0) = 0$. By positive-definiteness (IP3), $f(t_0) = \|u - t_0 v\|^2 = 0$ iff $u - t_0 v = 0_V$. This implies $u = t_0 v$, meaning u is a scalar multiple of v . Thus, equality holds iff u and v are linearly dependent. Q.E.D.

Remark 1.6 (Angle Between Vectors). Let V be a real inner product space and let $u, v \in V$ with $u, v \neq 0_V$. By Cauchy-Schwarz (Theorem 1.5),

$$|\langle u, v \rangle| \leq \|u\| \|v\|$$

Since $u, v \neq 0_V$, positive-definiteness (IP3) gives $\|u\|^2 > 0$ and $\|v\|^2 > 0$, so $\|u\| \neq 0$ and $\|v\| \neq 0$, and we may divide through by $\|u\| \|v\|$:

$$\left| \frac{\langle u, v \rangle}{\|u\| \|v\|} \right| \leq 1$$

That is, the ratio $\frac{\langle u, v \rangle}{\|u\| \|v\|}$ always lies in $[-1, 1]$, which is exactly the range of $\cos$. Since $\cos : [0, \pi] \rightarrow [-1, 1]$ is a bijection, there is a unique $\theta \in [0, \pi]$ satisfying

$$\cos \theta = \frac{\langle u, v \rangle}{\|u\| \|v\|}$$

This uniqueness is precisely what licenses calling θ **the angle between** u and v : without Cauchy-Schwarz bounding the ratio into $[-1, 1]$ first, no such θ would be well-defined at all.

Proposition 1.9 (Triangle Inequality). *Let V be a real inner product space. Then for all $u, v \in V$*

$$\|u + v\| \leq \|u\| + \|v\|$$

(a consequence of Cauchy-Schwarz, Theorem 1.5).

Proof. We bound $\|u + v\|^2$ and take square roots at the end.

By the definition of the induced norm and bilinearity (Remark 1.5),

$$\begin{aligned}\|u + v\|^2 &= \langle u + v, u + v \rangle \\ &= \langle u, u \rangle + \langle u, v \rangle + \langle v, u \rangle + \langle v, v \rangle \\ &= \|u\|^2 + 2\langle u, v \rangle + \|v\|^2\end{aligned}$$

using symmetry (IP1) to combine $\langle u, v \rangle$ and $\langle v, u \rangle$.

By Cauchy-Schwarz (Theorem 1.5), $|\langle u, v \rangle| \leq \|u\|\|v\|$, and in particular $\langle u, v \rangle \leq |\langle u, v \rangle| \leq \|u\|\|v\|$. Substituting this bound on the cross term,

$$\|u + v\|^2 \leq \|u\|^2 + 2\|u\|\|v\| + \|v\|^2$$

The right-hand side is a perfect square:

$$\|u\|^2 + 2\|u\|\|v\| + \|v\|^2 = (\|u\| + \|v\|)^2$$

So $\|u + v\|^2 \leq (\|u\| + \|v\|)^2$. Since both $\|u + v\|$ and $\|u\| + \|v\|$ are non-negative, taking square roots preserves the inequality:

$$\|u + v\| \leq \|u\| + \|v\|$$

Q.E.D.

1.3.2 Orthogonality

The angle between vectors (Remark 1.6) showed that $\theta = \pi/2$ if and only if $\cos \theta = 0$, which by that same remark holds if and only if $\langle u, v \rangle = 0$. This special case (i.e. an inner product of zero) is important enough to warrant its own name.

Definition 1.19 (Orthogonal Set / Orthonormal Set). Let V be an inner product space and let $v_1, \dots, v_m \in V$ be a set of vectors.

- i. The set $\{v_1, \dots, v_m\}$ is *orthogonal* if $\langle v_i, v_j \rangle = 0$ for all $i \neq j$.
- ii. The set $\{v_1, \dots, v_m\}$ is *orthonormal* if it is orthogonal and, in addition, $\|v_i\| = 1$ for all i .

Definition 1.20 (Orthogonal Matrix). A matrix $Q \in \text{Mat}(n, \mathbb{R})$ is *orthogonal* if its columns form an orthonormal set (Definition 1.19), equivalently, if

$$Q^T Q = \mathbb{I}$$

Remark 1.7. Both conditions in Definition 1.19 compress into a single statement using the Kronecker delta: $\{v_1, \dots, v_m\}$ is orthonormal if and only if

$$\langle v_i, v_j \rangle = \delta_{ij} \quad \text{for all } i, j$$

where $\delta_{ij} = 1$ if $i = j$ and 0 otherwise.

Proposition 1.10 (Orthogonality and Linear Independence). *Let V be an inner product space and let $v_1, \dots, v_m \in V$ be an orthogonal set of nonzero vectors. Then $v_1, \dots, v_m$ are linearly independent.*

Proof. Suppose

$$c_1 v_1 + \cdots + c_m v_m = 0_V$$

for some scalars $c_1, \dots, c_m \in \mathbb{R}$. We show $c_i = 0$ for all i .

Fix $k \in \{1, \dots, m\}$ and take the inner product of both sides with v_k :

$$\left\langle \sum_{i=1}^m c_i v_i, v_k \right\rangle = \langle 0_V, v_k \rangle$$

By symmetry (IP1) and the same argument as in the Cauchy–Schwarz proof, $\langle 0_V, v_k \rangle = \langle v_k, 0_V \rangle = 0$.

Expanding the left-hand side by bilinearity (Remark 1.5),

$$\sum_{i=1}^m c_i \langle v_i, v_k \rangle = 0$$

Since $\{v_1, \dots, v_m\}$ is orthogonal, $\langle v_i, v_k \rangle = 0$ for all $i \neq k$, so every term with $i \neq k$ vanishes, leaving only the $i = k$ term:

$$c_k \langle v_k, v_k \rangle = 0, \quad \text{i.e.} \quad c_k \|v_k\|^2 = 0$$

Since $v_k \neq 0_V$, positive-definiteness (IP3) gives $\|v_k\|^2 > 0$, so $\|v_k\|^2 \neq 0$, and hence $c_k = 0$.

Since k was arbitrary, $c_k = 0$ for every $k = 1, \dots, m$, so $v_1, \dots, v_m$ are linearly independent. Q.E.D.

Theorem 1.6 (Gram-Schmidt Process). *Let V be an inner product space and let $v_1, \dots, v_m \in V$ be linearly independent. Define*

$$u_1 = v_1, \quad u_k = v_k - \sum_{j=1}^{k-1} \frac{\langle v_k, u_j \rangle}{\langle u_j, u_j \rangle} u_j \quad \text{for } k = 2, \dots, m$$

Then $u_1, \dots, u_m$ is an orthogonal, hence linearly independent (Proposition 1.10), set of nonzero vectors satisfying

$$\text{span}\{u_1, \dots, u_k\} = \text{span}\{v_1, \dots, v_k\} \quad \text{for every } k = 1, \dots, m$$

Proof. A full treatment of this proof is given in (Golub & Van Loan 2013, §5.2). Q.E.D.

Corollary 1.3 (Existence of an Orthonormal Basis). *Let V be a finite-dimensional inner product space with $\dim V = n$. Then, by Theorem 1.6, V has an orthonormal basis.*

Proof. Since $\dim V = n$, V has a basis $v_1, \dots, v_n$.

Apply Gram-Schmidt (Theorem 1.6) to $v_1, \dots, v_n$, producing an orthogonal set $u_1, \dots, u_n$ with

$$\text{span}_{\mathbb{R}}\{u_1, \dots, u_n\} = \text{span}_{\mathbb{R}}\{v_1, \dots, v_n\} = V$$

By Theorem 1.6, $u_1, \dots, u_n$ is an orthogonal, linearly independent set of nonzero vectors, with nonzero vectors. Being n linearly independent vectors spanning the n -dimensional space V , $u_1, \dots, u_n$ form a basis of V by Proposition 1.4.

Define $e_k := u_k / \|u_k\|$ for each $k = 1, \dots, n$. Since $u_k \neq 0_V$, positive-definiteness (IP3) gives $\|u_k\| \neq 0$, so this division is well-defined.

For $i \neq k$, by bilinearity (Remark 1.5),

$$\langle e_i, e_k \rangle = \left\langle \frac{u_i}{\|u_i\|}, \frac{u_k}{\|u_k\|} \right\rangle = \frac{1}{\|u_i\|\|u_k\|} \langle u_i, u_k \rangle = \frac{1}{\|u_i\|\|u_k\|} \cdot 0 = 0$$

since $\langle u_i, u_k \rangle = 0$ by orthogonality of $u_1, \dots, u_n$, and dividing zero by the nonzero scalar $\|u_i\|\|u_k\|$ leaves zero.

By bilinearity (Remark 1.5),

$$\|\lambda v\|^2 = \langle \lambda v, \lambda v \rangle = \lambda^2 \langle v, v \rangle = \lambda^2 \|v\|^2$$

so $\|\lambda v\| = |\lambda| \|v\|$ (taking non-negative square roots). Applying this with $\lambda = 1/\|u_k\|$,

$$\|e_k\| = \left\| \frac{u_k}{\|u_k\|} \right\| = \frac{1}{\|u_k\|} \|u_k\| = 1$$

Hence $\{e_1, \dots, e_n\}$ is orthonormal (Definition 1.19). Since scaling each u_k by the nonzero scalar $1/\|u_k\|$ changes neither the span nor linear independence of $\{u_1, \dots, u_n\}$, $\{e_1, \dots, e_n\}$ remains a basis of V . Thus $\{e_1, \dots, e_n\}$ is an orthonormal basis of V . Q.E.D.

Remark 1.8 (Gram-Schmidt Fixes an Already-Orthonormal Set). If $u_1, \dots, u_r$ are already orthonormal, feeding them as the first r inputs to Gram-Schmidt (Theorem 1.6) changes nothing: at step $k \leq r$, the formula

$$u_k - \sum_{j=1}^{k-1} \frac{\langle u_k, u_j \rangle}{\langle u_j, u_j \rangle} u_j$$

subtracts off projections onto $u_1, \dots, u_{k-1}$, but these are all zero since the u_j are already mutually orthogonal — so the sum vanishes and u_k passes through unchanged. Since each u_k is already a unit vector, the normalisation $e_k := u_k/\|u_k\|$ (as in Corollary 1.3) leaves it unchanged too. So Gram-Schmidt only starts doing real work from step $r + 1$ onward.

1.3.3 QR Decompositions

Definition 1.21 (QR Decomposition). Let $A \in \text{Mat}(n, \mathbb{R})$. A QR decomposition of A is a factorisation $A = QR$ where $Q \in \text{Mat}(n, \mathbb{R})$ has orthonormal columns, that is, the columns $q_1, \dots, q_n$ form an orthonormal set (Definition 1.19); and $R \in \text{Mat}(n, \mathbb{R})$ is upper triangular with positive diagonal entries.

Theorem 1.7 (Existence and Uniqueness of the QR Decomposition). *Let $A \in \text{Mat}(n, \mathbb{R})$ be invertible. Then A admits a QR decomposition $A = QR$ (as in Definition 1.21), and this decomposition is unique.*

Proof. Let $a_1, \dots, a_n$ denote the columns of A . Since A is invertible, its columns are linearly independent.

Apply Gram-Schmidt (Theorem 1.6) to $a_1, \dots, a_n$ to obtain an orthogonal set $u_1, \dots, u_n$ with

$$\text{span}_{\mathbb{R}}\{u_1, \dots, u_k\} = \text{span}_{\mathbb{R}}\{a_1, \dots, a_k\} \quad \text{for every } k = 1, \dots, n$$

and normalise $q_k := u_k/\|u_k\|$ for each k , exactly as in the construction of Corollary 1.3; we do not re-derive well-definedness or orthonormality of $q_1, \dots, q_n$ here, having already established both.

Let Q be the matrix with columns $q_1, \dots, q_n$. By construction, Q has orthonormal columns.

We now construct R . Since $\text{span}_{\mathbb{R}}\{q_1, \dots, q_k\} = \text{span}_{\mathbb{R}}\{u_1, \dots, u_k\} = \text{span}_{\mathbb{R}}\{a_1, \dots, a_k\}$ for every k , each a_k lies in $\text{span}_{\mathbb{R}}\{q_1, \dots, q_k\}$, so

$$a_k = \sum_{j=1}^k c_j q_j$$

for some scalars $c_1, \dots, c_k$. Fix $i \leq k$ and take the inner product of both sides with q_i . By bilinearity (Remark 1.5),

$$\langle a_k, q_i \rangle = \sum_{j=1}^k c_j \langle q_j, q_i \rangle$$

Since $\{q_1, \dots, q_n\}$ is orthonormal, $\langle q_j, q_i \rangle = 0$ for $j \neq i$ and $\langle q_i, q_i \rangle = 1$, so every term vanishes except $j = i$, leaving $\langle a_k, q_i \rangle = c_i$. As $i \leq k$ was arbitrary, this identifies every coefficient: $c_j = \langle a_k, q_j \rangle$ for all $j = 1, \dots, k$, so

$$a_k = \sum_{j=1}^k \langle a_k, q_j \rangle q_j$$

Define $R_{jk} := \langle a_k, q_j \rangle$ for $j \leq k$ and $R_{jk} := 0$ for $j > k$. This is upper triangular by construction, and the identity above shows precisely that $a_k = \sum_{j=1}^n R_{jk} q_j$ is the k -th column of QR , so $A = QR$.

It remains to show $R_{kk} > 0$ for every k . Fix k . By the Gram-Schmidt relation,

$$u_k = a_k - \sum_{j < k} \text{proj}_{u_j}(a_k)$$

so

$$a_k = u_k + \sum_{j < k} \text{proj}_{u_j}(a_k)$$

Each $\text{proj}_{u_j}(a_k)$ is by definition a scalar multiple of u_j , hence of q_j . So $a_k = u_k + \sum_{j < k} d_j q_j$ for some scalars d_j . Taking the inner product with q_k and using bilinearity (Remark 1.5),

$$\langle a_k, q_k \rangle = \langle u_k, q_k \rangle + \sum_{j < k} d_j \langle q_j, q_k \rangle$$

Since $\{q_1, \dots, q_n\}$ is orthonormal, $\langle q_j, q_k \rangle = 0$ for $j \neq k$, so every term in the sum vanishes, leaving

$$R_{kk} = \langle a_k, q_k \rangle = \langle u_k, q_k \rangle = \left\langle u_k, \frac{u_k}{\|u_k\|} \right\rangle = \frac{\|u_k\|^2}{\|u_k\|} = \|u_k\|$$

Since $u_k \neq 0_V$ (Gram-Schmidt produces nonzero vectors, as used already in Corollary 1.3), positive-definiteness (IP3) gives $\|u_k\| > 0$. Hence $R_{kk} > 0$ for all k . This establishes existence of the decomposition $A = QR$ with the stated properties.

To prove uniqueness of the QR decomposition, we rely on (Golub & Van Loan 2013, §5.2). Q.E.D.

Proposition 1.11 (Solving $Ax = b$ via $A = QR$). *Let $A \in \text{Mat}(n, \mathbb{R})$ be invertible, let Q, R be such that $A = QR$ (Theorem 1.7), and let $b \in \mathbb{R}^n$. Then the system $Ax = b$ has a unique solution x , obtained by first computing $c := Q^\top b$, then solving $Rx = c$ for x by back substitution.*

Proof. Substituting $A = QR$, the system $Ax = b$ becomes

$$QRx = b$$

Since Q has orthonormal columns, $Q^\top Q = I$. As Q is square, this left inverse is automatically a two-sided inverse, so Q is invertible with $Q^{-1} = Q^\top$.

Multiplying both sides of $QRx = b$ on the left by $Q^\top$,

$$Q^\top QRx = Q^\top b \implies Rx = Q^\top b$$

Define $c := Q^\top b$. Unlike the analogous step in the $PA = LU$ method, computing c requires no triangular solve — it is a single direct matrix-vector product.

It remains to solve $Rx = c$. Since R is upper triangular with $R_{kk} > 0$ for all k (Theorem 1.7), R is invertible, so $Rx = c$ has a unique solution obtained by back substitution:

$$x_k = \frac{1}{R_{kk}} \left(c_k - \sum_{j=k+1}^n R_{kj}x_j \right) \quad k = n, n-1, \dots, 1$$

Existence and uniqueness of c and then x give existence and uniqueness of a solution to $Ax = b$. Q.E.D.

Computational Remark 1.2. `gramSchmidtQR()` (Appendix A.3) projects each R_{jk} against the running vector u_k and not the original column a_k as Theorem 1.6's formula suggests. This is fine mathematically: once earlier projections are removed from u_k , $\langle u_k, q_j \rangle = \langle a_k, q_j \rangle$ anyway, so the theorem's guarantees still hold. This is the modified Gram-Schmidt Process.

It matters numerically because projecting against a_k every time (classical Gram-Schmidt) lets rounding error compound independently in each coefficient; projecting against the already-partially-cleaned u_k (modified Gram-Schmidt) stops that error from carrying forward as badly. Refer to (Golub & Van Loan 2013, §5.2.7–5.2.9) for a more detailed explanation.

This is verified directly in `main()`: $Q^\top Q \approx \mathbb{1}$ to machine precision (10^{-16} – 10^{-17} off the diagonal), and $QR \approx A$, an exact reconstruction of the input.

1.4 Eigenvalues and the Spectral Theorem

This section develops eigenvalues, eigenvectors and the spectral theorem for symmetric matrices, culminating in the existence of an orthonormal eigenbasis.

Beyond its role in diagonalising symmetric matrices, this result is the algebraic foundation for the Singular Value Decomposition (or SVD), which this paper uses later in the chapter to analyse and decompose general $m \times n$ matrices.

The algorithm is implemented from scratch in C++ as `jacobi_eigensolver.cpp` (Appendix A.4), computing the eigendecomposition of a symmetric matrix via the classical Jacobi method.

1.4.1 Eigenvalues and Eigenvectors

Let $A \in \text{Mat}(n, \mathbb{R})$.

Definition 1.22 (Eigenvalue / Eigenvector). A scalar $\lambda \in \mathbb{F}$ is an eigenvalue of A if there exists a nonzero $v \in \mathbb{F}^n$ with $Av = \lambda v$. Such a v is an eigenvector.

Rewriting the above statement's equation, we obtain $Av = \lambda v \iff (A - \lambda \mathbb{1})v = 0$. So λ is an eigenvalue iff $A - \lambda \mathbb{1}$ is singular, i.e.

$$\det(A - \lambda \mathbb{1}) = 0$$

This constitutes the following definition.

Definition 1.23 (Characteristic Polynomial). The characteristic polynomial of a matrix A is a degree- n polynomial in λ of the form

$$p_A(\lambda) := \det(A - \lambda \mathbb{1})$$

Its roots are exactly the eigenvalues of A .

Remark 1.9. Over $\mathbb{C}$, the Fundamental Theorem of Algebra guarantees that p_A has exactly n roots counted with multiplicity, so every $A \in \text{Mat}(n, \mathbb{C})$ has n eigenvalues.

Example 1.8 (A Matrix with No Real Eigenvalues). Consider

$$A = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

The characteristic polynomial is

$$p_A(\lambda) = \det(A - \lambda \mathbb{1}) = \det \begin{bmatrix} -\lambda & -1 \\ 1 & -\lambda \end{bmatrix} = (-\lambda)(-\lambda) - (-1)(1) = \lambda^2 + 1$$

Setting $p_A(\lambda) = 0$ gives $\lambda^2 = -1$, which has no solution in $\mathbb{R}$; the roots are $\lambda = \pm i$. So A has no real eigenvalues, and hence no real eigenvectors.

Geometrically, A is the matrix of a 90 rotation (anticlockwise) in $\mathbb{R}^2$. An eigenvector would be a nonzero vector whose direction is unchanged by A but a 90 rotation fixes no direction at all: every nonzero vector in $\mathbb{R}^2$ is rotated onto a different line through the origin. This is the geometric content behind the absence of real eigenvalues here.

This motivates restricting our attention to a class of matrices for which real eigenvalues are guaranteed: symmetric matrices.

Definition 1.24 (Symmetric Matrix). Let $A \in \text{Mat}(n, \mathbb{R})$. A symmetric matrix is a matrix where $A^\top = A$.

Definition 1.25 (Self Adjoint). Let $A \in \text{Mat}(n, \mathbb{R})$, equipped with the standard inner product. When the following property is true for A ,

$$\langle Av, w \rangle = \langle v, Aw \rangle \quad \text{for all } v, w \in \mathbb{R}^n$$

A is called self-adjoint.

Remark 1.10. Some remarks to consider about self-adjointness are

- i. When A is symmetric, the property holds.

- ii. The property in Definition 1.25 can be generalised for any inner product, however for the scope of this paper we define this one case.

Theorem 1.8 (Real Eigenvalues of Symmetric Matrices). *Let $A \in \text{Mat}(n, \mathbb{R})$ be symmetric, i.e. $A = A^\top$. Then every eigenvalue of A is real.*

Proof. Let $\lambda \in \mathbb{C}$ be a root of the characteristic polynomial p_A , with corresponding eigenvector $v \in \mathbb{C}^n \setminus \{0\}$, so $Av = \lambda v$; a priori λ and v may be complex, since we have not yet shown λ is real. Let $v^* := \bar{v}^\top$ denote the conjugate transpose of v , and consider the scalar v^*Av .

Using $Av = \lambda v$,

$$v^*Av = v^*(\lambda v) = \lambda v^*v$$

We show v^*Av is real, by showing it equals its own complex conjugate. Conjugating the product $v^*Av = \bar{v}^\top Av$ entrywise (conjugation of a product preserves the order of the factors, unlike transpose),

$$\overline{v^*Av} = \overline{\bar{v}^\top Av} = \bar{\bar{v}}^\top \bar{A} \bar{v} = v^\top A \bar{v}$$

using $\bar{\bar{v}} = v$ and, since A is real, $\bar{A} = A$.

Separately, since v^*Av is a 1×1 matrix (a scalar), it equals its own transpose, and transposing a product reverses the order of the factors:

$$v^*Av = \bar{v}^\top Av = (\bar{v}^\top Av)^\top = v^\top A^\top \bar{v} = v^\top A \bar{v}$$

using symmetry, $A^\top = A$, in the last step.

Both computations give $v^\top A \bar{v}$, so $\overline{v^*Av} = v^*Av$, and hence v^*Av is real.

Writing $v = (v_1, \dots, v_n)^\top$,

$$v^*v = \bar{v}^\top v = \sum_{i=1}^n \bar{v}_i v_i = \sum_{i=1}^n |v_i|^2$$

which is real, and strictly positive since $v \neq 0$.

From the first computation, $v^*Av = \lambda v^*v$. The left-hand side is real, and v^*v is a positive real number; a real number divided by a positive real number is real, so

$$\lambda = \frac{v^*Av}{v^*v}$$

is real. Q.E.D.

Remark 1.11 (Standard Inner Product). On $\mathbb{R}^n$, the standard inner product coincides with the dot product, $\langle x, y \rangle := x \cdot y = \sum_{i=1}^n x_i y_i$, which for column vectors $x, y \in \mathbb{R}^n$ can equivalently be written as the matrix product

$$\langle x, y \rangle = x^\top y$$

a 1×1 matrix identified with its unique scalar entry. This identification is used throughout the results on symmetric matrices below.

Proposition 1.12 (Orthogonal Eigenvectors). *Let $A \in \text{Mat}(n, \mathbb{R})$ be symmetric, and let v_1, v_2 be eigenvectors of A with $Av_1 = \lambda_1 v_1$, $Av_2 = \lambda_2 v_2$, and $\lambda_1 \neq \lambda_2$. Then $\langle v_1, v_2 \rangle = 0$.*

Proof. Consider $\langle Av_1, v_2 \rangle$, computed two ways.

Using $Av_1 = \lambda_1 v_1$ and linearity in the first argument (Remark 1.5),

$$\langle Av_1, v_2 \rangle = \langle \lambda_1 v_1, v_2 \rangle = \lambda_1 \langle v_1, v_2 \rangle$$

Using the matrix-multiplication form of the inner product (Remark 1.11), $\langle Av_1, v_2 \rangle = (Av_1)^\top v_2 = v_1^\top A^\top v_2$, and symmetry, $A^\top = A$,

$$\langle Av_1, v_2 \rangle = v_1^\top A^\top v_2 = v_1^\top A v_2 = \langle v_1, Av_2 \rangle = \langle v_1, \lambda_2 v_2 \rangle = \lambda_2 \langle v_1, v_2 \rangle$$

where the last step uses linearity in the second argument (Remark 1.5).

Equating the two computations,

$$\lambda_1 \langle v_1, v_2 \rangle = \lambda_2 \langle v_1, v_2 \rangle \implies (\lambda_1 - \lambda_2) \langle v_1, v_2 \rangle = 0$$

Since $\lambda_1 \neq \lambda_2$, $\lambda_1 - \lambda_2 \neq 0$, so $\langle v_1, v_2 \rangle = 0$.

Q.E.D.

1.4.2 The Spectral Theorem

Definition 1.26 (Invariant Subspace). Let $A \in \text{Mat}(n, \mathbb{R})$ and let $U \subseteq \mathbb{R}^n$ be a subspace. U is invariant under A if $Av \in U$ for every $v \in U$.

Definition 1.27 (Orthogonal Complement). Let $U \subseteq \mathbb{R}^n$ be a subspace. The orthogonal complement of U is

$$U^\perp := \{w \in \mathbb{R}^n \mid \langle u, w \rangle = 0 \text{ for all } u \in U\}$$

Lemma 1.1 (Invariance of the Orthogonal Complement). Let $A \in \text{Mat}(n, \mathbb{R})$ be symmetric, and let $U \subseteq \mathbb{R}^n$ be a subspace invariant under A (Definition 1.26). Then $U^\perp$ (Definition 1.27) is also invariant under A .

Proof. Let $A \in \text{Mat}(n, \mathbb{R})$ be symmetric, and let $U \subseteq \mathbb{R}^n$ be a subspace invariant under A (Definition 1.26). We wish to show that $U^\perp$ (Definition 1.27) is also invariant under A .

Let $w \in U^\perp$ be arbitrary. By Definition 1.27, our goal is to show $Aw \in U^\perp$, i.e.

$$\langle u, Aw \rangle = 0 \quad \text{for every } u \in U$$

Fix an arbitrary $u \in U$; we verify the equality above for this u .

Since A is symmetric, $\langle u, Av \rangle = \langle Au, v \rangle$ for all $u, v \in \mathbb{R}^n$, by Definition 1.25 and Remark 1.10,

$$\langle u, Aw \rangle = \langle Au, w \rangle$$

Since U is invariant under A (Definition 1.26) and $u \in U$,

$$Au \in U$$

We now have $\langle Au, w \rangle$ with $Au \in U$ and $w \in U^\perp$. By Definition 1.27, the inner product of any element of U with any element of $U^\perp$ is zero:

$$\langle Au, w \rangle = 0$$

Chaining these equalities,

$$\langle u, Aw \rangle = \langle Au, w \rangle = 0$$

Since $u \in U$ was arbitrary, $Aw \in U^\perp$ by Definition 1.27. Since $w \in U^\perp$ was arbitrary, A maps every vector of $U^\perp$ back into $U^\perp$; that is, $U^\perp$ is invariant under A (Definition 1.26). Q.E.D.

Corollary 1.4 (Orthogonal Complement of the Kernel). *Let $A \in \text{Mat}(m, n, \mathbb{R})$. Then*

$$\ker(A)^\perp = \text{im}(A^\top)$$

Proof. For any $w = A^\top y \in \text{im}(A^\top)$ and $v \in \ker(A)$,

$$\langle w, v \rangle = (A^\top y)^\top v = y^\top (Av) = y^\top 0 = 0$$

so $w \in \ker(A)^\perp$. Hence $\text{im}(A^\top) \subseteq \ker(A)^\perp$.

By Rank-Nullity (Theorem 1.2), $\dim \ker(A) = n - \text{rank}(A)$. For any subspace $U \subseteq \mathbb{R}^n$, $\dim U^\perp = n - \dim U$ (Horn & Johnson 2013), so $\dim \ker(A)^\perp = \text{rank}(A)$. Also, row rank equals column rank, $\text{rank}(A^\top) = \text{rank}(A)$ by Horn & Johnson (2013), so $\dim \text{im}(A^\top) = \text{rank}(A)$ too. So $\ker(A)^\perp$ and $\text{im}(A^\top)$ have equal dimension.

Equal dimension plus containment gives equality: $\ker(A)^\perp = \text{im}(A^\top)$. Q.E.D.

Theorem 1.9 (Real Spectral Theorem). *Let $A \in \text{Mat}(n, \mathbb{R})$ be symmetric. Then there exists an orthonormal basis $\{q_1, \dots, q_n\}$ of $\mathbb{R}^n$ consisting of eigenvectors of A .*

Equivalently, there exists an orthogonal matrix $Q \in \text{Mat}(n, \mathbb{R})$ (Definition 1.20) with columns $q_1, \dots, q_n$, and a diagonal matrix $\Lambda \in \text{Mat}(n, \mathbb{R})$ with the corresponding eigenvalues $\lambda_1, \dots, \lambda_n$ on the diagonal, such that

$$A = Q\Lambda Q^\top$$

Proof. We proceed by induction on n .

Base case ($n = 1$). Let $A = [\lambda] \in \text{Mat}(1, \mathbb{R})$. Any nonzero $v \in \mathbb{R}$ satisfies $Av = \lambda v$, so v is an eigenvector (Definition 1.22). Setting $q_1 := v/|v|$ gives an orthonormal basis $\{q_1\}$ of $\mathbb{R}^1$.

Inductive step. Assume every symmetric $(n-1) \times (n-1)$ matrix has an orthonormal eigenbasis. Let $A \in \text{Mat}(n, \mathbb{R})$ be symmetric.

By Definition 1.23, A 's characteristic polynomial p_A has degree n , so by Remark 1.9 (FTA) it has n roots over $\mathbb{C}$, all real by Theorem 1.8. Let $\lambda_1 \in \mathbb{R}$ be one such root. Since $A - \lambda_1 \mathbb{1}$ is a real matrix with $\det(A - \lambda_1 \mathbb{1}) = 0$, it is not invertible, so by the Rank-Nullity Theorem (Theorem 1.2) it has nontrivial kernel: there exists nonzero $v \in \mathbb{R}^n$ with $(A - \lambda_1 \mathbb{1})v = 0$, i.e. $Av = \lambda_1 v$. Normalising, $q_1 := v/\|v\|$ satisfies $Aq_1 = \lambda_1 q_1$ and $\|q_1\| = 1$.

Let $U := \text{span}_{\mathbb{R}}\{q_1\}$; since $Aq_1 = \lambda_1 q_1 \in U$, U is invariant under A (Definition 1.26), and by Lemma 1.1, so is $U^\perp$ (Definition 1.27), with $\dim U^\perp = n - 1$.

By Corollary 1.3, $U^\perp$ has an orthonormal basis $\{e_1, \dots, e_{n-1}\}$ (Definition 1.19). Since $U^\perp$ is invariant, each $Ae_j \in U^\perp$ can be expanded in this basis, defining $A' \in \text{Mat}(n-1, \mathbb{R})$ by

$$A'_{ij} := \langle e_i, Ae_j \rangle$$

via the same coefficient-extraction argument used to construct R in Theorem 1.7 (Remark 1.11). Since A is symmetric, $\langle u, Av \rangle = \langle Au, v \rangle$ for all $u, v \in \mathbb{R}^n$ (Definition 1.25/Remark 1.10); restricting to $u = e_i, v = e_j$ gives $A'_{ij} = \langle e_i, Ae_j \rangle = \langle Ae_i, e_j \rangle = \langle e_j, Ae_i \rangle = A'_{ji}$, using symmetry of the inner product (IP1). So A' is symmetric.

By the inductive hypothesis, A' has an orthonormal eigenbasis $\{q'_2, \dots, q'_n\} \subseteq \mathbb{R}^{n-1}$. The coordinate map $\phi : \mathbb{R}^{n-1} \rightarrow U^\perp$, $\phi(v') := \sum_{i=1}^{n-1} v'_i e_i$, is an isometry, since $\{e_1, \dots, e_{n-1}\}$ is

itself orthonormal: for $v', w' \in \mathbb{R}^{n-1}$, $\langle \phi(v'), \phi(w') \rangle = \sum_{i,j} v'_i w'_j \langle e_i, e_j \rangle = \sum_i v'_i w'_i = \langle v', w' \rangle$. Hence ϕ carries the orthonormal basis $\{q'_2, \dots, q'_n\}$ to an orthonormal set $q_2, \dots, q_n \in U^\perp$, with $Aq_k = \lambda_k q_k$, $\lambda_k \in \mathbb{R}$ real by Theorem 1.8 applied to A' .

Since $q_1 \in U$ and $q_2, \dots, q_n \in U^\perp$, Definition 1.27 gives $\langle q_1, q_k \rangle = 0$ for $k \geq 2$; together with the orthonormality of $\{q_2, \dots, q_n\}$, $\{q_1, \dots, q_n\}$ is an orthonormal set (Definition 1.19) of n eigenvectors of A , hence an orthonormal basis of $\mathbb{R}^n$ by Proposition 1.10 and Proposition 1.4.

Let Q have columns $q_1, \dots, q_n$ (orthogonal, Definition 1.20) and let $\Lambda := \text{diag}(\lambda_1, \dots, \lambda_n)$. Then $AQ = Q\Lambda$ column-by-column, so, using $Q^{-1} = Q^\top$,

$$A = Q\Lambda Q^\top$$

This completes the induction.

Q.E.D.

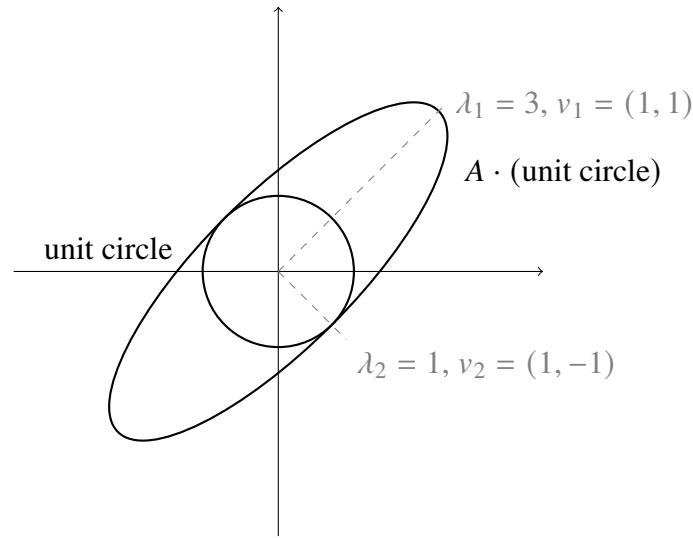


Figure 1.2: The symmetric matrix $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ maps the unit circle (blue) to an ellipse (red) whose semi-axes are exactly A 's eigenvalues, 3 and 1, along its orthogonal eigenvectors, $(1, 1)$ and $(1, -1)$. This is what Theorem 1.9 guarantees geometrically: a symmetric matrix acts as pure stretching along an orthogonal pair of directions, with no rotation or shearing, unlike a general matrix.

Computational Remark 1.3. `jacobi_eigensolver.cpp` (Appendix A.4) makes two algorithmic choices worth justifying. These are choices about how quickly and reliably the method converges in floating point, not consequences of Theorem 1.9 itself. The theorem guarantees that an orthonormal eigenbasis exists, but doesn't tell us how to compute one.

- i. **Iteration cap scaled with n .** The loop stops once the largest off-diagonal entry is below tolerance, or after $50n^2$ iterations. Since each sweep scans $O(n^2)$ pairs, scaling the cap with n^2 keeps it proportional to the actual problem size, instead of a fixed constant that's too small for large matrices or too big for small ones.
- ii. **Choosing the root of the same sign as τ .** `computeRotationAngle()` solves for $t = \tan \theta$ using $\tau = (A_{qq} - A_{pp})/(2A_{pq})$, and picks the root with the same sign as τ (check lines

40-44). Both roots zero out A_{pq} equally well algebraically, so this is a purely numerical choice: the two roots involve $\tau \pm \sqrt{1 + \tau^2}$, and for large $|\tau|$, $\sqrt{1 + \tau^2} \approx |\tau|$, so subtracting same-signed terms would cancel almost entirely and lose precision. Adding instead avoids this, and also picks the smaller rotation angle, which converges faster too.

This is verified on the 5×5 tridiagonal test case in `main()` by cross-checking the recovered eigenvalues printed against an independently computed trace and determinant of A .

1.5 The Singular Value Decomposition

This section develops the singular value decomposition (SVD), derived from the eigendecomposition of $A^\top A$, together with the Moore-Penrose pseudoinverse and the low-rank approximation guaranteed by the Eckart-Young theorem.

Beyond its use elsewhere in this chapter for solving least-squares problems and characterising a matrix's best low-rank approximations, the SVD is one of the central tools of applied linear algebra: it reveals a matrix's rank, range and null space directly from its singular values and underlies dimensionality reduction techniques such as principal component analysis.

The algorithm is implemented from scratch in C++ as `svd_jacobi.cpp` (Appendix A.5), computing the SVD via Jacobi diagonalisation of $A^\top A$.

Theorem 1.10 (Singular Value Decomposition). *Let $A \in \text{Mat}(m, n, \mathbb{R})$. Then there exists a singular value decomposition (SVD) i.e. an orthogonal matrix $U \in \text{Mat}(m, \mathbb{R})$, an orthogonal matrix $V \in \text{Mat}(n, \mathbb{R})$ and a matrix $\Sigma \in \text{Mat}(m, n, \mathbb{R})$ that is diagonal with ordered entries $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(m, n)} \geq 0$, such that*

$$A = U\Sigma V^\top$$

Moreover, the columns $v_1, \dots, v_n$ of V are an orthonormal eigenbasis of $A^\top A$ (Theorem 1.9), the columns $u_1, \dots, u_m$ of U are an orthonormal basis of $\mathbb{R}^m$, and the two are paired rather than chosen independently: for each i with $\sigma_i \neq 0$,

$$u_i = \frac{1}{\sigma_i} A v_i$$

where $\sigma_i := \sqrt{\lambda_i}$ and λ_i is the eigenvalue of $A^\top A$ corresponding to v_i .

Proof. Consider $A^\top A \in \text{Mat}(n, \mathbb{R})$. It is symmetric and $\langle A^\top A v, v \rangle = \|A v\|^2 \geq 0$ for all v , so all its eigenvalues are non-negative. By the Spectral Theorem (Theorem 1.9), $A^\top A$ has an orthonormal eigenbasis $v_1, \dots, v_n$ with eigenvalues $\lambda_1 \geq \dots \geq \lambda_n \geq 0$.

Define $\sigma_i := \sqrt{\lambda_i}$, and let r be the number of strictly positive σ_i (so $\sigma_1 \geq \dots \geq \sigma_r > 0$ and $\sigma_{r+1} = \dots = \sigma_n = 0$).

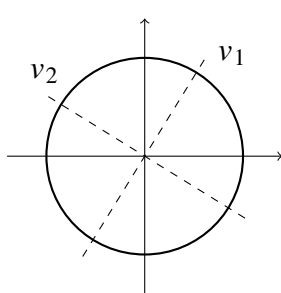
For $i \leq r$, set $u_i := A v_i / \sigma_i$. These are orthonormal: for $j, k \leq r$,

$$\langle u_j, u_k \rangle = \frac{1}{\sigma_j \sigma_k} \langle A v_j, A v_k \rangle = \frac{1}{\sigma_j \sigma_k} \langle A^\top A v_j, v_k \rangle = \frac{\lambda_j}{\sigma_j \sigma_k} \langle v_j, v_k \rangle = \frac{\sigma_j}{\sigma_k} \delta_{jk}$$

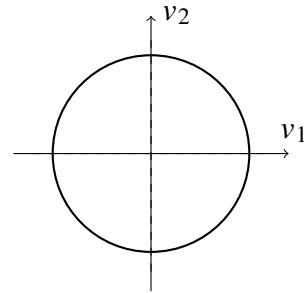
which is 1 if $j = k$ and 0 otherwise. Extend $\{u_1, \dots, u_r\}$ to a full orthonormal basis $\{u_1, \dots, u_m\}$ of $\mathbb{R}^m$. Since $\{u_1, \dots, u_r\}$ is orthonormal, it is linearly independent (Proposition 1.10), so Corollary 1.1 extends it to a basis of $\mathbb{R}^m$; applying Gram-Schmidt (Theorem 1.6) and normalising

(Corollary 1.3) then gives an orthonormal basis $\{u_1, \dots, u_m\}$, with $u_1, \dots, u_r$ unchanged by Remark 1.8. Let U, V be the matrices with these columns; both are orthogonal by construction.

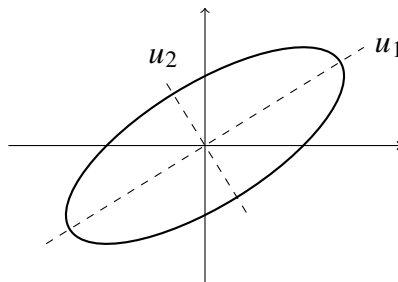
It suffices to check $AV = U\Sigma$, i.e. $Av_k = \sigma_k u_k$ for every k . For $k \leq r$ this is immediate from how u_k was defined. For $k > r$, $\sigma_k = 0$ and $\lambda_k = 0$, so $\|Av_k\|^2 = \langle A^\top Av_k, v_k \rangle = 0$, forcing $Av_k = 0 = \sigma_k u_k$ too. So $Av_k = \sigma_k u_k$ holds for every k , giving $AV = U\Sigma$. Multiplying by $V^\top = V^{-1}$ on the right gives $A = U\Sigma V^\top$. Q.E.D.



(a) Unit circle, input directions v_1, v_2



(b) After $V^\top$: same circle, axes realigned



(c) After Σ and U : ellipse along u_1, u_2

Figure 1.3: The SVD of the shear $A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ (Theorem 1.10). The circle's input axes v_1, v_2 (eigenvectors of $A^\top A$) sit at 58.28° ; $V^\top$ leaves the circle unchanged, just re-expressed in this frame. Scaling by $\Sigma \approx \text{diag}(1.618, 0.618)$ and rotating by U gives the final ellipse, with axes u_1, u_2 at 31.72° , a different angle from v_1, v_2 . This input/output asymmetry is what distinguishes SVD from the spectral decomposition of a symmetric matrix, where the two angles coincide.

Computational Remark 1.4. `svd_jacobi.cpp`'s `computeSigmaAndU()` (Appendix A.5) uses a thin U with zero columns, not the full orthonormal extension Theorem 1.10's proof builds. That proof works to extend $\{u_1, \dots, u_r\}$ into a full $m \times m$ orthogonal matrix (Corollary 1.1, Gram-Schmidt, Remark 1.8). The code skips this: U is $m \times n$, and columns past r stay zero by default. This is safe because those columns get multiplied by zero entries of Σ anyway, so they never affect $U\Sigma V^\top$. The full extension makes U mathematically complete, but isn't needed for reconstruction.

This is exercised directly in `main()`: the test matrix has rank 3, so at least three columns of U are necessarily zero, and reconstruction still matches A to machine precision (2.66×10^{-15} max error).

Definition 1.28 (Moore-Penrose Pseudoinverse). Let $A \in \text{Mat}(m, n, \mathbb{R})$ with SVD $A = U\Sigma V^\top$ (Theorem 1.10), where Σ has non-zero entries $\sigma_1, \dots, \sigma_r$ and zeros elsewhere. We define

$\Sigma^\dagger \in \text{Mat}(n, m, \mathbb{R})$ by replacing each non-zero σ_i with the reciprocal $1/\sigma_i$ and transposing the shape. The pseudoinverse of A is

$$A^\dagger := V\Sigma^\dagger U^\top$$

Remark 1.12. If A is square and invertible, then $A^\dagger = A^{-1}$. $r = n = m$, so $\Sigma^\dagger = \Sigma^{-1}$ exactly.

Proposition 1.13 (Least Squares Property of the Pseudoinverse). *Let $A \in \text{Mat}(m, n, \mathbb{R})$ and $b \in \mathbb{R}^m$, and let $x^\dagger := A^\dagger b$ (Definition 1.28). Then:*

i. *For all $x \in \mathbb{R}^n$,*

$$\|Ax^\dagger - b\| \leq \|Ax - b\|$$

so $x^\dagger$ minimises the residual $\|Ax - b\|$ over all $x \in \mathbb{R}^n$. Equality holds if and only if $x \in x^\dagger + \ker(A)$ ($\text{rank}(B) \leq k$).

ii. *Among all minimisers of the residual, $x^\dagger$ has the smallest norm: if $x \in \mathbb{R}^n$ satisfies $\|Ax - b\| = \|Ax^\dagger - b\|$, then*

$$\|x^\dagger\| \leq \|x\|$$

with equality if and only if $x = x^\dagger$.

Proof. By Theorem 1.10, $A = U\Sigma V^\top$ with U, V orthogonal and Σ having singular values $\sigma_1 \geq \dots \geq \sigma_r > 0$ on the diagonal, zeros elsewhere.

Let $y := V^\top x$ (so $x = Vy$, $\|x\| = \|y\|$, since V is orthogonal) and $c := U^\top b$. Since U is orthogonal, $\|Uz - b\| = \|z - U^\top b\|$ for any z ; applying this with $z = \Sigma y$,

$$\|Ax - b\| = \|U\Sigma V^\top x - b\| = \|\Sigma y - c\|$$

Since $(\Sigma y)_i = \sigma_i y_i$ for $i \leq r$ and 0 for $i > r$,

$$\|\Sigma y - c\|^2 = \sum_{i=1}^r (\sigma_i y_i - c_i)^2 + \sum_{i=r+1}^m c_i^2$$

The second sum is fixed; the first depends only on $y_1, \dots, y_r$, and is minimised (to zero) exactly when $y_i = c_i/\sigma_i$ for $i \leq r$. The remaining coordinates $y_{r+1}, \dots, y_n$ don't affect the residual at all, so any choice of them gives a minimiser.

By Definition 1.28, $x^\dagger := A^\dagger b = V\Sigma^\dagger c$, so $V^\top x^\dagger = (c_1/\sigma_1, \dots, c_r/\sigma_r, 0, \dots, 0)$ — exactly the minimiser above, with the free coordinates set to 0. So $x^\dagger$ minimises the residual, giving the inequality in (i).

Equality condition. $x = Vy$ achieves equality iff $y_i = c_i/\sigma_i$ for $i \leq r$ and $y_{r+1}, \dots, y_n$ arbitrary, i.e. $x = x^\dagger + \sum_{i>r} t_i v_i$ for arbitrary t_i .

We identify this set as exactly $\ker(A)$. Since $Av_i = \sigma_i u_i = 0$ for $i > r$ (shown in the proof of Theorem 1.10), $\text{span}_{\mathbb{R}}\{v_{r+1}, \dots, v_n\} \subseteq \ker(A)$. Conversely, take any $x \in \ker(A)$ and write $x = Vy$; then

$$0 = Ax = U\Sigma V^\top(Vy) = U\Sigma y$$

Since U is orthogonal, hence injective, this forces $\Sigma y = 0$, i.e. $\sigma_i y_i = 0$ for each $i \leq r$; since $\sigma_i \neq 0$ for $i \leq r$, this gives $y_i = 0$ for $i \leq r$, with $y_{r+1}, \dots, y_n$ unconstrained. So $x = Vy = \sum_{i>r} y_i v_i \in \text{span}_{\mathbb{R}}\{v_{r+1}, \dots, v_n\}$. Combined with the other inclusion, $\ker(A) = \text{span}_{\mathbb{R}}\{v_{r+1}, \dots, v_n\}$.

So equality holds iff $x \in x^\dagger + \ker(A)$.

Minimum norm. If $x = x^\dagger + z$ with $z \in \ker(A)$ also minimises the residual, then $z \in \text{span}_{\mathbb{R}}\{v_{r+1}, \dots, v_n\}$ while $x^\dagger \in \text{span}_{\mathbb{R}}\{v_1, \dots, v_r\}$, so $\langle x^\dagger, z \rangle = 0$ by orthonormality of $\{v_i\}$. Then

$$\|x\|^2 = \|x^\dagger + z\|^2 = \|x^\dagger\|^2 + \|z\|^2 \geq \|x^\dagger\|^2,$$

using bilinearity (Remark 1.5), with equality iff $z = 0$, i.e. $x = x^\dagger$. Q.E.D.

Lemma 1.2 (Unitary invariance of the Frobenius norm). *For any $M \in \text{Mat}(m, n, \mathbb{R})$ and orthogonal matrices $U \in \text{Mat}(m, \mathbb{R})$ and $V \in \text{Mat}(n, \mathbb{R})$ we have:*

$$\|UMV^\top\|_F = \|M\|_F$$

Hence multiplying M by orthogonal matrices doesn't change its Frobenius norm.

Proof. First, the trace identity $\text{tr}(AB) = \text{tr}(BA)$ for any $A \in \text{Mat}(m, n, \mathbb{R})$, $B \in \text{Mat}(n, m, \mathbb{R})$: writing out entries,

$$\text{tr}(AB) = \sum_i (AB)_{ii} = \sum_i \sum_j A_{ij} B_{ji} = \sum_j \sum_i B_{ji} A_{ij} = \sum_j (BA)_{jj} = \text{tr}(BA)$$

simply swapping the order of summation.

By Definition 1.18 and Example 1.7,

$$\|UMV^\top\|_F^2 = \text{tr}((UMV^\top)^\top (UMV^\top))$$

Since transposing a product reverses the order of factors, $(UMV^\top)^\top = VM^\top U^\top$, so

$$(UMV^\top)^\top (UMV^\top) = VM^\top U^\top U MV^\top = VM^\top MV^\top$$

using $U^\top U = \mathbb{1}$, since U is orthogonal (Definition 1.20). Hence

$$\|UMV^\top\|_F^2 = \text{tr}(VM^\top MV^\top)$$

Applying the trace identity with $X := V$, $Y := M^\top MV^\top$,

$$\text{tr}(VM^\top MV^\top) = \text{tr}(XY) = \text{tr}(YX) = \text{tr}(M^\top MV^\top V) = \text{tr}(M^\top M)$$

using $V^\top V = \mathbb{1}$, since V is orthogonal, in the last step. So

$$\|UMV^\top\|_F^2 = \text{tr}(M^\top M) = \|M\|_F^2$$

and, as both sides are non-negative, taking square roots gives $\|UMV^\top\|_F = \|M\|_F$. Q.E.D.

Theorem 1.11 (Eckart-Young). *Let $A \in \text{Mat}(m, n, \mathbb{R})$ with SVD $A = U\Sigma V^\top$ (Theorem 1.10) and singular values $\sigma_1 \geq \dots \geq \sigma_r > 0$. For $0 \leq k < r$, we define*

$$A_k := \sum_{i=1}^k \sigma_i u_i v_i^\top$$

Then for every $B \in \text{Mat}(m, n, \mathbb{R})$ whose image has dimension at most k ,

$$\|A - B\|_F \geq \|A - A_k\|_F$$

so A_k is a best rank- k approximation of A in the Frobenius norm (Example 1.7). Moreover,

$$\|A - A_k\|_F = \sqrt{\sigma_{k+1}^2 + \dots + \sigma_r^2}$$

Proof. For any $M \in \text{Mat}(m, n, \mathbb{R})$, writing its own SVD $M = U_M \Sigma_M V_M^\top$ and applying Lemma 1.2 gives $\|M\|_F = \|\Sigma_M\|_F = \sqrt{\sum_i \sigma_i(M)^2}$. We use this fact twice.

First, $A - A_k = \sum_{i=k+1}^r \sigma_i u_i v_i^\top$ is itself an SVD, with singular values $\sigma_{k+1}, \dots, \sigma_r$, so

$$\|A - A_k\|_F = \sqrt{\sigma_{k+1}^2 + \dots + \sigma_r^2}$$

which is also the "moreover" clause of the theorem.

Now let B have $\text{rank}(B) \leq k$. By Weyl's inequality for singular values (Horn & Johnson 2013), $\sigma_{i+k}(A) \leq \sigma_i(A - B) + \sigma_{k+1}(B)$ for every $i \geq 1$; since B has at most k nonzero singular values, $\sigma_{k+1}(B) = 0$, so

$$\sigma_{i+k}(A) \leq \sigma_i(A - B).$$

Squaring and summing over $i = 1, \dots, r - k$,

$$\sum_{i=1}^{r-k} \sigma_{i+k}(A)^2 \leq \sum_{i=1}^{r-k} \sigma_i(A - B)^2 \leq \|A - B\|_F^2$$

the second inequality since the right sum is only part of the full non-negative sum defining $\|A - B\|_F^2$. Reindexing $j := i + k$, the left side is $\sum_{j=k+1}^r \sigma_j(A)^2 = \|A - A_k\|_F^2$. So

$$\|A - A_k\|_F^2 \leq \|A - B\|_F^2$$

and taking square roots gives $\|A - A_k\|_F \leq \|A - B\|_F$.

Q.E.D.

Chapter 2

Calculus and Optimisation

2.1 Multivariable Calculus

This section develops differentiation in several variables: the partial, directional and total derivative, the Jacobian and Hessian, the chain rule and the gradient's geometric meaning as the direction of steepest ascent.

The chain rule proved here is the mathematical content of backpropagation in the Neural Networks Chapter: computing gradients through a composition of layers is precisely repeated application of Theorem 2.2. Steepest ascent and the Hessian are used more immediately, in this same chapter, to justify the gradient descent update direction and to state its convergence conditions.

The gradient computations developed here are checked in C++ as `gradient_checker.cpp` (Appendix A.6), which compares an analytically computed gradient against a numerical finite-difference approximation.

2.1.1 Differentiability and the Jacobian

Definition 2.1 (Partial Derivative). Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and $a \in \mathbb{R}^n$. The *partial derivative* of f at a with respect to x_i is

$$\frac{\partial f}{\partial x_i}(a) := \lim_{h \rightarrow 0} \frac{f(a + he_i) - f(a)}{h}$$

when this limit exists, where e_i is the i -th standard basis vector.

Remark 2.1 (Partial Derivatives Are Weak). Existence of all partial derivatives at one point does not imply continuity there.

Example 2.1 (Discontinuity). Consider

$$f(x, y) := \begin{cases} \frac{xy}{x^2 + y^2} & (x, y) \neq (0, 0) \\ 0 & (x, y) = (0, 0) \end{cases}$$

with $\frac{\partial f}{\partial x}(0, 0) = \frac{\partial f}{\partial y}(0, 0) = 0$, yet f is not continuous at $(0, 0)$ (approaching along $y = x$ gives $f(x, x) = 1/2$ for all $x \neq 0$, not 0).

Definition 2.2 (Differentiability and the Total Derivative). Let $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and $a \in \mathbb{R}^n$. f is *differentiable* at a if there exists a linear map $L : \mathbb{R}^n \rightarrow \mathbb{R}^m$ such that

$$\lim_{h \rightarrow 0} \frac{\|f(a+h) - f(a) - L(h)\|}{\|h\|} = 0$$

If such L exists, it is unique and is called the (total) derivative of f at a , denoted $f'(a)$.

Proposition 2.1 (Jacobian Matrix). If $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is differentiable at a (Definition 2.2), then every partial derivative $\frac{\partial f_i}{\partial x_j}(a)$ exists, and $f'(a)$, expressed as a matrix in the standard bases, is given by

$$J_f(a) := f'(a) \quad (J_f(a))_{ij} = \frac{\partial f_i}{\partial x_j}(a)$$

The converse does not hold: by Remark 2.1, existence of all partial derivatives at a does not by itself imply f is differentiable at a .

Proof. Let $L = f'(a)$, so

$$\lim_{h \rightarrow 0} \frac{\|f(a+h) - f(a) - L(h)\|}{\|h\|} = 0$$

Fix $i \in \{1, \dots, n\}$. Since this limit holds for all $h \rightarrow 0$, it holds in particular along the path $h = te_i, t \rightarrow 0$; as $\|te_i\| = |t|$,

$$\lim_{t \rightarrow 0} \frac{\|f(a+te_i) - f(a) - tL(e_i)\|}{|t|} = 0 \quad \Longleftrightarrow \quad \lim_{t \rightarrow 0} \frac{f(a+te_i) - f(a)}{t} = L(e_i)$$

using linearity, $L(te_i) = tL(e_i)$. Applying Definition 2.1 componentwise to each f_j , the left side is the vector of partials $\frac{\partial f}{\partial x_i}(a) := (\frac{\partial f_1}{\partial x_i}(a), \dots, \frac{\partial f_m}{\partial x_i}(a))$; so this limit existing shows each $\frac{\partial f_j}{\partial x_i}(a)$ exists, with

$$L(e_i) = \frac{\partial f}{\partial x_i}(a)$$

Since $J_f(a)$'s i -th column is $L(e_i)$, this gives $(J_f(a))_{ji} = \frac{\partial f_j}{\partial x_i}(a)$ for every j . Since $i \in \{1, \dots, n\}$ was arbitrary, this holds for every column, giving the full matrix identity. This is exactly Proposition 2.1's formula, with the roles of i, j swapped since i here indexes the input coordinate and j the output coordinate. Q.E.D.

Definition 2.3 (Gradient). Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be differentiable at a , so $f'(a) : \mathbb{R}^n \rightarrow \mathbb{R}$ is linear. Define $v \in \mathbb{R}^n$ by $v_i := f'(a)(e_i)$ for each standard basis vector e_i . For any $h = \sum_i h_i e_i \in \mathbb{R}^n$, linearity of $f'(a)$ gives

$$f'(a)(h) = \sum_{i=1}^n h_i f'(a)(e_i) = \sum_{i=1}^n h_i v_i = \langle v, h \rangle$$

This v is the unique vector with $f'(a)(h) = \langle v, h \rangle$ for all h : if v' also satisfied this, then $\langle v - v', h \rangle = 0$ for all h , and taking $h = v - v'$ gives $\|v - v'\|^2 = 0$, so $v = v'$. This v is the *gradient* of f at a , written $\nabla f(a)$, and coincides with the column vector of partial derivatives, $\nabla f(a) = J_f(a)^\top$.

Computational Remark 2.1. The gradient computed by Definition 2.3 is checked in C++ as `gradient_checker.cpp` (Appendix A.6).

The checker uses the central difference, $\partial f / \partial x_i(a) \approx (f(a + he_i) - f(a - he_i)) / (2h)$, rather than the forward difference $(f(a + he_i) - f(a)) / h$. By Taylor's theorem, the forward difference's leading error is $O(h)$, while the central difference's second-order terms cancel between the $+h$ and $-h$ expansions, leaving $O(h^2)$, which is a strictly better approximation for the same h , at the cost of one extra evaluation per coordinate.

On the test function in `main()`, the maximum error was 5.42×10^{-10} , smaller than the naive $O(h^2) \approx 10^{-8}$ estimate would suggest. This is because the test function is an exact quadratic: its third derivative vanishes identically, so the $O(h^2)$ truncation term vanishes too, leaving only floating-point round-off.

Definition 2.4 (Hessian Matrix). Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be such that $\nabla f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is itself differentiable at a . The *Hessian* of f at a is the Jacobian of ∇f at a

$$H_f(a) := J_{\nabla f}(a) \in \text{Mat}(n, \mathbb{R}) \quad (H_f(a))_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}(a)$$

Theorem 2.1 (Symmetry of the Hessian Matrix). Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be twice differentiable at a with continuous second partial derivatives in a neighbourhood of a . Then $H_f(a)$ is symmetric, i.e.

$$\frac{\partial^2 f}{\partial x_i \partial x_j}(a) = \frac{\partial^2 f}{\partial x_j \partial x_i}(a) \quad \text{for all } i, j$$

Proof. See (Rudin 1976, Chapter 9)

Q.E.D.

2.1.2 The Chain Rule

Theorem 2.2 (Chain Rule). Let $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ be differentiable at $a \in \mathbb{R}^n$, and let $g : \mathbb{R}^m \rightarrow \mathbb{R}^k$ be differentiable at $f(a)$. Then $g \circ f : \mathbb{R}^n \rightarrow \mathbb{R}^k$ is differentiable at a , and

$$(g \circ f)'(a) = g'(f(a)) \circ f'(a)$$

Proof. Let $L := f'(a)$ and $M := g'(f(a))$. By differentiability, write

$$f(a + h) = f(a) + Lh + r(h) \quad g(f(a) + k) = g(f(a)) + Mk + s(k)$$

where $\|r(h)\|/\|h\| \rightarrow 0$ as $h \rightarrow 0$ and $\|s(k)\|/\|k\| \rightarrow 0$ as $k \rightarrow 0$ (with $r(0) = s(0) = 0$).

Set $k(h) := f(a + h) - f(a) = Lh + r(h)$. Then

$$g(f(a + h)) = g(f(a)) + Mk(h) + s(k(h)) = g(f(a)) + M(Lh) + (Mr(h) + s(k(h)))$$

So $(g \circ f)(a + h) = (g \circ f)(a) + (M \circ L)h + e(h)$, where $e(h) := Mr(h) + s(k(h))$. It remains to show $\|e(h)\|/\|h\| \rightarrow 0$.

Since M is linear (on $\mathbb{R}^m$, finite-dimensional), $\|Mr(h)\| \leq C\|r(h)\|$ for some constant C , so $\|Mr(h)\|/\|h\| \rightarrow 0$.

As $h \rightarrow 0$, $k(h) = Lh + r(h) \rightarrow 0$, and $\|k(h)\| \leq \|L\|\|h\| + \|r(h)\| \leq C'\|h\|$ for small h (some constant C' , since $r(h)/\|h\| \rightarrow 0$ is eventually bounded). If $k(h) \neq 0$,

$$\frac{\|s(k(h))\|}{\|h\|} = \frac{\|s(k(h))\|}{\|k(h)\|} \frac{\|k(h)\|}{\|h\|} \leq \frac{\|s(k(h))\|}{\|k(h)\|} C'$$

As $h \rightarrow 0$, $k(h) \rightarrow 0$, so $\|s(k(h))\|/\|k(h)\| \rightarrow 0$; hence this term $\rightarrow 0$ too. (If $k(h) = 0$, $s(k(h)) = 0$ and the bound holds trivially.)

So $\|e(h)\|/\|h\| \rightarrow 0$, meaning $g \circ f$ is differentiable at a with derivative $M \circ L = g'(f(a)) \circ f'(a)$. Q.E.D.

Corollary 2.1 (Matrix Form of the Chain Rule). *Under the hypotheses of Theorem 2.2, the Jacobians satisfy*

$$J_{g \circ f}(a) = J_g(f(a)) J_f(a)$$

where the right-hand side is ordinary matrix multiplication.

Proof. By Theorem 2.2, $(g \circ f)'(a) = g'(f(a)) \circ f'(a)$, a composition of linear maps. By Corollary 1.2, expressing both sides in the standard bases gives $J_{g \circ f}(a) = J_g(f(a)) J_f(a)$ directly. Q.E.D.

2.1.3 Gradient as Steepest Ascent

Definition 2.5 (Directional Derivative). Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and $a \in \mathbb{R}^n$, and let $u \in \mathbb{R}^n$ be a unit vector ($\|u\| = 1$). The *directional derivative* of f at a in direction u is

$$D_u f(a) := \lim_{t \rightarrow 0} \frac{f(a + tu) - f(a)}{t}$$

when this limit exists. Taking $u = e_i$ recovers the partial derivative, $D_{e_i} f(a) = \frac{\partial f}{\partial x_i}(a)$ (Definition 2.1), so the directional derivative generalises the partial derivative to an arbitrary direction.

Remark 2.2 (Directional Derivative via the Gradient). If f is differentiable at a , then $D_u f(a) = f'(a)(u)$ for any unit vector u : this is exactly the limit defining $f'(a)(h)$ (Definition 2.2) restricted to $h = tu$, $t \rightarrow 0$, the same specialisation used in the proof of Proposition 2.1. By Definition 2.3, $f'(a)(h) = \langle \nabla f(a), h \rangle$ for all h , so in particular

$$D_u f(a) = \langle \nabla f(a), u \rangle$$

Theorem 2.3 (Gradient as Direction of Steepest Ascent). *Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be differentiable at a with $\nabla f(a) \neq 0$. Among all unit vectors u , $D_u f(a)$ is maximised uniquely at*

$$u = \frac{\nabla f(a)}{\|\nabla f(a)\|}$$

with maximum value $D_u f(a) = \|\nabla f(a)\|$.

Proof. By Remark 2.2, $D_u f(a) = \langle \nabla f(a), u \rangle$. By Cauchy-Schwarz (Theorem 1.5),

$$D_u f(a) = \langle \nabla f(a), u \rangle \leq |\langle \nabla f(a), u \rangle| \leq \|\nabla f(a)\| \|u\| = \|\nabla f(a)\|$$

using $\|u\| = 1$. So $D_u f(a) \leq \|\nabla f(a)\|$ for every unit vector u , with $u = \nabla f(a)/\|\nabla f(a)\|$ achieving $D_u f(a) = \langle \nabla f(a), \nabla f(a)/\|\nabla f(a)\| \rangle = \|\nabla f(a)\|^2/\|\nabla f(a)\| = \|\nabla f(a)\|$, so the bound is attained and this is the maximum.

For uniqueness, suppose u is any unit vector achieving the maximum, $D_u f(a) = \|\nabla f(a)\|$. From the chain of inequalities above,

$$\|\nabla f(a)\| = D_u f(a) = \langle \nabla f(a), u \rangle \leq |\langle \nabla f(a), u \rangle| \leq \|\nabla f(a)\|$$

both inequalities must be equalities. The second is exactly the equality condition of Cauchy-Schwarz (Theorem 1.5), so $\nabla f(a)$ and u are linearly dependent: $u = t \nabla f(a)$ for some $t \in \mathbb{R}$. Since $\|u\| = 1$, $|t| = 1/\|\nabla f(a)\|$. The first equality, $\langle \nabla f(a), u \rangle = |\langle \nabla f(a), u \rangle|$, forces $\langle \nabla f(a), u \rangle \geq 0$; substituting $u = t \nabla f(a)$ gives $t \|\nabla f(a)\|^2 \geq 0$, so $t \geq 0$, hence $t = 1/\|\nabla f(a)\|$ (not its negative). So $u = \nabla f(a)/\|\nabla f(a)\|$ is the unique maximiser. Q.E.D.

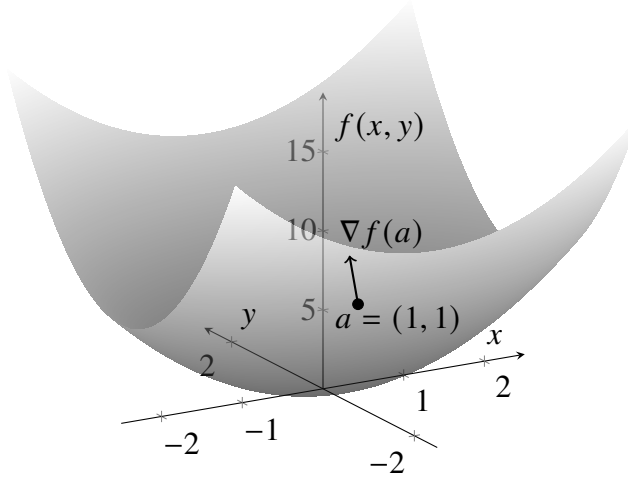


Figure 2.1: The surface $f(x, y) = x^2 + 2y^2$, with the point $a = (1, 1)$ marked and $\nabla f(a) = (2, 4)$ shown as a horizontal arrow anchored at a . By Theorem 2.3, this direction maximises the directional derivative $D_u f(a)$ among all unit vectors u .

2.2 Gradient Descent

This section covers convexity and L -smoothness, the two hypothesis classes gradient descent's guarantees rest on; the Descent Lemma, built from the multivariate Taylor theorem and a Hessian bound derived via the spectral theorem; and the convergence theorem itself, which combines the Descent Lemma with convexity in a telescoping argument to bound how quickly gradient descent approaches a minimiser.

This is where this Chapter's calculus machinery (the gradient and the spectral theorem via the Descent Lemma) becomes a working algorithm. Gradient descent is defined and proven to converge. This is the training algorithm Chapter 4 uses.

The algorithm is implemented from scratch in C++ as `gradient_descent.cpp` (Appendix A.7).

2.2.1 Convexity and L -Smoothness

Definition 2.6 (Convex Function). Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be differentiable. f is convex if

$$f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle \quad \text{for all } x, y \in \mathbb{R}^n$$

Remark 2.3. Definition 2.6 is the first-order characterisation of convexity. For differentiable f , it is equivalent to the more general line-segment definition of convexity, $f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$ for all x, y and $\lambda \in [0, 1]$. See (Boyd & Vandenberghe 2004, §3.1).

Definition 2.7 (Lipschitz Continuity and Gradient). A differentiable function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is Lipschitz continuous (L -continuous) if

$$|f(x) - f(y)| \leq L\|x - y\| \quad \text{for all } x, y \in \mathbb{R}^n$$

And, the function's Lipschitz gradient (L -smoothness) is

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\| \quad \text{for all } x, y \in \mathbb{R}^n$$

Example 2.2 ($\sin(x)$ is 1-Lipschitz continuous). Let $f(x) = \sin(x)$ on $\mathbb{R}$. Since $f'(x) = \cos(x)$ and $|\cos(x)| \leq 1$ for all x , f is Lipschitz with constant $L = 1$: for any $x, y \in \mathbb{R}$, by the Mean Value Theorem (Rudin 1976) there exists c between x and y with

$$|f(x) - f(y)| = |f'(c)||x - y| = |\cos(c)||x - y| \leq |x - y|$$

2.2.2 The Descent Lemma

Theorem 2.4 (Taylor's Theorem, second-order multivariate form). *Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ have continuous second partial derivatives on an open set containing the line segment between a and $a + h$. Then*

$$f(a + h) = f(a) + \langle \nabla f(a), h \rangle + \frac{1}{2} h^\top H_f(\xi) h$$

for some ξ on the segment between a and $a + h$, where H_f is the Hessian (Definition 2.4).

Proof. See (Rudin 1976, §9.8). Q.E.D.

Proposition 2.2 (Hessian Bound Given a Spectral Bound). *Let $H \in \text{Mat}(n, \mathbb{R})$ be symmetric with all eigenvalues $\lambda \leq L$. Then for every $h \in \mathbb{R}^n$,*

$$h^\top H h \leq L \|h\|^2.$$

Proof. By the Spectral Theorem (Theorem 1.9), H has an orthonormal eigenbasis $q_1, \dots, q_n$ with real eigenvalues $\lambda_1, \dots, \lambda_n$, each $\lambda_i \leq L$. Write $h = \sum_i c_i q_i$; then

$$h^\top H h = \sum_i \lambda_i c_i^2 \leq L \sum_i c_i^2 = L \|h\|^2$$

using orthonormality of $\{q_i\}$ for both equalities, and $\lambda_i \leq L$ together with $c_i^2 \geq 0$ for the inequality. Q.E.D.

Remark 2.4 (L -Smoothness and the Hessian). For $f : \mathbb{R}^n \rightarrow \mathbb{R}$ with continuous second partial derivatives, f is L -smooth (Definition 2.7) iff every eigenvalue of $H_f(x)$ is at most L . See (Boyd & Vandenberghe 2004, §9.1).

Lemma 2.1 (Descent Lemma). *Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be L -smooth (Definition 2.7) with continuous second partial derivatives on $\mathbb{R}^n$. Then for all $x, h \in \mathbb{R}^n$,*

$$f(x + h) \leq f(x) + \langle \nabla f(x), h \rangle + \frac{L}{2} \|h\|^2$$

Proof. By Taylor's theorem (Theorem 2.4), there exists ξ on the segment between x and $x + h$ with

$$f(x + h) = f(x) + \langle \nabla f(x), h \rangle + \frac{1}{2} h^\top H_f(\xi) h$$

Since f has continuous second partial derivatives, $H_f(\xi)$ is symmetric by Theorem 2.1. Since f is L -smooth, Remark 2.4 gives that every eigenvalue of $H_f(\xi)$ is at most L . So $H_f(\xi)$ satisfies the hypotheses of Proposition 2.2 (symmetric, eigenvalues $\leq L$), giving

$$h^\top H_f(\xi) h \leq L \|h\|^2$$

Substituting, we obtain

$$f(x + h) = f(x) + \langle \nabla f(x), h \rangle + \frac{1}{2} h^\top H_f(\xi) h \leq f(x) + \langle \nabla f(x), h \rangle + \frac{L}{2} \|h\|^2$$

Q.E.D.

2.2.3 Convergence of Gradient Descent

Definition 2.8 (Gradient Descent). Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be differentiable and let $x_0 \in \mathbb{R}^n$. For a fixed step size $\eta > 0$, the gradient descent sequence $(x_k)_{k \geq 0}$ is defined by

$$x_{k+1} := x_k - \eta \nabla f(x_k) \quad \text{for } k = 0, 1, 2, \dots$$

At each step, x_{k+1} moves from x_k in the direction $-\nabla f(x_k)$, the direction of steepest descent (Theorem 2.3), scaled by η .

Computational Remark 2.2. Theorem 2.5's bound is checked with `gradient_descent.cpp` (Appendix A.7). Two choices are worth justifying.

- i. **Step size** $\eta = 1/L$. This is the boundary value the theorem allows, giving the fastest guaranteed rate within its hypothesis.
- ii. **L computed numerically.** `computeL()` uses the Jacobi eigensolver to find A 's largest eigenvalue, straightforward only because the test function $f(x) = x^\top A x$ has constant Hessian $2A$, so L is exactly twice that eigenvalue (Remark 2.4). A general f 's Hessian varies with x , and finding a global Lipschitz constant for ∇f isn't normally this direct.

On this test case, gradient descent converged in 261 iterations to $f(x_K) \approx 4.25 \times 10^{-14}$, against the theorem's bound of ≈ 68.7 . The theorem holds, but the bound isn't tight: it's a worst-case guarantee over all convex, L -smooth functions, while this well-conditioned quadratic converges far faster than that guarantee requires.

Theorem 2.5 (Convergence of Gradient Descent). Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be convex (Definition 2.6), L -smooth (Definition 2.7) and have continuous second partial derivatives on $\mathbb{R}^n$; and let $(x_k)_{k \geq 0}$ be the gradient descent sequence (Definition 2.8) with step size $\eta \leq \frac{1}{L}$. Assume a minimiser $x^\star$ exists with $f(x^\star) \leq f(x)$ for all $x \in \mathbb{R}^n$. Then, for every natural number $K \geq 1$

$$f(x_K) - f(x^\star) \leq \frac{\|x_0 - x^\star\|^2}{2\eta K}$$

Proof. The following argument adapts a standard technique for gradient descent under convexity and L -smoothness; see (Bubeck 2015, §3.2) for the original.

By the Descent Lemma (Lemma 2.1) applied at $x = x_k$, $h = x_{k+1} - x_k = -\eta \nabla f(x_k)$ (Definition 2.8),

$$f(x_{k+1}) \leq f(x_k) + \langle \nabla f(x_k), -\eta \nabla f(x_k) \rangle + \frac{L}{2} \|\eta \nabla f(x_k)\|^2 = f(x_k) - \eta \left(1 - \frac{L\eta}{2}\right) \|\nabla f(x_k)\|^2$$

Since $\eta \leq 1/L$, $1 - L\eta/2 \geq 1/2$, so

$$f(x_{k+1}) \leq f(x_k) - \frac{\eta}{2} \|\nabla f(x_k)\|^2 \tag{A}$$

By Definition 2.8, $x_{k+1} - x^\star = (x_k - x^\star) - \eta \nabla f(x_k)$, so

$$\|x_{k+1} - x^\star\|^2 = \|x_k - x^\star\|^2 - 2\eta \langle \nabla f(x_k), x_k - x^\star \rangle + \eta^2 \|\nabla f(x_k)\|^2 \tag{B}$$

By Definition 2.6 applied at $x = x_k$, $y = x^\star$,

$$f(x^\star) \geq f(x_k) + \langle \nabla f(x_k), x^\star - x_k \rangle \implies \langle \nabla f(x_k), x_k - x^\star \rangle \geq f(x_k) - f(x^\star) \tag{C}$$

Substituting (C) into (B),

$$\|x_{k+1} - x^\star\|^2 \leq \|x_k - x^\star\|^2 - 2\eta(f(x_k) - f(x^\star)) + \eta^2 \|\nabla f(x_k)\|^2 \quad (\text{D})$$

From (A), $\eta \|\nabla f(x_k)\|^2 \leq 2(f(x_k) - f(x_{k+1}))$, so $\eta^2 \|\nabla f(x_k)\|^2 \leq 2\eta(f(x_k) - f(x_{k+1}))$. Substituting into (D),

$$\|x_{k+1} - x^\star\|^2 \leq \|x_k - x^\star\|^2 - 2\eta(f(x_k) - f(x^\star)) + 2\eta(f(x_k) - f(x_{k+1})) \quad (\text{E})$$

The two $f(x_k)$ terms in (E) cancel:

$$\|x_{k+1} - x^\star\|^2 \leq \|x_k - x^\star\|^2 - 2\eta(f(x_{k+1}) - f(x^\star))$$

reordering,

$$2\eta(f(x_{k+1}) - f(x^\star)) \leq \|x_k - x^\star\|^2 - \|x_{k+1} - x^\star\|^2 \quad (\text{F})$$

Summing (F) over $k = 0, \dots, K-1$, the right-hand side telescopes:

$$2\eta \sum_{k=0}^{K-1} (f(x_{k+1}) - f(x^\star)) \leq \|x_0 - x^\star\|^2 - \|x_K - x^\star\|^2 \leq \|x_0 - x^\star\|^2$$

By (A), $f(x_{k+1}) \leq f(x_k)$ for every k , so $f(x_k)$ is non-increasing; in particular $f(x_K) \leq f(x_{k+1})$ for every $k \leq K-1$.

Since $f(x_k)$ is non-increasing (shown above), every term in the preceding sum satisfies $f(x_{k+1}) - f(x^\star) \geq f(x_K) - f(x^\star)$, so

$$2\eta K(f(x_K) - f(x^\star)) \leq 2\eta \sum_{k=0}^{K-1} (f(x_{k+1}) - f(x^\star)) \leq \|x_0 - x^\star\|^2$$

Dividing by $2\eta K$,

$$f(x_K) - f(x^\star) \leq \frac{\|x_0 - x^\star\|^2}{2\eta K}$$

This concludes the proof.

Q.E.D.

2.3 Lagrange Multipliers

This section covers the constrained optimisation problem of minimising f subject to $g(x) = 0$, then develops the Implicit Function Theorem, the Lagrange multiplier theorem itself and the Lagrangian, closing with a short geometric interpretation.

This section reuses Chapter 1's Rank-Nullity Theorem and orthogonal complements. Its machinery is reused again in Chapter 3 to derive the categorical maximum likelihood estimator, where maximising a likelihood subject to probabilities summing to one is a direct application of the Lagrange multiplier theorem proved here.

The theorem is applied to a quadratic objective with linear equality constraints, reducing the problem to solving a single linear system, known as the Karush-Kuhn-Tucker (KKT) system. This system is solved in C++ as `kkt_solver.cpp` (Appendix A.8).

Theorem 2.6 (Implicit Function Theorem). *Let $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$ (with $m < n$) be differentiable with continuous partial derivatives and let $x^\star \in \mathbb{R}^n$ satisfy $F(x^\star) = 0$. Suppose the $m \times n$ Jacobian $DF(x^\star)$ has full row rank m .*

Then, after reordering coordinates so that $x = (y, z)$ with $y \in \mathbb{R}^{n-m}$, $z \in \mathbb{R}^m$ and the $m \times m$ block $\frac{\partial F}{\partial z}(x^\star)$ is invertible, write $x^\star = (y_0, z_0)$. There exists an open set $U \subseteq \mathbb{R}^{n-m}$ containing y_0 , an open set $V \subseteq \mathbb{R}^n$ containing $x^\star$ and a continuously differentiable function $h : U \rightarrow \mathbb{R}^m$ with $h(y_0) = z_0$, such that for all $(y, z) \in V$,

$$F(y, z) = 0 \iff z = h(y)$$

In particular, defining $\varphi(y) := (y, h(y))$ for $y \in U$ gives a local parametrisation of $\{x \in V : F(x) = 0\}$ near $x^\star$, with $\varphi(y_0) = x^\star$.

Proof. See (Rudin 1976, §9.28).

Q.E.D.

Theorem 2.7 (Lagrange Multipliers). *Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$ (with $m < n$) be differentiable with continuous partial derivatives. Suppose $x^\star$ is a local minimiser of f subject to $g(x) = 0$ and that $Dg(x^\star)$ has full row rank m . Then there exists $\lambda \in \mathbb{R}^m$ such that*

$$\nabla f(x^\star) = Dg(x^\star)^\top \lambda$$

This is a necessary condition for a constrained local minimum and does not assert the converse. Satisfying it does not by itself imply $x^\star$ is a minimiser.

Proof. Since $g(x^\star) = 0$ and $Dg(x^\star)$ has full row rank m , Theorem 2.6 gives a local parametrisation $\varphi(y) = (y, h(y))$ of $\{x : g(x) = 0\}$ near $x^\star$, with $\varphi(y_0) = x^\star$.

Differentiating $g(\varphi(y)) = 0$ at y_0 (Chain Rule, Theorem 2.2) gives $Dg(x^\star)D\varphi(y_0) = 0$, so $\text{im}(D\varphi(y_0)) \subseteq \ker(Dg(x^\star))$. By Rank-Nullity (Theorem 1.2), $\dim \ker(Dg(x^\star)) = n - m$; by its block form, $D\varphi(y_0)$ has rank $n - m$ too. Equal dimensions plus containment gives equality.

Let $v \in \ker(Dg(x^\star))$ be arbitrary. Since $\text{im}(D\varphi(y_0)) = \ker(Dg(x^\star))$, there exists u with $D\varphi(y_0)u = v$. Set $\gamma(t) := \varphi(y_0 + tu)$. Then $\gamma(0) = x^\star$, γ stays feasible, and $\gamma'(0) = v$.

Since $x^\star$ locally minimises f subject to $g = 0$ and γ stays feasible, $t = 0$ locally minimises $f(\gamma(t))$, so its derivative vanishes there:

$$0 = \frac{d}{dt} \Big|_{t=0} f(\gamma(t)) = \langle \nabla f(x^\star), \gamma'(0) \rangle = \langle \nabla f(x^\star), v \rangle$$

This holds for every $v \in \ker(Dg(x^\star))$, so $\nabla f(x^\star) \in \ker(Dg(x^\star))^\perp$. By Corollary 1.4, $\ker(Dg(x^\star))^\perp = \text{im}(Dg(x^\star)^\top)$, so $\nabla f(x^\star) = Dg(x^\star)^\top \lambda$ for some $\lambda \in \mathbb{R}^m$. Q.E.D.

Computational Remark 2.3. Applied to a quadratic objective with linear equality constraints, Theorem 2.7 reduces to solving a KKT linear system, computed here in `kkt_solver.cpp` (Appendix A.8).

- i. **LU solver.** With the problem reduced to $Kz = d$, it is an ordinary linear system, so the LU solver from Section 1.2 is reused directly.
- ii. **Scoped loops.** `assembleKKT()` fills the K matrix's four blocks with four separate loops rather than one loop with branching, matching the block structure directly and avoiding a branch check on every entry of the already-zero block.

Geometrically, $Q = I_2$, $A = (1 \ 1)$, $b = 1$, $c = 0$ asks for the point on the line $x_1 + x_2 = 1$ closest to the origin, which is exactly what minimising $\frac{1}{2}\|x\|^2$ subject to $x_1 + x_2 = 1$ means. By symmetry, that point should be $(0.5, 0.5)$: this is also the image that Remark 2.6 describes, a circle around the origin growing until it's tangent to the line. `solveKKT()` returns exactly that, $x = (0.5, 0.5)$, $\lambda = 0.5$, with both residuals vanishing exactly, since every value here is exactly representable in floating point.

Remark 2.5 (The Lagrangian). The conclusion of Theorem 2.7 is often restated by defining the *Lagrangian* $\mathcal{L} : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}$,

$$\mathcal{L}(x, \lambda) := f(x) - \langle \lambda, g(x) \rangle$$

so that $\nabla_x \mathcal{L}(x^*, \lambda) = \nabla f(x^*) - Dg(x^*)^\top \lambda = 0$ is exactly the theorem's conclusion and $\nabla_\lambda \mathcal{L}(x^*, \lambda) = -g(x^*) = 0$ recovers feasibility. So x^* (together with the λ given by the theorem) is a stationary point of $\mathcal{L}$.

Remark 2.6 (Geometric Interpretation). The proof above shows $\ker(Dg(x^*)) \subseteq \{v : \langle \nabla f(x^*), v \rangle = 0\}$ (Remark 2.2): every tangent direction of the constraint surface $\{g = 0\}$ is also tangent to f 's level set through x^* . So the contours of f touch the constraint surface at x^* rather than crossing it. The tangent spaces don't coincide in general, however: since $\ker(Dg(x^*))$ has dimension $n - m$ while the level set's tangent hyperplane has dimension $n - 1$, they coincide only when $m = 1$.

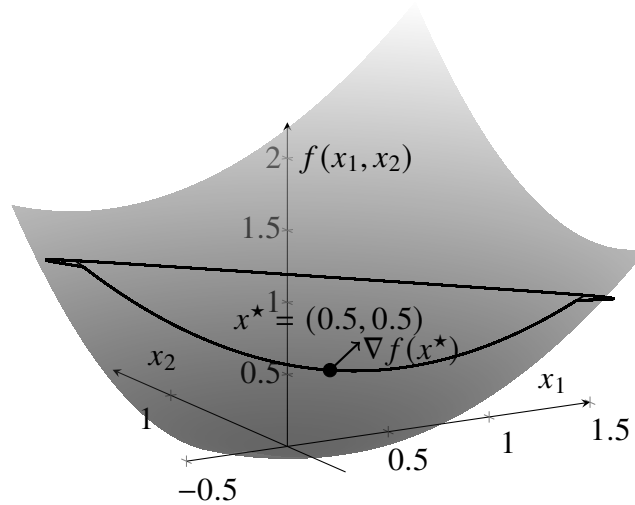


Figure 2.2: The surface $f(x_1, x_2) = \frac{1}{2}(x_1^2 + x_2^2)$ with the constraint curve $x_1 + x_2 = 1$ lifted onto it, tracing the objective's value as x_1, x_2 move along the constraint. At the marked point $x^* = (0.5, 0.5)$, where $f(x^*) = 0.25$, the curve's tangent direction is horizontal, zero slope along the constraint, which is exactly why it is the constrained minimum. The arrow shows $\nabla f(x^*) = (0.5, 0.5)$: it points in the same direction as the constraint line's own normal $\nabla g(x^*) = (1, 1)$, the literal content of $\nabla f(x^*) = Dg(x^*)^\top \lambda$ (Theorem 2.7).

Chapter 3

Probability and Statistics

3.1 Probability Foundations

This section develops probability spaces, conditional probability and independence, the Bernoulli and categorical distributions, random variables and expectation, and closes with Jensen's inequality and KL divergence.

The categorical distribution developed here is the object this Chapter builds its maximum likelihood estimator toward and the parameter that estimator fits is precisely what Chapter 4's softmax output layer models.

3.1.1 Probability Spaces and Kolmogorov's Axioms

Definition 3.1 (Probability Space). A probability space is a pair (Ω, P) , where Ω is a finite (or countable) set (known as the sample space) and $P : \Omega \rightarrow [0, 1]$ (known as the probability mass function) satisfies

$$\sum_{\omega \in \Omega} P(\omega) = 1$$

A subset $A \subseteq \Omega$ is called an event and its probability is defined by

$$P(A) := \sum_{\omega \in A} P(\omega)$$

Proposition 3.1 (Kolmogorov's Axioms). *Let (Ω, P) be a probability space (Definition 3.1). Then*

(K_1) $P(A) \geq 0$ for every event $A \subseteq \Omega$.

(K_2) $P(\Omega) = 1$.

(K_3) For any finite (or countable) collection of pairwise, disjoint events $A_1, A_2, \dots \subseteq \Omega$,

$$P\left(\bigcup_i A_i\right) = \sum_i P(A_i)$$

Proof. (K_1) Immediate from Definition 3.1: $P(A) = \sum_{\omega \in A} P(\omega)$ is a sum of non-negative terms, since $P(\omega) \in [0, 1]$ for each ω .

(K₂) By Definition 3.1, $P(\Omega) = \sum_{\omega \in \Omega} P(\omega) = 1$.

(K₃) Since the A_i are pairwise disjoint, every $\omega \in \bigcup_i A_i$ belongs to exactly one A_i , so

$$P\left(\bigcup_i A_i\right) = \sum_{\omega \in \bigcup_i A_i} P(\omega) = \sum_i \sum_{\omega \in A_i} P(\omega) = \sum_i P(A_i)$$

rearranging the (at most countable) sum, valid since every term is non-negative. Q.E.D.

Corollary 3.1 (Finite Additivity for Disjoint Events). *If $A, B \subseteq \Omega$ are disjoint events, then $P(A \cup B) = P(A) + P(B)$.*

Proof. First, $P(\emptyset) = \sum_{\omega \in \emptyset} P(\omega) = 0$, an empty sum, by Definition 3.1. The result then follows from (K₃), taking the collection $A_1 = A$, $A_2 = B$, $A_i = \emptyset$ for $i > 2$: these are pairwise disjoint, and

$$P(A \cup B) = P\left(\bigcup_i A_i\right) = \sum_i P(A_i) = P(A) + P(B) + \sum_{i>2} P(\emptyset) = P(A) + P(B)$$

Q.E.D.

Corollary 3.2 (Complement Rule). *For any event $A \subseteq \Omega$, $P(A^c) = 1 - P(A)$, where $A^c := \Omega \setminus A$.*

Proof. A and A^c are disjoint with $A \cup A^c = \Omega$. By Corollary 3.1 and (K₂),

$$1 = P(\Omega) = P(A \cup A^c) = P(A) + P(A^c)$$

so $P(A^c) = 1 - P(A)$. Q.E.D.

3.1.2 Conditional Probability and Independence

Definition 3.2 (Conditional Probability). Let (Ω, P) be a probability space and $A, B \subseteq \Omega$ events with $P(B) > 0$. The conditional probability of A given B is

$$P(A | B) := \frac{P(A \cap B)}{P(B)}$$

This is undefined when $P(B) = 0$.

Corollary 3.3 (Multiplication Rule). *If $P(B) > 0$, then $P(A \cap B) = P(A | B)P(B)$.*

Proof. This is an immediate rearrangement of Definition 3.2. Q.E.D.

Definition 3.3 (Independence of Events). Events $A, B \subseteq \Omega$ are independent if

$$P(A \cap B) = P(A)P(B)$$

Remark 3.1. If $P(B) > 0$, Definition 3.3 is equivalent to $P(A | B) = P(A)$: dividing $P(A \cap B) = P(A)P(B)$ by $P(B)$ (Definition 3.2) gives $P(A | B) = P(A)$ directly, and conversely multiplying $P(A | B) = P(A)$ by $P(B)$ recovers $P(A \cap B) = P(A)P(B)$ via Corollary 3.3.

Definition 3.4 (Random Variable). Let (Ω, P) be a probability space. A random variable is a function $X : \Omega \rightarrow \mathbb{R}$. For $x \in \mathbb{R}$, write

$$P(X = x) := P(\{\omega \in \Omega : X(\omega) = x\})$$

Definition 3.5 (Independence of Random Variables). Random variables X, Y on (Ω, P) are independent if

$$P(X = x, Y = y) = P(X = x) P(Y = y) \quad \text{for all } x, y$$

where $P(X = x, Y = y) := P(\{\omega : X(\omega) = x\} \cap \{\omega : Y(\omega) = y\})$.

More generally, random variables $X_1, \dots, X_n$ are independent if

$$P(X_1 = x_1, \dots, X_n = x_n) = \prod_{i=1}^n P(X_i = x_i) \quad \text{for all } x_1, \dots, x_n$$

$X_1, \dots, X_n$ are independent and identically distributed (i.i.d.) if they are independent and $P(X_i = x) = P(X_j = x)$ for all i, j and all x (they share a common probability mass function).

3.1.3 Bernoulli and Categorical Distributions

Definition 3.6 (Categorical Distribution). Let (Ω, P) be a probability space and $X : \Omega \rightarrow \{1, \dots, K\}$ a random variable. X is categorical with parameter $p = (p_1, \dots, p_K)$, where $p_k \geq 0$ and $\sum_{k=1}^K p_k = 1$, if

$$P(X = k) = p_k \quad \text{for each } k = 1, \dots, K$$

Remark 3.2. This is the distribution the rest of the chapter is aimed at: its parameter p is what the maximum likelihood estimator to come will fit, and what Chapter 4's softmax layer produces as its output.

Remark 3.3 (Bernoulli Distribution). X is Bernoulli(p) if it is categorical (Definition 3.6) with $K = 2$, $p_1 = p$, $p_2 = 1 - p$; relabelling the outcomes $\{1, 2\} \rightarrow \{0, 1\}$ recovers the familiar $\{0, 1\}$ -valued convention, $P(X = 1) = p$, $P(X = 0) = 1 - p$.

Remark 3.4 (I.I.D. Sample Counts). If $X_1, \dots, X_n$ are i.i.d. Categorical(p) (Definition 3.5, Definition 3.6), write

$$n_k := |\{i : X_i = k\}|$$

for the number of samples taking value k , so that $n_1 + \dots + n_K = n$.

3.1.4 Random Variables and Expectation

Definition 3.7 (Expectation). Let (Ω, P) be a probability space and $X : \Omega \rightarrow \mathbb{R}$ a random variable with finite range. The expectation of X is

$$\mathbb{E}[X] := \sum_{\omega \in \Omega} X(\omega) P(\omega)$$

Lemma 3.1 (Expectation via the Distribution). Let $X : \Omega \rightarrow \mathbb{R}$ have finite range $\{x_1, \dots, x_r\}$, and let $g : \{x_1, \dots, x_r\} \rightarrow \mathbb{R}$. Then

$$\mathbb{E}[g(X)] = \sum_{i=1}^r g(x_i) P(X = x_i)$$

Proof. The preimages $\{\omega : X(\omega) = x_i\}$ for $i = 1, \dots, r$ partition Ω , since X has finite range $\{x_1, \dots, x_r\}$ and every $\omega \in \Omega$ satisfies $X(\omega) = x_i$ for exactly one i . Grouping the sum in

Definition 3.7 by which part of the partition each ω falls into,

$$\begin{aligned}
\mathbb{E}[g(X)] &= \sum_{\omega \in \Omega} g(X(\omega))P(\omega) \\
&= \sum_{i=1}^r \sum_{\omega: X(\omega)=x_i} g(x_i)P(\omega) \\
&= \sum_{i=1}^r g(x_i) \sum_{\omega: X(\omega)=x_i} P(\omega) \\
&= \sum_{i=1}^r g(x_i)P(X = x_i)
\end{aligned}$$

using $P(X = x_i) = \sum_{\omega: X(\omega)=x_i} P(\omega)$ (Definition 3.4) in the last step. Q.E.D.

Corollary 3.4 (Linearity of Expectation). *For random variables X, Y with finite range and $a, b \in \mathbb{R}$,*

$$\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$$

Proof. This is immediate from Definition 3.7 and linearity of summation:

$$\mathbb{E}[aX + bY] = \sum_{\omega \in \Omega} (aX(\omega) + bY(\omega))P(\omega) = a \sum_{\omega \in \Omega} X(\omega)P(\omega) + b \sum_{\omega \in \Omega} Y(\omega)P(\omega) = a\mathbb{E}[X] + b\mathbb{E}[Y]$$

Q.E.D.

3.1.5 Jensen's Inequality and Kullback-Leibler Divergence

Theorem 3.1 (Jensen's Inequality). *Let $X : \Omega \rightarrow \mathbb{R}$ have finite range and let $\varphi : \mathbb{R} \rightarrow \mathbb{R}$ be convex and differentiable (Remark 2.3). Then*

$$\varphi(\mathbb{E}[X]) \leq \mathbb{E}[\varphi(X)]$$

This is the finite discrete form of Jensen's inequality.

Proof. Let $\{x_1, \dots, x_r\}$ be the range of X , with $p_i := P(X = x_i)$. By Lemma 3.1, $\mathbb{E}[X] = \sum_i p_i x_i$ and $\mathbb{E}[\varphi(X)] = \sum_i p_i \varphi(x_i)$, so it suffices to show, for any $x_1, \dots, x_r \in \mathbb{R}$ and $p_i \geq 0$ with $\sum_i p_i = 1$,

$$\varphi\left(\sum_i p_i x_i\right) \leq \sum_i p_i \varphi(x_i)$$

by induction on r .

Base case: $p_1 = 1$, so both sides equal $\varphi(x_1)$.

Inductive step: assume the claim for $r - 1$ points, and take $x_1, \dots, x_r, p_1, \dots, p_r$ as above. If $p_r = 1$ the claim is immediate as in the base case, so assume $p_r < 1$ and set $\lambda := 1 - p_r \in (0, 1]$, $q_i := p_i / \lambda$ for $i < r$; then $q_i \geq 0$, $\sum_{i < r} q_i = 1$, and writing $z := \sum_{i < r} q_i x_i$,

$$\sum_i p_i x_i = \lambda z + (1 - \lambda)x_r$$

By convexity (Remark 2.3) then the inductive hypothesis,

$$\varphi\left(\sum_i p_i x_i\right) = \varphi(\lambda z + (1 - \lambda)x_r) \leq \lambda \varphi(z) + (1 - \lambda)\varphi(x_r) \leq \lambda \sum_{i < r} q_i \varphi(x_i) + p_r \varphi(x_r) = \sum_i p_i \varphi(x_i)$$

using $\lambda q_i = p_i$. Q.E.D.

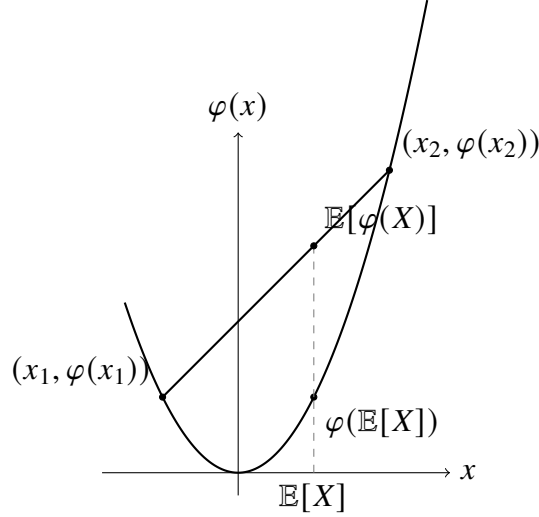


Figure 3.1: Jensen's inequality (Theorem 3.1) for $\varphi(x) = x^2$: the curve lies below its own chord. With $x_1 = -1, x_2 = 2$ and weights $p_1 = \frac{1}{3}, p_2 = \frac{2}{3}$, $\mathbb{E}[X] = 1$, so $\varphi(\mathbb{E}[X]) = 1$ while $\mathbb{E}[\varphi(X)] = 3$, the chord's height at the same point.

Corollary 3.5 (Jensen's Inequality on an Interval). *Let $I \subseteq \mathbb{R}$ be an interval, let $X : \Omega \rightarrow \mathbb{R}$ have finite range contained in I , and let $\varphi : I \rightarrow \mathbb{R}$ be convex on I (i.e. satisfy Remark 2.3's inequality for all $x, y \in I, \lambda \in [0, 1]$). Then*

$$\varphi(\mathbb{E}[X]) \leq \mathbb{E}[\varphi(X)]$$

Proof. The induction in the proof of Theorem 3.1 only ever evaluates φ at convex combinations of points in the range of X , all of which lie in I (since I is an interval and the range of X is a finite subset of I), so the same argument applies verbatim with φ restricted to I . Q.E.D.

Definition 3.8 (Kullback-Leibler Divergence). Let P, Q be probability distributions on the same finite set, with $Q(x) > 0$ whenever $P(x) > 0$. The Kullback-Leibler (KL) divergence of Q from P is

$$D_{KL}(P||Q) := \sum_x P(x) \log \frac{P(x)}{Q(x)}$$

where the sum is over x with $P(x) > 0$.

Proposition 3.2 (Non-negativity of KL Divergence). *For any two probability distributions P, Q as in Definition 3.8,*

$$D_{KL}(P||Q) \geq 0$$

Proof. Let $X \sim P$ and $Y := Q(X)/P(X)$, a random variable with finite range contained in $(0, \infty)$. By Lemma 3.1,

$$\mathbb{E}_P[Y] = \sum_x P(x) \cdot \frac{Q(x)}{P(x)} = \sum_x Q(x) \leq 1$$

the sum being a sub-sum, over x with $P(x) > 0$, of Q 's total mass, which is at most 1.

The function $\varphi(t) = -\log t$ is convex on $(0, \infty)$ (Boyd & Vandenberghe 2004, §3.1.5). Since Y 's range is a finite subset of $(0, \infty)$, Corollary 3.5 applies:

$$-\log(\mathbb{E}_P[Y]) \leq \mathbb{E}_P[-\log Y]$$

Since $-\log$ is decreasing and $\mathbb{E}_P[Y] \leq 1$, $-\log(\mathbb{E}_P[Y]) \geq -\log(1) = 0$. Combined with the previous line,

$$\mathbb{E}_P[-\log Y] \geq -\log(\mathbb{E}_P[Y]) \geq 0$$

By Lemma 3.1,

$$\mathbb{E}_P[-\log Y] = \sum_x P(x) \left(-\log \frac{Q(x)}{P(x)} \right) = \sum_x P(x) \log \frac{P(x)}{Q(x)} = D_{KL}(P\|Q)$$

So $D_{KL}(P\|Q) \geq 0$.

Q.E.D.

3.2 Maximum Likelihood Estimation and Cross-Entropy

This section develops the general maximum likelihood framework, specialises it to i.i.d. categorical data and bridges the resulting log-likelihood to cross-entropy and KL divergence.

This is the result Chapter 4's loss function is built on: cross-entropy loss is what maximising likelihood on categorical data reduces to.

The algorithm is implemented from scratch in C++ as `categorical_mle.cpp` (Appendix A.9).

3.2.1 Maximum Likelihood Estimation

Definition 3.9 (Likelihood Function). Let $\{P_\theta\}_{\theta \in \Theta}$ be a family of distributions on the same finite set, indexed by a parameter θ ranging over some set Θ , and let $X_1, \dots, X_n$ be i.i.d. (Definition 3.5) with common distribution P_θ for some $\theta \in \Theta$. Given realised values $x_1, \dots, x_n$ of $X_1, \dots, X_n$, the likelihood function is

$$L(\theta) := \prod_{i=1}^n P(X_i = x_i)$$

viewed as a function of θ (which determines the distribution against which each $P(X_i = x_i)$ is computed), with the realised values $x_1, \dots, x_n$ fixed.

Definition 3.10 (Log-Likelihood). The log-likelihood is

$$\ell(\theta) := \log L(\theta) = \sum_{i=1}^n \log P(X_i = x_i)$$

Remark 3.5. On any domain where $L(\theta) > 0$ for every θ under consideration, $\log$ is strictly increasing on the range of L , so $\arg \max_\theta L(\theta) = \arg \max_\theta \log L(\theta) = \arg \max_\theta \ell(\theta)$: maximising L and maximising ℓ give the same maximisers. This is why it suffices to work with $\ell(\theta)$ which is typically more convenient since it turns the product in Definition 3.9 into a sum.

Definition 3.11 (Maximum Likelihood Estimator). The maximum likelihood estimator (MLE) of θ , given realised values $x_1, \dots, x_n$ of $X_1, \dots, X_n$, is

$$\hat{\theta} := \arg \max_{\theta \in \Theta} L(\theta) = \arg \max_{\theta \in \Theta} \ell(\theta)$$

where the equality is justified by Remark 3.5.

Proposition 3.3 (Likelihood of an I.I.D. Categorical Sample). *Let $X_1, \dots, X_n$ be i.i.d. Categorical(p) (Definition 3.6, Definition 3.5), with realised values $x_1, \dots, x_n \in \{1, \dots, K\}$, and n_k as in Remark 3.4. Then*

$$L(p) = \prod_{k=1}^K p_k^{n_k}, \quad \ell(p) = \sum_{k=1}^K n_k \log p_k$$

Proof. By Definition 3.9 and Definition 3.6 ($P(X_i = x_i) = p_{x_i}$ for each i),

$$L(p) = \prod_{i=1}^n P(X_i = x_i) = \prod_{i=1}^n p_{x_i}$$

Group the factors by the value x_i takes: since exactly n_k of the indices i satisfy $x_i = k$, for each $k = 1, \dots, K$,

$$\prod_{i=1}^n p_{x_i} = \prod_{k=1}^K \prod_{i:x_i=k} p_{x_i} = \prod_{k=1}^K \prod_{i:x_i=k} p_k = \prod_{k=1}^K p_k^{n_k}$$

using that $p_{x_i} = p_k$ for every i with $x_i = k$, and there are n_k such factors. So $L(p) = \prod_{k=1}^K p_k^{n_k}$. Taking logs (Definition 3.10),

$$\ell(p) = \log L(p) = \log \prod_{k=1}^K p_k^{n_k} = \sum_{k=1}^K \log(p_k^{n_k}) = \sum_{k=1}^K n_k \log p_k$$

Q.E.D.

We solve this via Lagrange multipliers, converting the maximisation of $\ell(p) = \sum_{k=1}^K n_k \log p_k$ subject to $g(p) := \sum_k p_k - 1 = 0$ into a minimisation, as required by Theorem 2.7's hypotheses.

Proposition 3.4 (MLE for the Categorical Distribution). *Let $X_1, \dots, X_n$ be i.i.d. Categorical(p) (Definition 3.6), with realised counts n_k (Remark 3.4) satisfying $n_k > 0$ for every $k = 1, \dots, K$. Then the maximum likelihood estimator (Definition 3.11) of p is*

$$\hat{p}_k = \frac{n_k}{n} \quad \text{for each } k = 1, \dots, K$$

Proof. By Proposition 3.3, $\ell(p) = \sum_k n_k \log p_k$, defined on $\{p : p_k > 0 \forall k\}$. Set $f := -\ell$; maximising ℓ subject to $g = 0$ is minimising f subject to $g = 0$. Theorem 2.7 applies on this open domain, since its proof only needs differentiability locally, not on all of $\mathbb{R}^K$. Since $Dg(p) = (1, \dots, 1)$ has rank 1 = m (using $K \geq 2$), the rank hypothesis holds.

At a local maximiser p^* , $\nabla f(p^*) = Dg(p^*)^\top \lambda$ for some $\lambda \in \mathbb{R}$, i.e. $-n_k/p_k^* = \lambda$ for each k , so $p_k^* = -n_k/\lambda$. The constraint $\sum_k p_k^* = 1$ gives $-n/\lambda = 1$, so $\lambda = -n$ (using $n = \sum_k n_k \geq 1$), hence $p_k^* = n_k/n = \hat{p}_k$.

$-\log t$ is convex on $(0, \infty)$, so $p \mapsto n_k(-\log p_k)$ (each coordinate function composed with a projection, hence convex on $\mathbb{R}^K$) is convex, and $f(p) = \sum_k n_k(-\log p_k)$ is convex as a non-negative-weighted sum of such terms. Since g is affine, a point satisfying the Lagrange stationarity condition for a convex objective over an affine constraint set is automatically a global minimiser (Boyd & Vandenberghe 2004). As $\hat{p}$ satisfies stationarity, it globally minimises f and so globally maximises ℓ : $\hat{p}_k = n_k/n$ is the MLE. Q.E.D.

Computational Remark 3.1. `categorical_mle.cpp` (Appendix A.9) implements sampling, counting and MLE fitting for the categorical distribution.

- i. **Cumulative-sum sampling.** `sampleCategorical()` partitions $[0, 1)$ into intervals of length $p_1, \dots, p_K$ and returns the index whose interval contains a draw $u \sim \text{Uniform}(0, 1)$. This is a direct computational reading of $P(X = k) = p_k$ (Definition 3.6), not an approximation of it.
- ii. **$K-1$ fallback.** `samples[i]` defaults to $K-1$ before the scan. Since $\text{cdf}[K-1] = \sum_k p_k = 1$ always covers u , the scan can only miss due to floating-point rounding. Any u that triggers this genuinely belongs to the last interval anyway, so the fallback is correct, not arbitrary.

Verified in `main()` by perturbing away from $\hat{p}$ in three directions: $\ell(\hat{p}) = -1031.78$, against -1032.04 , -1032.19 , -1032.14 perturbed, which is consistent with $\hat{p}$ being the maximiser (Proposition 3.4).

3.2.2 Cross-Entropy

Definition 3.12 (Cross-Entropy). Let P, Q be probability distributions on the same finite set, with $Q(x) > 0$ whenever $P(x) > 0$. The cross-entropy of Q relative to P is

$$H(P, Q) := - \sum_x P(x) \log Q(x)$$

where the sum is over x with $P(x) > 0$.

Remark 3.6 (The Empirical Distribution). The MLE $\hat{p}_k = n_k/n$ from Proposition 3.4 is also the empirical distribution of the sample: the categorical distribution (Definition 3.6) placing mass n_k/n on outcome k , i.e. the proportion of the sample actually observed to equal k . The same object $\hat{p}$ therefore plays two roles: it is the parameter estimate produced by maximum likelihood, and separately, it is the frequency distribution of the observed data itself.

Proposition 3.5 (Log-Likelihood as Cross-Entropy). Let $X_1, \dots, X_n$ be i.i.d. $\text{Categorical}(p)$ with realised counts n_k (Remark 3.4), $n_k > 0$ for every k , and let $\hat{p}$ be the empirical distribution (Remark 3.6). Then, for p in the domain $\{p : p_k > 0 \forall k\}$,

$$-\frac{1}{n} \ell(p) = H(\hat{p}, p)$$

Proof. By Proposition 3.3, $\ell(p) = \sum_k n_k \log p_k$. Substituting $n_k = n\hat{p}_k$,

$$-\frac{1}{n} \ell(p) = -\frac{1}{n} \sum_k n\hat{p}_k \log p_k = - \sum_k \hat{p}_k \log p_k = H(\hat{p}, p)$$

by Definition 3.12, noting the support condition there ($p_k > 0$ whenever $\hat{p}_k > 0$) holds automatically on this domain. Q.E.D.

Corollary 3.6 (Maximum Likelihood is Minimum Cross-Entropy). With notation as in Proposition 3.5,

$$\arg \max_p \ell(p) = \arg \min_p H(\hat{p}, p)$$

Proof. By Proposition 3.5, $H(\hat{p}, p) = -\ell(p)/n$. Since $n > 0$, dividing by n and negating are each strictly monotonic (order-reversing, in the case of negation), so they do not change which p attains an extremum, only whether it is a maximum or minimum: maximising $\ell(p)$ is exactly minimising $-\ell(p)/n = H(\hat{p}, p)$. Q.E.D.

Remark 3.7 (Cross-Entropy and KL Divergence). Expanding Definition 3.8,

$$D_{KL}(\hat{p}||p) = \sum_k \hat{p}_k \log \hat{p}_k - \sum_k \hat{p}_k \log p_k$$

so, by Definition 3.12,

$$H(\hat{p}, p) = - \sum_k \hat{p}_k \log \hat{p}_k + D_{KL}(\hat{p}||p)$$

The first term does not depend on p , so minimising $H(\hat{p}, p)$ over p is the same problem as minimising $D_{KL}(\hat{p}||p)$ over p . By Proposition 3.2, $D_{KL}(\hat{p}||p) \geq 0$ for every such p , giving a lower bound $H(\hat{p}, p) \geq - \sum_k \hat{p}_k \log \hat{p}_k$; this states the inequality only, with no claim about when equality holds.

Remark 3.8 (Single Distribution to a Model per Example). Proposition 3.4 fits one distribution $\hat{p}$ to an entire sample, in closed form. Chapter 4 instead fits a distribution $Q_\theta(x) = \text{softmax}(f_\theta(x))$ per input x , sharing parameters θ across examples. No closed form exists for this, so θ is instead fit by gradient descent (Section 2.2), minimising the same cross-entropy objective derived here, one step at a time.

Chapter 4

Neural Networks

4.1 Network Architecture

This section defines affine maps and shows why composing them alone never gives more expressive power than a single affine map, introduces componentwise activation functions and the resulting feedforward network architecture, and closes with the softmax output layer that turns a network's raw output into a categorical distribution.

This is where the pieces built across the previous three chapters assemble into a single object: a network whose expressive power comes specifically from nonlinearity, not depth alone (Remark 4.2), and whose output is provably a categorical distribution rather than merely something shaped like one (Proposition 4.2), which is what licenses training it by minimising cross-entropy as derived in Chapter 3 and previewed in Remark 4.6.

The forward pass, which is the system that evaluates the network layer by layer from input to output (Remark 4.4), is implemented from scratch in C++ as `network.hpp` (Appendix A.10).

4.1.1 Affine Maps

Definition 4.1 (Affine Map). Let $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ be a linear operator (Definition 1.9), represented in the standard bases by a matrix $A \in \text{Mat}(m, n, \mathbb{R})$ (Proposition 1.5), and let $b \in \mathbb{R}^m$ be a fixed vector. The map $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$ defined by

$$F(x) := Ax + b$$

is called an affine map. A is the linear part of F and b is its translation (or bias).

Remark 4.1. An affine map $F(x) = Ax + b$ (Definition 4.1) is a linear operator (Definition 1.9) only when $b = 0$: for $b \neq 0$, $F(0) = b \neq 0$, violating linearity, which forces $T(0) = 0$ (take $\mu = \nu = 0$ in Definition 1.9).

Proposition 4.1 (Composition of Affine Maps is Affine). Let $F_1 : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and $F_2 : \mathbb{R}^m \rightarrow \mathbb{R}^k$ be affine maps (Definition 4.1), with $F_1(x) = A_1x + b_1$ and $F_2(y) = A_2y + b_2$. Then $F_2 \circ F_1 : \mathbb{R}^n \rightarrow \mathbb{R}^k$ is affine, with

$$(F_2 \circ F_1)(x) = (A_2A_1)x + (A_2b_1 + b_2)$$

Proof. By direct computation:

$$\begin{aligned}
(F_2 \circ F_1)(x) &= F_2(F_1(x)) \\
&= F_2(A_1x + b_1) \\
&= A_2(A_1x + b_1) + b_2 \\
&= A_2A_1x + A_2b_1 + b_2 \\
&= (A_2A_1)x + (A_2b_1 + b_2)
\end{aligned}$$

Here A_2A_1 is ordinary matrix multiplication, which by Corollary 1.2 represents the composition of the linear operators A_1, A_2 correctly in the standard bases. Setting $A := A_2A_1$ and $b := A_2b_1 + b_2$ is exactly the form $F(x) = Ax + b$ of Definition 4.1, so $F_2 \circ F_1$ is also affine with linear part A_2A_1 and translation $A_2b_1 + b_2$. Q.E.D.

Corollary 4.1 (Depth Collapse of Affine Compositions). *Let $F_1, \dots, F_L$ be affine maps, with $F_\ell : \mathbb{R}^{n_{\ell-1}} \rightarrow \mathbb{R}^{n_\ell}$, $F_\ell(x) = A_\ell x + b_\ell$, for $\ell = 1, \dots, L$. Then the composition*

$$F := F_L \circ \dots \circ F_1 : \mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_L}$$

is affine: there exist $A \in \text{Mat}(n_L, n_0, \mathbb{R})$ and $b \in \mathbb{R}^{n_L}$ such that $F(x) = Ax + b$ for all x .

Proof. We proceed by induction on L .

Base Case: $F = F_1$ is affine by hypothesis.

Inductive Step: Suppose $G := F_{L-1} \circ \dots \circ F_1$ is affine, say $G(x) = A'x + b'$ for some A', b' . Then

$$F = F_L \circ G$$

a composition of the two affine maps G and F_L . By Proposition 4.1, $F_L \circ G$ is affine, with linear part $A_L A'$ and translation $A_L b' + b_L$. So F is affine, with $A := A_L A'$, $b := A_L b' + b_L$.

By induction, $F = F_L \circ \dots \circ F_1$ is affine for every $L \geq 1$. Q.E.D.

Computational Remark 4.1. `network.hpp` implements `Network::forward()` in Appendix A.10 as a loop over layers, applying each layer's affine map and activation in sequence.

This is worth justifying against Corollary 4.1: layer 0 uses `identityScalar` and the output layer applies no activation (Definition 4.3), so no non-affine function appears anywhere, and by that corollary this two-layer network is guaranteed to collapse to a single affine map. `forward()` could skip the loop entirely here. It doesn't, because the collapse only holds when every activation is the identity: a nonlinearity like ReLU breaks it (Remark 4.2), so the layer-by-layer structure is what has to generalise. This identity-activation case just gives a hand-computable answer to check that structure against, not a reason to shortcut it.

Verified in `main()`: `net.forward(x0)` and the independently-computed $A_2(A_1x_0 + b_1) + b_2$ both give $(32, -23)$, an exact match.

4.1.2 Activation Functions

Remark 4.2 (Nonlinearity is Necessary). Corollary 4.1 has a direct consequence for network architecture: if every layer of an L -layer network were purely affine, with no nonlinearity applied between layers, the entire network would collapse to a single affine map $F(x) = Ax + b$, which would be exactly as expressive as one layer.

Depth alone, without nonlinearity interleaved between the affine maps, buys nothing no matter how many layers are stacked, the composition is indistinguishable from a single affine transformation of the input. This is why every layer of a network is followed by a nonlinear activation function, which is the subject of this next subsection.

Definition 4.2 (Activation Function). Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a function. The activation of σ on $\mathbb{R}^n$ is the map $\sigma : \mathbb{R}^n \rightarrow \mathbb{R}^n$ defined componentwise, i.e.

$$\sigma(x) := (\sigma(x_1), \dots, \sigma(x_n)) \quad \text{for } x = (x_1, \dots, x_n) \in \mathbb{R}^n$$

applying σ independently to each coordinate of x , with no interaction between coordinates.

Example 4.1 (ReLU and Sigmoid). Two activation functions (Definition 4.2) used throughout this chapter are:

- i. The *Rectified Linear Unit* (ReLU) (Nair & Hinton 2010):

$$\text{ReLU}(t) := \max(t, 0) = \begin{cases} t & t \geq 0 \\ 0 & t < 0 \end{cases}$$

- ii. The *Sigmoid* function:

$$\varsigma(t) := \frac{1}{1 + e^{-t}}$$

Both are applied componentwise on $\mathbb{R}^n$ per Definition 4.2: for $x = (x_1, \dots, x_n) \in \mathbb{R}^n$, $\text{ReLU}(x) = (\text{ReLU}(x_1), \dots, \text{ReLU}(x_n))$, and likewise for ς .

Remark 4.3 (Non-Differentiability of ReLU at 0). $\text{ReLU}(t) = \max(t, 0)$ (Example 4.1, i.) is not differentiable at $t = 0$: the one-sided derivatives are 0 (approaching from below) and 1 (approaching from above), which disagree. So the Chain Rule (Theorem 2.2) does not literally apply at any point where an intermediate quantity equals exactly 0 after a ReLU is applied.

In practice, this is resolved by adopting the convention:

$$\text{ReLU}'(0) := 0$$

and treating ReLU as differentiable everywhere else, with $\text{ReLU}'(t) = 1$ for $t > 0$ and $\text{ReLU}'(t) = 0$ for $t < 0$. This is a convention, not a mathematical fact derived from Theorem 2.2 or any other result in this paper: the single point $t = 0$ has measure zero and is reached only for specific inputs, so the convention has no practical effect on computation, but it is not rigorously justified by the calculus developed in Chapter 2.

4.1.3 Feedforward Networks

Definition 4.3 (Feedforward Network). Let $L \geq 1$ be an integer (the depth), and let $n_0, n_1, \dots, n_L$ be positive integers (the widths of each layer). For each $\ell = 1, \dots, L$, let $A_\ell \in \text{Mat}(n_\ell, n_{\ell-1}, \mathbb{R})$ and $b_\ell \in \mathbb{R}^{n_\ell}$, defining the affine map $F_\ell(x) := A_\ell x + b_\ell$ (Definition 4.1), and let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be an activation function (Definition 4.2). Write

$$\theta := (A_1, b_1, \dots, A_L, b_L)$$

for the collection of all these matrices and vectors, the parameters of the network.

Given an input $x_0 \in \mathbb{R}^{n_0}$ (the data, not a parameter), define, for $\ell = 1, \dots, L$,

$$z_\ell := F_\ell(x_{\ell-1}) = A_\ell x_{\ell-1} + b_\ell \quad (\text{the pre-activation at layer } \ell)$$

$$x_\ell := \begin{cases} \sigma(z_\ell) & \ell < L \\ z_\ell & \ell = L \end{cases} \quad (\text{the post-activation at layer } \ell)$$

applying σ componentwise (Definition 4.2). The resulting map

$$f_\theta : \mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_L} \quad f_\theta(x_0) := x_L$$

is the feedforward network with parameters θ .

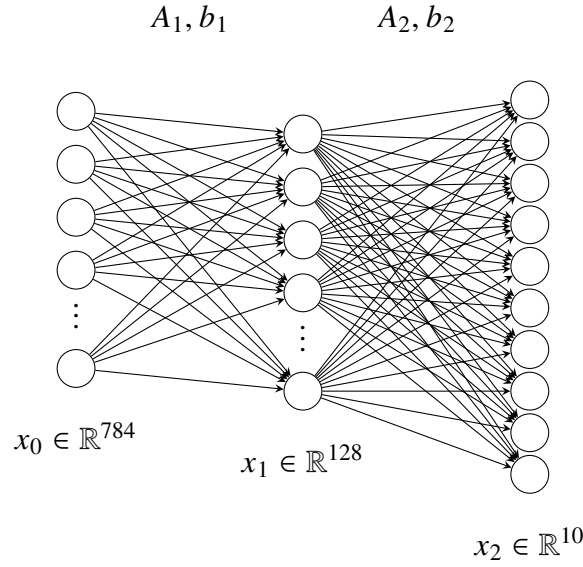


Figure 4.1: The feedforward network (Definition 4.3) used later in this paper: an input layer of 784 nodes ($x_0 \in \mathbb{R}^{784}$), a hidden layer of 128 nodes ($x_1 \in \mathbb{R}^{128}$) with ReLU activation, and an output layer of 10 nodes ($x_2 \in \mathbb{R}^{10}$). Each arrow bundle represents the affine map $A_\ell x_{\ell-1} + b_\ell$; the nonlinearity is applied at the nodes it feeds into, not on the arrows themselves.

Remark 4.4 (The Forward Pass). Evaluating $f_\theta(x_0)$ (Definition 4.3), computing $x_1, x_2, \dots, x_L$ in order from a given input x_0 , is called the forward pass. Writing it out,

$$f_\theta = F_L \circ \sigma \circ F_{L-1} \circ \sigma \circ \dots \circ \sigma \circ F_1$$

makes explicit what Definition 4.3 already encodes: a feedforward network is precisely an alternating composition of affine maps (Definition 4.1) and activations (Definition 4.2), exactly the kind of object this section set out to build.

Remark 4.5 (Parameter Count). The total number of scalar parameters in $\theta = (A_1, b_1, \dots, A_L, b_L)$ (Definition 4.3) is

$$\sum_{\ell=1}^L (n_\ell n_{\ell-1} + n_\ell)$$

since $A_\ell \in \text{Mat}(n_\ell, n_{\ell-1}, \mathbb{R})$ contributes $n_\ell n_{\ell-1}$ entries and $b_\ell \in \mathbb{R}^{n_\ell}$ contributes n_ℓ entries, at each layer ℓ .

4.1.4 The Softmax Output Layer

Definition 4.4 (Softmax). The softmax function is the map $\text{softmax} : \mathbb{R}^K \rightarrow \mathbb{R}^K$ defined by

$$\text{softmax}(z)_k := \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} \quad \text{for } k = 1, \dots, K$$

for $z = (z_1, \dots, z_K) \in \mathbb{R}^K$.

Proposition 4.2 (Softmax Output as a Valid Categorical Parameter). *For any $z \in \mathbb{R}^K$, the vector $p := \text{softmax}(z)$ (Definition 4.4) satisfies $p_k \geq 0$ for every k and $\sum_{k=1}^K p_k = 1$, which are the conditions Definition 3.6 requires of a parameter p .*

Proof. By Definition 4.4, $p_k = e^{z_k} / \sum_{j=1}^K e^{z_j}$ for each k . Since $e^{z_k} > 0$ for every real z_k , and the denominator $\sum_j e^{z_j}$ is a sum of such positive terms, $p_k > 0$ (in particular $p_k \geq 0$) for every k .

Summing,

$$\sum_{k=1}^K p_k = \sum_{k=1}^K \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} = \frac{\sum_{k=1}^K e^{z_k}}{\sum_{j=1}^K e^{z_j}} = 1$$

since the numerator and denominator are the same sum.

So $p = \text{softmax}(z)$ satisfies both conditions of Definition 3.6.

Q.E.D.

Proposition 4.3 (Shift Invariance of Softmax). *For any $z \in \mathbb{R}^K$ and $c \in \mathbb{R}$,*

$$\text{softmax}(z + c\mathbb{1}) = \text{softmax}(z)$$

where $z + c\mathbb{1} := (z_1 + c, \dots, z_K + c)$.

Proof. By Definition 4.4, for each k ,

$$\text{softmax}(z + c\mathbb{1})_k = \frac{e^{z_k + c}}{\sum_{j=1}^K e^{z_j + c}} = \frac{e^{z_k} e^c}{\sum_{j=1}^K e^{z_j} e^c} = \frac{e^c e^{z_k}}{e^c \sum_{j=1}^K e^{z_j}} = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} = \text{softmax}(z)_k$$

using $e^{z_j + c} = e^{z_j} e^c$ and cancelling the common factor e^c from numerator and denominator.

Q.E.D.

Computational Remark 4.2. `network.hpp` (Appendix A.10) also implements `softmax()`, which subtracts m , the maximum logit, before calling `std::exp`.

This is justified by Proposition 4.3: subtracting any constant c leaves softmax's output unchanged, so subtracting m is not an approximation as it's one exact member of a family the proposition already guarantees is correct. It's the safest member, though: subtracting m keeps the largest exponent at $e^0 = 1$, so nothing overflows, unlike subtracting nothing when logits are large.

Verified in `main()`: with logits as far apart as 32 and -23 , `softmax`'s output still summed to exactly 1 (Proposition 4.2), with one entry rounding to $\approx 1.3 \times 10^{-24}$.

Remark 4.6 (The Loss Function is Cross-Entropy). Applying softmax to the network's output, $Q_\theta(x) := \text{softmax}(f_\theta(x))$, produces a valid categorical parameter for every input x (Proposition 4.2), exactly the object anticipated in Remark 3.8.

For a training pair $(x^{(i)}, y^{(i)})$, with $\hat{p}^{(i)}$ the one-hot distribution on the observed label $y^{(i)}$, the same argument behind Corollary 3.6 gives a per-example loss $H(\hat{p}^{(i)}, Q_\theta(x^{(i)}))$; summing over the training set gives the objective

$$\sum_i H(\hat{p}^{(i)}, Q_\theta(x^{(i)}))$$

So the loss used to train this network in Section 4.2.1 is cross-entropy by derivation, not convention: applying the Chapter 3 argument example-by-example rather than to one shared distribution.

This per-example extension of Corollary 3.6 is not itself proved anywhere in this document; it is stated here as a direct analogue of the cited result, not a cited consequence of it.

4.2 Backpropagation

This section formalises the network’s training objective as cross-entropy loss (Section 4.2.1), then derives the backward pass: the recursion for the per-layer error term δ_ℓ and the resulting gradients of the loss with respect to every layer’s weights and biases via the chain rule established in Chapter 2 (Section 4.2.2).

This is where the paper’s central claim is settled: backpropagation is not a separate algorithm bolted onto the network, it is Theorem 2.2 applied repeatedly (Rumelhart et al. 1986), layer by layer and every gradient a training run computes is provably the true gradient of the loss this chapter derived from maximum likelihood, not an approximation or a heuristic.

The corresponding C++ implementation, shared between the forward and backward pass, is in `network.hpp` (Appendix A.10), with the backward pass specifically verified in `backward_pass.cpp` (Appendix A.11).

4.2.1 The Loss Function

Definition 4.5 (Training Loss). Let $\{(x^{(i)}, y^{(i)})\}_{i=1}^n$ be a training set, with each $y^{(i)}$ a class label in $\{1, \dots, K\}$, where $K = n_L$ is the network’s output width (Definition 4.3). Write $\hat{p}^{(i)}$ for the one-hot distribution on the observed label $y^{(i)}$, i.e. $\hat{p}_k^{(i)} = 1$ if $k = y^{(i)}$ and $\hat{p}_k^{(i)} = 0$ otherwise, and $Q_\theta(x) := \text{softmax}(f_\theta(x))$ (Remark 4.6). The training loss is

$$L(\theta) := \sum_{i=1}^n H(\hat{p}^{(i)}, Q_\theta(x^{(i)}))$$

the sum of per-example cross-entropy terms (Goodfellow 2016).

Remark 4.7 (Training Loss in Closed Form). Since $\hat{p}^{(i)}$ is one-hot on $y^{(i)}$, every term in $H(\hat{p}^{(i)}, Q_\theta(x^{(i)})) = -\sum_k \hat{p}_k^{(i)} \log Q_\theta(x^{(i)})_k$ vanishes except $k = y^{(i)}$ (Definition 4.5), so

$$L(\theta) = -\sum_{i=1}^n \log Q_\theta(x^{(i)})_{y^{(i)}}$$

Remark 4.8 (Reduction to a Single Example). By linearity of differentiation, computing $\nabla_\theta L(\theta)$ (Definition 4.5) reduces to computing the gradient of a single term, $-\log Q_\theta(x)_y$, and summing over the training set. The next section, Section 4.2.2, derives this gradient for one example, dropping the (i) superscript accordingly.

4.2.2 The Backward Pass

Definition 4.6 (Backward Error Term). Fix a single training example (x, y) , dropping the (i) superscript as in the closing remark of the above section, Section 4.2.1. For each layer $\ell = 1, \dots, L$, the backward error term at layer ℓ is

$$\delta_\ell := \frac{\partial L}{\partial z_\ell}$$

the gradient of the loss with respect to that layer's pre-activation (Definition 4.3).

Proposition 4.4 (Output-Layer Error Term). *The output layer error term is*

$$\delta_L = Q_\theta(x) - \hat{p}$$

Proof. $L = -\log Q_\theta(x)_y = -\log \text{softmax}(z_L)_y = -z_{L,y} + \log \sum_{j=1}^K e^{z_{L,j}}$ (Definition 4.4). Differentiating with respect to $(z_L)_k$, by cases:

$$\text{If } k = y: \frac{\partial L}{\partial (z_L)_k} = -1 + \frac{e^{z_{L,k}}}{\sum_j e^{z_{L,j}}} = Q_\theta(x)_k - 1.$$

$$\text{If } k \neq y: \frac{\partial L}{\partial (z_L)_k} = 0 + \frac{e^{z_{L,k}}}{\sum_j e^{z_{L,j}}} = Q_\theta(x)_k.$$

In both cases, since $\hat{p}_k = 1$ if $k = y$ and 0 otherwise, this is $Q_\theta(x)_k - \hat{p}_k$. So $\delta_L = Q_\theta(x) - \hat{p}$. Q.E.D.

Definition 4.7 (Hadamard Product). For $u, v \in \mathbb{R}^n$, the Hadamard product $u \odot v \in \mathbb{R}^n$ is defined entrywise by $(u \odot v)_j := u_j v_j$.

Theorem 4.1 (Backward Recursion). *For $\ell = 1, \dots, L-1$,*

$$\delta_\ell = (A_{\ell+1}^\top \delta_{\ell+1}) \odot \sigma'(z_\ell)$$

Proof. By the Chain Rule (Theorem 2.2) applied through $z_\ell \rightarrow x_\ell \rightarrow z_{\ell+1}$: since $x_\ell = \sigma(z_\ell)$ (componentwise, Definition 4.2) and $(z_{\ell+1})_j = \sum_m (A_{\ell+1})_{jm} (x_\ell)_m + (b_{\ell+1})_j$, for each coordinate k ,

$$(\delta_\ell)_k = \frac{\partial L}{\partial (z_\ell)_k} = \sum_j \frac{\partial L}{\partial (z_{\ell+1})_j} \frac{\partial (z_{\ell+1})_j}{\partial (x_\ell)_k} \frac{\partial (x_\ell)_k}{\partial (z_\ell)_k} = \sum_j (\delta_{\ell+1})_j (A_{\ell+1})_{jk} \sigma'((z_\ell)_k)$$

using $\partial (z_{\ell+1})_j / \partial (x_\ell)_k = (A_{\ell+1})_{jk}$ and, since $(x_\ell)_k = \sigma((z_\ell)_k)$ depends only on coordinate k (Definition 4.2), $\partial (x_\ell)_k / \partial (z_\ell)_k = \sigma'((z_\ell)_k)$. This is

$$(\delta_\ell)_k = (A_{\ell+1}^\top \delta_{\ell+1})_k \sigma'((z_\ell)_k)$$

i.e. $\delta_\ell = (A_{\ell+1}^\top \delta_{\ell+1}) \odot \sigma'(z_\ell)$ (Definition 4.7). When $\sigma = \text{ReLU}$, $\sigma'((z_\ell)_k)$ at any coordinate where $(z_\ell)_k = 0$ is understood via the convention $\text{ReLU}'(0) := 0$ (Remark 4.3). Q.E.D.

Computational Remark 4.3. `backwardStep()` in `network.hpp`, Appendix A.11, implements Theorem 4.1.

It reuses `applyActivation()`, the same componentwise-apply function used to compute $\sigma(z_\ell)$ during the forward pass, to compute $\sigma'(z_\ell)$ as well, simply by passing it a different function pointer.

This is justified directly by Definition 4.2: componentwise application doesn't care what scalar function it's given, σ or σ' , only that it's applied coordinate-by-coordinate with no interaction between them, so the same generic machinery correctly computes either.

This is verified by the gradient check at `net.layers[0].A(0,0)`: numerical 0.853139 against analytic 0.853139, a difference of 6.3396×10^{-12} . This is the one non-degenerate check among those run, the entry whose gradient depends on the recursion propagating δ backward through both layers, so it is the check that genuinely stresses Theorem 4.1 rather than confirming a simpler, more local computation.

Proposition 4.5 (Parameter Gradients). *For $\ell = 1, \dots, L$,*

$$\frac{\partial L}{\partial A_\ell} = \delta_\ell x_{\ell-1}^\top, \quad \frac{\partial L}{\partial b_\ell} = \delta_\ell$$

Proof. Since $(z_\ell)_j = \sum_k (A_\ell)_{jk} (x_{\ell-1})_k + (b_\ell)_j$, for each entry $(A_\ell)_{jk}$,

$$\frac{\partial L}{\partial (A_\ell)_{jk}} = \frac{\partial L}{\partial (z_\ell)_j} \frac{\partial (z_\ell)_j}{\partial (A_\ell)_{jk}} = (\delta_\ell)_j (x_{\ell-1})_k$$

which is exactly the (j, k) entry of $\delta_\ell x_{\ell-1}^\top$, so $\partial L / \partial A_\ell = \delta_\ell x_{\ell-1}^\top$. Similarly, $\partial (z_\ell)_j / \partial (b_\ell)_j = 1$, so $\partial L / \partial (b_\ell)_j = (\delta_\ell)_j$, i.e. $\partial L / \partial b_\ell = \delta_\ell$. Q.E.D.

Computational Remark 4.4. `parameterGradients()` in `network.hpp`, Appendix A.11, implements Proposition 4.5.

The outer product $\delta_\ell x_{\ell-1}^\top$ is computed directly via the existing `Matrix operator*` and `.transpose()`, rather than a dedicated outer-product routine: since `operator*` already handles matrix multiplication for any compatible dimensions, a column vector times a row vector is already exactly what it computes, so no special-cased routine is needed.

Verified by `net.layers[1].A(1,2)` and `net.layers[0].b(2,0)`, both exactly 0: the third hidden unit's pre-activation is negative ($z_1(2) = -0.25$), so ReLU' is 0 there (Remark 4.3), zeroing the corresponding entry of δ_1 via the backward recursion (Theorem 4.1) and propagating through to both formulas exactly.

Remark 4.9 (Backpropagation). Computing the network's output requires one forward sweep through the layers $\ell = 1, \dots, L$, producing every z_ℓ, x_ℓ (Remark 4.4).

Afterward, δ_L is computed first (Proposition 4.4), then Theorem 4.1 gives $\delta_{L-1}, \dots, \delta_1$ in order, traversing the layers opposite to the forward sweep. This is the reason for the name: the error term propagates backward, against the direction the input flowed forward.

The gradients are not a separate pass: at each layer ℓ , Proposition 4.5 gives $\partial L / \partial A_\ell$ and $\partial L / \partial b_\ell$ directly from δ_ℓ and the cached $x_{\ell-1}$, so every gradient falls out of the same backward sweep: one forward pass plus one backward pass, total, regardless of L .

This two-phase procedure, forward to compute the output, backward to read off the error terms and gradients together, is what the term backpropagation refers to.

4.3 Universal Approximation

This section constructs a single-hidden-layer ReLU network that approximates any continuous function on a compact interval to arbitrary accuracy, by building tent functions from ReLUs, showing their sums interpolate a target function, bounding the interpolation error via uniform continuity, and assembling these into the Universal Approximation Theorem.

This closes a gap the rest of this chapter leaves open. Sections 4.2 and 4.1 built the network architecture and showed how to train it, but neither addresses whether the architecture can represent what training is asked to find in the first place.

Chapter 2's gradient descent convergence theorem answers whether training can find a good solution, given one exists; this section answers whether one exists at all. It also depends specifically on the choice made in Section 4.1 (Example 4.1): the entire construction rests on ReLU's piecewise-linear shape, which is exactly what makes tent functions expressible as a finite sum of ReLUs (Proposition 4.6) whereas a smooth activation like sigmoid would not admit nearly as direct a construction.

4.3.1 Tent Functions

Definition 4.8 (Tent Function). Let $x_{k-1} < x_k < x_{k+1}$ be three points. The tent function $T_k : \mathbb{R} \rightarrow \mathbb{R}$ is the piecewise-linear function that is 0 outside the interval $[x_{k-1}, x_{k+1}]$, rises linearly from 0 to 1 on $[x_{k-1}, x_k]$, and falls linearly from 1 to 0 on $[x_k, x_{k+1}]$.

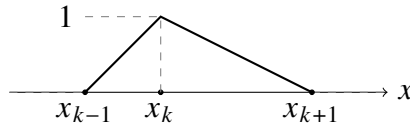


Figure 4.2: The tent function T_k (Definition 4.8) for the asymmetric choice $x_{k-1} = 0$, $x_k = 1$, $x_{k+1} = 3$: piecewise-linear, rising to height 1 at x_k and falling back to 0 at x_{k+1} , zero outside $[x_{k-1}, x_{k+1}]$.

Proposition 4.6 (Tent Functions as a Sum of ReLUs). With T_k as in Definition 4.8,

$$T_k(x) = c_1 \text{ReLU}(x - x_{k-1}) + c_2 \text{ReLU}(x - x_k) + c_3 \text{ReLU}(x - x_{k+1})$$

where

$$c_1 = \frac{1}{x_k - x_{k-1}} \quad c_2 = -\frac{1}{x_k - x_{k-1}} - \frac{1}{x_{k+1} - x_k} \quad c_3 = \frac{1}{x_{k+1} - x_k}$$

Proof. Each $\text{ReLU}(x - c)$ has slope 0 below c and slope 1 above it, so a sum of such terms has, at any point, a slope equal to the sum of coefficients of the breakpoints already passed.

Write $R(x) := c_1 \text{ReLU}(x - x_{k-1}) + c_2 \text{ReLU}(x - x_k) + c_3 \text{ReLU}(x - x_{k+1})$. Its slope in the four regions determined by x_{k-1}, x_k, x_{k+1} is 0, c_1 , $c_1 + c_2$, $c_1 + c_2 + c_3$, respectively. By Definition 4.8, T_k 's slopes there are 0, $\frac{1}{x_k - x_{k-1}}$, $\frac{-1}{x_{k+1} - x_k}$, 0. The first region already matches; equating the rest gives

$$c_1 = \frac{1}{x_k - x_{k-1}} \quad c_1 + c_2 = \frac{-1}{x_{k+1} - x_k} \quad c_1 + c_2 + c_3 = 0$$

which solve to the claimed c_1, c_2, c_3 .

Matching slopes alone only pins down each piece up to a constant, so we check one anchor point: $T_k(x_{k-1}) = 0$, and $R(x_{k-1}) = 0$ too, since every $\text{ReLU}(x_{k-1} - \cdot)$ term vanishes there. With the same breakpoints, same slopes, and the same value at x_{k-1} , T_k and R agree everywhere. Q.E.D.

Remark 4.10 (Boundary Tents). Definition 4.8 needs a neighbour on each side of x_k , but the endpoints x_0, x_n of a partition $x_0 < \dots < x_n$ have only one. Introduce phantom points $x_{-1} < x_0$ and $x_{n+1} > x_n$ (any choice works) so T_0, T_n are still well-defined by the same formula. Neither phantom point matters beyond this: on $[x_0, x_n]$, T_0 and T_n just reduce to a single linear ramp.

4.3.2 Piecewise-Linear Interpolation

Definition 4.9 (Piecewise-Linear Interpolant). Let $x_0 < \dots < x_n$ and let f be defined at each x_j . The *piecewise-linear interpolant* of f through $(x_j, f(x_j))_{j=0}^n$ is the function $I : [x_0, x_n] \rightarrow \mathbb{R}$, linear on each $[x_{j-1}, x_j]$, with $I(x_j) = f(x_j)$ for every j .

Proposition 4.7 (The Interpolant as a Sum of Tents). *With I as above and T_j the tent centred at x_j (Definition 4.8, Remark 4.10),*

$$I(x) = \sum_{j=0}^n f(x_j) T_j(x) \quad \text{for all } x \in [x_0, x_n]$$

Proof. Fix $x \in [x_0, x_n]$, with $x \in [x_{k-1}, x_k]$ for the appropriate k (at the endpoints x_0, x_n only one tent survives; both cases below specialise cleanly).

For $|j - k| \geq 2$, $T_j(x) = 0$: either x lies strictly outside T_j 's support $[x_{j-1}, x_{j+1}]$, or x coincides with one of that support's endpoints, where T_j itself takes the value 0 (Definition 4.8). Either way, only $j = k - 1, k$ can contribute.

Writing $x = (1 - \lambda)x_{k-1} + \lambda x_k$ for $\lambda \in [0, 1]$: T_{k-1} falls linearly from 1 to 0 on this interval, so $T_{k-1}(x) = 1 - \lambda$; T_k rises from 0 to 1, so $T_k(x) = \lambda$. Hence

$$\sum_{j=0}^n f(x_j) T_j(x) = f(x_{k-1})(1 - \lambda) + f(x_k)\lambda$$

the line segment through $(x_{k-1}, f(x_{k-1}))$ and $(x_k, f(x_k))$ at x — exactly $I(x)$ (Definition 4.9). Since x was arbitrary, this holds on every subinterval. Q.E.D.

4.3.3 Uniform Continuity and the Approximation Bound

Theorem 4.2 (Uniform Continuity on a Compact Interval). *Let $f : [a, b] \rightarrow \mathbb{R}$ be continuous. Then f is uniformly continuous for every $\varepsilon > 0$, there exists $\delta > 0$ such that for all $x, x' \in [a, b]$,*

$$|x - x'| < \delta \implies |f(x) - f(x')| < \varepsilon$$

Proof. See (Rudin 1976, §4.19). Q.E.D.

Proposition 4.8 (Piecewise-Linear Interpolation Error Bound). *Let $f : [a, b] \rightarrow \mathbb{R}$ be continuous and let $\varepsilon > 0$. Let $\delta > 0$ be as given by Theorem 4.2 for $\varepsilon/2$. If $x_0 < x_1 < \dots < x_n$ is a partition of $[a, b]$ with mesh $\max_j (x_j - x_{j-1}) < \delta$, and I is the piecewise-linear interpolant of f through this partition (Definition 4.9), then*

$$|f(x) - I(x)| < \varepsilon \quad \text{for all } x \in [a, b]$$

Proof. Fix $x \in [a, b]$, with $x \in [x_{k-1}, x_k]$ for the appropriate k . Since the mesh is $< \delta$, $|x - x_{k-1}| < \delta$ and $|x - x_k| < \delta$, so by Theorem 4.2 (applied with $\varepsilon/2$),

$$|f(x) - f(x_{k-1})| < \frac{\varepsilon}{2} \quad |f(x) - f(x_k)| < \frac{\varepsilon}{2}$$

By Proposition 4.7's proof, $I(x) = (1 - \lambda)f(x_{k-1}) + \lambda f(x_k)$ for some $\lambda \in [0, 1]$. So

$$f(x) - I(x) = f(x) - (1 - \lambda)f(x_{k-1}) - \lambda f(x_k) = (1 - \lambda)(f(x) - f(x_{k-1})) + \lambda(f(x) - f(x_k))$$

using $(1 - \lambda) + \lambda = 1$ to rewrite $f(x) = (1 - \lambda)f(x) + \lambda f(x)$ first. By the triangle inequality,

$$|f(x) - I(x)| \leq (1 - \lambda)|f(x) - f(x_{k-1})| + \lambda|f(x) - f(x_k)| < (1 - \lambda)\frac{\varepsilon}{2} + \lambda\frac{\varepsilon}{2} = \frac{\varepsilon}{2}$$

using $0 \leq \lambda \leq 1$. So $|f(x) - I(x)| < \varepsilon/2 < \varepsilon$, and since $x \in [a, b]$ was arbitrary, this holds for every x . Q.E.D.

4.3.4 The Universal Approximation Theorem

Theorem 4.3 (Universal Approximation, One Dimension). *Let $f : [a, b] \rightarrow \mathbb{R}$ be continuous, and let $\varepsilon > 0$. Then there exists a single-hidden-layer feedforward network N with ReLU activation (Definition 4.3, Example 4.1) such that*

$$|N(x) - f(x)| < \varepsilon \quad \text{for all } x \in [a, b]$$

This is a version of the classical Universal Approximation Theorem, not the theorem in its full generality: Cybenko (Cybenko 1989) established it for sigmoidal activations and Hornik (Hornik 1991) generalised it to a broader class of activation functions, both over compact subsets of $\mathbb{R}^n$, whereas Theorem 4.3 is restricted to the ReLU activation specifically and a one-dimensional domain $[a, b]$.

Proof. By Proposition 4.8, choose $\delta > 0$ so that any partition of $[a, b]$ with mesh $< \delta$ yields a piecewise-linear interpolant I (Definition 4.9) satisfying $|f(x) - I(x)| < \varepsilon$ for all $x \in [a, b]$. Fix such a partition $x_0 < x_1 < \dots < x_n$.

It remains to show I itself is the output of a single-hidden-layer ReLU network. By Proposition 4.7,

$$I(x) = \sum_{j=0}^n f(x_j) T_j(x)$$

and by Proposition 4.6, each tent T_j is itself a linear combination of at most three ReLU terms of affine functions of x ,

$$T_j(x) = c_1^{(j)} \text{ReLU}(x - x_{j-1}) + c_2^{(j)} \text{ReLU}(x - x_j) + c_3^{(j)} \text{ReLU}(x - x_{j+1})$$

Substituting,

$$I(x) = \sum_{j=0}^n f(x_j) \left(c_1^{(j)} \text{ReLU}(x - x_{j-1}) + c_2^{(j)} \text{ReLU}(x - x_j) + c_3^{(j)} \text{ReLU}(x - x_{j+1}) \right)$$

which, expanding and collecting terms, is a finite sum of the form $\sum_{i=1}^m w_i \text{ReLU}(x - t_i)$ for some weights w_i and breakpoints t_i , with $m \leq 3(n + 1)$.

This is precisely a single-hidden-layer network in the sense of Definition 4.3: each term $\text{ReLU}(x - t_i)$ is the ReLU activation applied to the affine map $x \mapsto x - t_i$ (a hidden unit with weight 1 and bias $-t_i$), and $I(x)$ is a linear combination of these hidden units with weights w_i and no further activation — exactly $L = 2$ with a ReLU hidden layer of width $m \leq 3(n + 1)$ and a linear (activation-free) output layer of width 1. Setting $N := I$ under this construction, N is such a network with

$$|N(x) - f(x)| = |I(x) - f(x)| < \varepsilon \quad \text{for all } x \in [a, b]$$

Q.E.D.

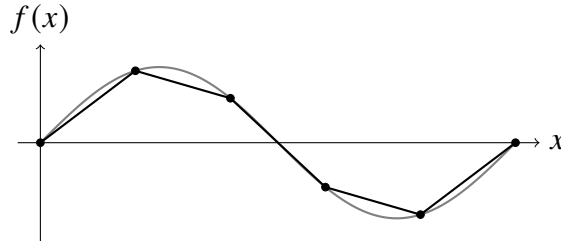


Figure 4.3: $f(x) = \sin(x)$ on $[0, 2\pi]$ (grey) and its piecewise-linear interpolant $I(x)$ (black) through 6 evenly spaced points (Definition 4.9). The visible gap between the two curves is exactly what Proposition 4.8 bounds.

Remark 4.11 (Representability). Theorem 4.3 shows that a network with the right weights exists, however it says nothing about whether gradient descent (Theorem 2.5) can find those weights from a random initialisation, nor how many training steps that would take.

The construction here is entirely explicit and non-adaptive, chosen directly from f and a fine enough partition; nothing in this proof involves training a network on data. Universal approximation is a statement about what functions are representable within the network's architecture, not a guarantee about learnability.

Chapter 5

MNIST

This chapter trains the network built in Chapter 4 on real data and reports what happened.

This is where the abstract machinery built across four chapters is tested against something concrete: every piece used here, the architecture, the loss, the gradients, the update rule, was defined and proved correct in earlier chapters, and this chapter is the point at which that machinery either produces a working digit classifier or it doesn't.

The implementation is in `impl/mnist/`.

5.1 Data and Network Setup

This section covers getting real data into the network's expected input form, and getting the network itself into a trainable starting state.

Neither step is proved correct by anything in Chapters 1–4: both are engineering decisions, made explicit here and justified against the architecture those chapters actually built, rather than left implicit.

Implemented in `mnist_loader.hpp` (Appendix A.12), specifically `loadImages()`, `loadLabels()`, and `initializeWeights()`.

MNIST is distributed in the IDX file format: a short binary header, a magic number identifying the file type followed by dimension counts (number of images, then rows and columns per image, or just a count for the label file), followed by the raw data itself as one unsigned byte per pixel or per label, with no delimiters between records. Reading it is a matter of parsing this header correctly and then reading the expected number of bytes directly into memory.

Each 28×28 image is flattened into a single 784×1 column vector. This is not an incidental detail of the loader: it is exactly the input vector $x_0 \in \mathbb{R}^{n_0}$ of Definition 4.3, with $n_0 = 784$ the network built in Chapter 4 takes a vector as input, so an image has to be presented as one before the network can act on it at all.

Each pixel, originally an integer in $[0, 255]$, is rescaled to $[0, 1]$ by dividing by 255.

Computational Remark 5.1. `loadImages()` (`mnist_loader.hpp`, Appendix A.12) rescales every pixel from $[0, 255]$ to $[0, 1]$ before returning the image as a 784×1 `Matrix`.

This is an engineering decision, not something proved in earlier chapters, but it is not an arbitrary one. Nothing in the forward pass (Definition 4.3, `forward_pass.cpp`) includes any

built-in normalisation of its input. Feeding raw pixel values in $[0, 255]$ directly into the first layer would push pre-activation magnitudes up to 255 times higher than expected, far outside the input scale the fan-in-scaled weight initialisation below assumes.

Every layer’s weight matrix A_ℓ must also be initialised before training can begin.

Initialising every entry of A_ℓ to zero fails, and this follows directly from Proposition 4.5. If every weight feeding into a layer is identical, every neuron in that layer receives the same input and computes the same pre-activation. Since $\partial L / \partial A_\ell = \delta_\ell x_{\ell-1}^\top$ depends on δ_ℓ , which in turn depends on $A_{\ell+1}^\top \delta_{\ell+1}$ via the backward recursion (Theorem 4.1) applied symmetrically to every identical neuron, every neuron in the layer receives an identical gradient at every step. The layer can never break this symmetry: its neurons remain identical to each other for the entire duration of training.

`initializeWeights()` instead draws each entry of A_ℓ independently and uniformly from $\left[-\frac{1}{\sqrt{n_{\ell-1}}}, \frac{1}{\sqrt{n_{\ell-1}}}\right]$, where $n_{\ell-1}$ is the layer’s fan-in (Definition 4.3), with each bias b_ℓ left at zero. Unlike the zero-initialisation failure above, this specific scheme is not something Chapters 1–3 prove correct. It is a standard engineering convention, scaling the initial weight range down as the number of inputs to a neuron grows to keep pre-activation magnitudes from growing uncontrollably with layer width, rather than a derived result.

5.2 Mini-Batch Stochastic Gradient Descent

This section covers the update procedure used to train the network.

Implemented in `train_mnist.cpp`, specifically `applyGradients()` and the batch loop in `main()`.

Theorem 2.5 was proved for full-batch gradient descent: every step uses the gradient of the loss over the entire training set. At the other extreme, stochastic gradient descent (SGD) takes a step using the gradient from a single training example at a time. Mini-batch SGD sits between these: each step uses the average gradient over a small, randomly chosen subset (a batch) of the training set, here of size 32, rather than the full set of 60,000 or a single example.

The batch’s gradient is averaged, not summed, over the examples in it. This keeps the effective step size independent of batch size: doubling the batch size would double a summed gradient’s magnitude, requiring the learning rate to be re-tuned every time the batch size changes, whereas an averaged gradient’s magnitude does not depend on how many examples contributed to it. Averaging also handles the last batch of an epoch correctly when the training set size is not an exact multiple of the batch size, since that batch is smaller than the rest.

The training order is reshuffled at the start of every epoch, so that each epoch sees the training examples grouped into different batches rather than the same fixed groupings repeatedly.

The most important thing to say about this section is what it does not have: Theorem 2.5, the only convergence guarantee this paper has proved for gradient descent, requires the objective to be convex and L -smooth. The cross-entropy loss composed with this multi-layer network is, in general, neither. Running mini-batch SGD on this network is therefore not licensed by anything proved in this paper. It is standard practice, but Chapters 1–4 provide no theorem that says it has to work here.

Computational Remark 5.2. `applyGradients()` (`train_mnist.cpp`, Appendix A.13) implements the gradient descent update $A_\ell \leftarrow A_\ell - \eta \partial L / \partial A_\ell$, $b_\ell \leftarrow b_\ell - \eta \partial L / \partial b_\ell$ (Definition 2.8), applied to the batch-averaged gradients rather than a single example’s.

No theoretical verification is offered here, consistent with the caveat above: Theorem 2.5’s hypotheses do not hold for this objective, so there is no theorem-predicted quantity to check the implementation against. The only verification available is empirical, and is deferred to Section 5.3’s accuracy trajectory.

5.3 Results

This section reports what happened when the procedure of Section 5.2 was run.

This is arguably the payoff of the whole paper: everything from Chapter 1 onward was building toward a network that can be trained, and this section is where that claim is checked against an actual run rather than asserted.

The network used is $784 \rightarrow 128$ (ReLU) $\rightarrow 10$ (softmax, cross-entropy loss), trained with batch size 32, learning rate $\eta = 0.1$, for 100 epochs.

Test accuracy and loss after each epoch:

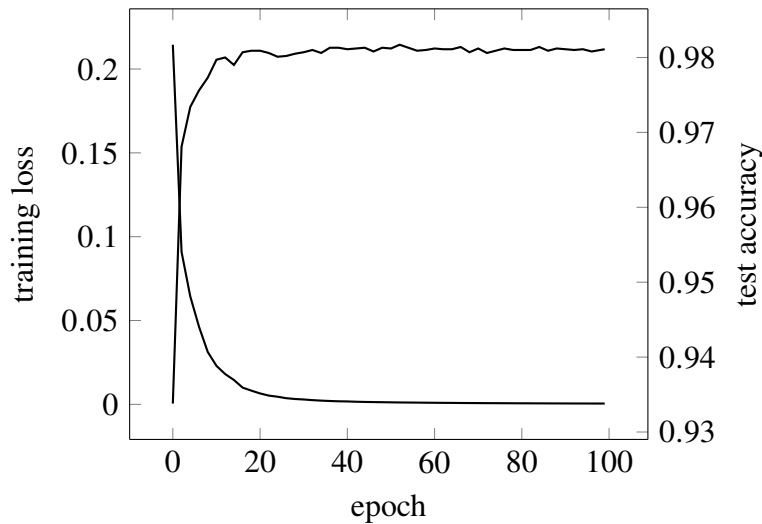


Figure 5.1: Training loss (left axis) and test accuracy (right axis) after each epoch of Section 5.3, read from `training_log.csv`.

Training loss falls monotonically across all one hundred epochs. Test accuracy, however, rises only for roughly the first 15 to 20 epochs, then plateaus and oscillates in a narrow 98.0% to 98.2% band for the remainder of training, rather than continuing to climb alongside the loss. Neither trend is something Section 5.2 predicted, since it admitted no theorem in this paper guarantees either behaviour on this non-convex objective; both are empirical results standing in for the theoretical guarantee that section explicitly does not have.

The interesting fact is not these two observations in isolation, but that they diverge: the loss keeps shrinking long after accuracy stops improving. This is suggestive of the standard overfitting story. The network keeps growing more confident on the training examples, pushing $Q_\theta(x)_y$ closer to 1 for the correct label y , without that increasing confidence translating into

further gains on the held-out test set. This is a suggestion as only test accuracy was tracked, and the textbook signature of overfitting is test loss itself rising, which cannot be checked from the data collected in this run.

This is an expected result for a minimal, from-scratch, two-layer fully connected network and is presented as exactly that: not a claim of state-of-the-art performance, competitive accuracy, or anything beyond what a small MLP trained for a few epochs on MNIST ordinarily achieves.

Hyperparameter tuning and more expressive architectures such as convolutional networks would all likely improve on this result; none are attempted here since demonstrating that the machinery built in this paper trains at all is the object of this chapter, not maximal accuracy.

Appendix A

Implementation

A.1 Core Matrix Class

```
1  #pragma once
2
3  #include <iostream>
4  #include <stdexcept>
5  #include <vector>
6  #include <cmath>
7
8  // complete Matrix object
9
10 class Matrix {
11     private:
12         int rows_, cols_;
13         std::vector<double> data_;
14
15     public:
16         Matrix(int rows, int cols) : rows_(rows), cols_(cols),
17         data_(rows * cols) {}
18
19         double& operator()(int row, int col) {
20             return data_[row * cols_ + col];
21         }
22         const double& operator()(int row, int col) const {
23             return data_[row * cols_ + col];
24         }
25
26         int rows() const {
27             return rows_;
28         }
29         int cols() const {
30             return cols_;
31         }
32
33         double& operator()(int i) {
34             return data_[i];
35         }
```

```

35     const double& operator()(int i) const {
36         return data_[i];
37     }
38
39     // Define matrix addition
40     Matrix operator+(const Matrix& mat) const {
41         if (rows_ != mat.rows_ || cols_ != mat.cols_) {
42             throw std::invalid_argument("Matrix dimensions not
equal");
43         };
44
45         Matrix result(rows_, cols_);
46         for (int i = 0; i < rows_; i++) {
47             for (int j = 0; j < cols_; j++) {
48                 result.data_[i * cols_ + j] = data_[i * cols_ +
j] + mat.data_[i * cols_ + j];
49             }
50         }
51         return result;
52     }
53
54     // Define matrix multiplication
55     Matrix operator*(const Matrix& mat) const {
56         if (cols_ != mat.rows_) {
57             throw std::invalid_argument("Matrix dimensions
incompatible for multiplication");
58         }
59
60         Matrix result(rows_, mat.cols_);
61         for (int i = 0; i < rows_; i++) {
62             for (int j = 0; j < mat.cols_; j++) {
63                 double sum = 0;
64                 for (int k = 0; k < cols_; k++) {
65                     sum += data_[i * cols_ + k] * mat.data_[k *
mat.cols_ + j];
66                 }
67                 result.data_[i * mat.cols_ + j] = sum;
68             }
69         }
70         return result;
71     }
72
73     // Define transposition
74     Matrix transpose() const {
75
76         Matrix result(cols_, rows_);
77         for (int i = 0; i < rows_; i++) {
78             for (int j = 0; j < cols_; j++) {
79                 result(j, i) = data_[i * cols_ + j];
80             }
81         }

```

```

82         return result;
83     }
84 };
85
86 // Other Operators
87
88 // dot(u, v): dot product of two n x 1 column vectors, stored as
89 // Matrix(n,1)
90 double dot(const Matrix& u, const Matrix& v) {
91     double sum = 0.0;
92     for (int i = 0; i < u.rows(); ++i) {
93         sum += u(i, 0) * v(i, 0);
94     }
95     return sum;
96 }
97
98 // norm(v): sqrt(dot(v,v))
99 double norm(const Matrix& v) {
100     return std::sqrt(dot(v, v));
101 }
102
103 // scale(v, c): returns c * v, elementwise, for any matrix v
104 Matrix scale(const Matrix& v, double c) {
105     int n = v.rows();
106     int m = v.cols();
107     Matrix result(n, m);
108     for (int i = 0; i < n; ++i) {
109         for (int j = 0; j < m; ++j) {
110             result(i, j) = c * v(i, j);
111         }
112     }
113     return result;
114 }
115
116 // subtractInPlace(u, w): u -= w, elementwise, for matrices of
117 // equal dimensions
118 void subtractInPlace(Matrix& u, const Matrix& w) {
119     for (int i = 0; i < u.rows(); ++i) {
120         for (int j = 0; j < u.cols(); ++j) {
121             u(i, j) -= w(i, j);
122         }
123     }
124 }

```

matrix.hpp

A.2 LU Decomposition

```

1 #pragma once
2

```

```

3 #include <cmath>
4 #include <utility>
5 #include "matrix.hpp"
6
7 // Factorises A=LU and then solves a linear system via forward and
  // back substitution
8
9 std::vector<int> factoriseLU(Matrix& A) {
10     int n = A.rows();
11     std::vector<int> perm(n);
12
13     for (int i = 0; i < n; ++i) perm[i] = i;
14
15     for (int k = 0; k < n - 1; ++k) {
16         // Step 1: find pivot row
17         int r = k;
18         double maxVal = std::abs(A(k, k));
19
20         for (int i = k + 1; i < n; ++i) {
21             double val = std::abs(A(i, k));
22             if (val > maxVal) {
23                 maxVal = val;
24                 r = i;
25             }
26         }
27
28         // Step 2: swap if needed
29         if (r != k) {
30             for (int j = 0; j < n; ++j) {
31                 std::swap(A(r, j), A(k, j));
32             }
33             std::swap(perm[r], perm[k]);
34         }
35
36         // Step 3: eliminate
37         for (int i = k + 1; i < n; ++i) {
38             double m = A(i, k) / A(k, k);
39             A(i, k) = m;
40             for (int j = k + 1; j < n; ++j) {
41                 A(i, j) = A(i, j) - m * A(k, j);
42             }
43         }
44     }
45
46     return perm;
47 };
48
49
50 Matrix solveLU(const Matrix& LU, const std::vector<int>& perm,
51               const Matrix& b) {
52     int n = LU.rows();

```



```

52
53 // Construct bPrime
54 Matrix bPrime(n, 1);
55 for (int i = 0; i < n; ++i) {
56     bPrime(i) = b(perm[i], 0);
57 }
58
59 // Forward substitution
60 Matrix y(n, 1);
61 for (int k = 0; k < n; ++k) {
62     double sum = 0.0;
63     for (int j = 0; j < k; ++j) {
64         sum += LU(k, j) * y(j);
65     }
66     y(k) = bPrime(k) - sum;
67 }
68
69 // Backward substitution
70 Matrix x(n, 1);
71 for (int k = n - 1; k >= 0; --k) {
72     double sum = 0.0;
73     for (int j = k + 1; j < n; ++j) {
74         sum += LU(k, j) * x(j);
75     }
76     x(k) = (y(k) - sum) / LU(k, k);
77 }
78
79 return x;
80 }

```

lu_solver.hpp

```

1 #include <iostream>
2 #include "lu_solver.hpp"
3
4
5 int main() {
6
7     Matrix A(2, 2);
8     A(0, 0) = 1; A(0, 1) = 1;
9     A(1, 0) = 2; A(1, 1) = 3;
10
11     Matrix b(2, 1);
12     b(0, 0) = 2;
13     b(1, 0) = 5;
14
15     std::vector<int> perm = factoriseLU(A); // A is overwritten in
16     place with LU
17     Matrix x = solveLU(A, perm, b);
18
19     for (int i = 0; i < 2; ++i) {
20         std::cout << "x[" << i << "] = " << x(i) << std::endl;
21     }
22 }

```

```

20     }
21
22     return 0;
23 }

```

lu_solver.cpp

A.3 QR via Gram-Schmidt

```

1  #include <iostream>
2  #include <cmath>
3  #include "matrix.hpp"
4
5  // Implements a A=QR Decomposition via the Gram Schmidt Process
6
7  // Helpers
8
9  // getColumn(A, k): extract column k of A as an n x 1 Matrix
10 Matrix getColumn(const Matrix& A, int k) {
11     int n = A.rows();
12     Matrix col(n, 1);
13     for (int i = 0; i < n; ++i) {
14         col(i, 0) = A(i, k);
15     }
16     return col;
17 }
18
19 // setColumn(M, k, v): write n x 1 vector v into column k of M
20 void setColumn(Matrix& M, int k, const Matrix& v) {
21     int n = M.rows();
22     for (int i = 0; i < n; ++i) {
23         M(i, k) = v(i, 0);
24     }
25 }
26
27
28 // Loop
29 void gramSchmidtQR(const Matrix& A, Matrix& Q, Matrix& R) {
30     int n = A.rows(); // assumption: A is square, n x n; not
31                        // checked
32
33     for (int k = 0; k < n; ++k) {
34         Matrix uk = getColumn(A, k);
35
36         for (int j = 0; j < k; ++j) {
37             Matrix qj = getColumn(Q, j);
38             double Rjk = dot(uk, qj); //  $R_{jk} := \langle u_k, q_j \rangle$  , MGS
39             algorithm
40             R(j, k) = Rjk;
41             Matrix proj = scale(qj, Rjk);

```

```

40         subtractInPlace(uk, proj);
41     }
42
43     double Rkk = norm(uk);
44     R(k, k) = Rkk;
45
46     Matrix qk = scale(uk, 1.0 / Rkk);
47     setColumn(Q, k, qk);
48 }
49 }
50
51 // main() verification
52
53 int main() {
54     int n = 3;
55     Matrix A(n, n);
56
57     A(0,0) = 12; A(0,1) = -51; A(0,2) = 4;
58     A(1,0) = 6;  A(1,1) = 167; A(1,2) = -68;
59     A(2,0) = -4; A(2,1) = 24;  A(2,2) = -41;
60
61     Matrix Q(n, n);
62     Matrix R(n, n);
63
64     gramSchmidtQR(A, Q, R);
65
66     std::cout << "Q:\n";
67     for (int i = 0; i < n; ++i) {
68         for (int j = 0; j < n; ++j) {
69             std::cout << Q(i, j) << " ";
70         }
71         std::cout << "\n";
72     }
73
74     std::cout << "\nR:\n";
75     for (int i = 0; i < n; ++i) {
76         for (int j = 0; j < n; ++j) {
77             std::cout << R(i, j) << " ";
78         }
79         std::cout << "\n";
80     }
81
82     Matrix QtQ = Q.transpose() * Q;
83     std::cout << "\nQ^T Q (should be approx identity):\n";
84     for (int i = 0; i < n; ++i) {
85         for (int j = 0; j < n; ++j) {
86             std::cout << QtQ(i, j) << " ";
87         }
88         std::cout << "\n";
89     }
90

```

```

91     Matrix QR = Q * R;
92     std::cout << "\nQR (should be approx A):\n";
93     for (int i = 0; i < n; ++i) {
94         for (int j = 0; j < n; ++j) {
95             std::cout << QR(i, j) << " ";
96         }
97         std::cout << "\n";
98     }
99
100     return 0;
101 }

```

qr_gram_schmidt.cpp

A.4 Jacobi Eigensolver

```

1  #pragma once
2
3  #include <cmath>
4  #include "matrix.hpp"
5
6  // Applies a Jacobi Eigen Decomposition
7
8  double findLargestOffDiagonal(const Matrix& A, int& p, int& q) {
9      int n = A.rows();
10     double maxVal = 0.0;
11     p = -1;
12     q = -1;
13
14     // Iterate through matrix entries and find the biggest off
15     // diagonal value.
16     for (int i = 0; i < n; ++i) {
17         for (int j = i + 1; j < n; ++j) { // A is symmetric so only
18             // scan the upper triangle
19             double val = std::abs(A(i, j));
20             if (val > maxVal) {
21                 maxVal = val;
22                 p = i;
23                 q = j;
24             }
25         }
26     }
27
28     return maxVal;
29 }
30
31 void computeRotationAngle(double App, double Aqq, double Apq,
32     double& c, double& s) {
33
34     if (Apq == 0.0) {

```

```

32     // Already zero at this position; rotation is identity.
33     c = 1.0;
34     s = 0.0;
35     return;
36 }
37
38 double tau = (Aqq - App) / (2.0 * Apq);
39
40 double t;
41 if (tau >= 0.0) {
42     t = 1.0 / (tau + std::sqrt(1.0 + tau * tau));
43 } else {
44     t = 1.0 / (tau - std::sqrt(1.0 + tau * tau));
45 }
46
47 c = 1.0 / std::sqrt(1.0 + t * t);
48 s = t * c;
49 }
50
51 Matrix buildRotationMatrix(int n, int p, int q, double c, double s)
52 {
53     Matrix J(n,n);
54     for (int i = 0; i < n; ++i) {
55         for (int j = 0; j < n; ++j) {
56             J(i, j) = (i == j) ? 1.0 : 0.0; // Shorthand to write
57             Identity
58         }
59     }
60
61     J(p, p) = c;
62     J(q, q) = c;
63     J(p, q) = s;
64     J(q, p) = -s;
65
66     return J;
67 }
68
69 Matrix applyJacobiRotation(const Matrix& A, const Matrix& J) {
70     return J.transpose() * A * J;
71 }
72
73 // A is the initial symmetric Matrix, Q is the Matrix of
74 // Orthonormal Eigenvectors
75 void jacobiEigenDecomposition(const Matrix& A, Matrix& Q, Matrix&
76     eigenvalues) {
77
78     int p, q;
79     double c, s;
80
81     for (int i = 0; i < A.rows(); ++i) {
82         for (int j = 0; j < A.cols(); ++j) {

```

```

79         Q(i, j) = (i == j) ? 1.0 : 0.0; // Shorthand to write
      Identity
80     }
81 }
82
83 const double tol = 1e-10;
84 int iter = 0;
85 Matrix Awork = A;
86 int n = Awork.rows();
87
88 while (true) { // Iterate the Jacobi Eigen Decomposition until
      biggest value is within a tolerance
89     double maxVal = findLargestOffDiagonal(Awork, p, q);
90     if (maxVal < tol || iter > 50 * n * n) {
91         break;
92     }
93
94     computeRotationAngle(Awork(p,p), Awork(q,q), Awork(p,q), c,
      s); // Computes theta
95     Matrix J = buildRotationMatrix(n, p, q, c, s); //
      Constructs Matrix
96     Awork = applyJacobiRotation(Awork, J); // Applies the
      rotation
97     Q = Q * J; // One rotation at a time goes from the basis to
      the Eigenbasis
98
99     ++iter;
100 }
101
102 for (int i = 0; i < Awork.rows(); ++i) {
103     eigenvalues(i, 0) = Awork(i, i);
104 }
105 }

```

jacobi_eigsolver.hpp

```

1  #include <iostream>
2  #include "jacobi_eigsolver.hpp"
3
4
5  int main() {
6      Matrix A(5, 5);
7      A(0,0) = 4; A(0,1) = 1; A(0,2) = 0; A(0,3) = 0; A(0,4) = 0;
8      A(1,0) = 1; A(1,1) = 3; A(1,2) = 1; A(1,3) = 0; A(1,4) = 0;
9      A(2,0) = 0; A(2,1) = 1; A(2,2) = 2; A(2,3) = 1; A(2,4) = 0;
10     A(3,0) = 0; A(3,1) = 0; A(3,2) = 1; A(3,3) = 3; A(3,4) = 1;
11     A(4,0) = 0; A(4,1) = 0; A(4,2) = 0; A(4,3) = 1; A(4,4) = 4;
12
13     int n = A.rows();
14     Matrix Q(n,n), eigenvalues(n, 1);
15
16     jacobiEigenDecomposition(A, Q, eigenvalues);

```

```

17
18     for (int i = 0; i < n; ++i) {
19         for (int j = 0; j < n; ++j) {
20             std::cout << "Q[" << i << j << "] = " << Q(i, j) << std
::endl;
21         }
22     }
23     for (int i = 0; i < n; ++i) {
24         std::cout << "eigenvalue[" << i << "] = " << eigenvalues(i,
0) << std::endl;
25     }
26     return 0;
27 }

```

jacobi_eigsolver.cpp

A.5 SVD

```

1  #include <iostream>
2  #include <cmath>
3  #include <vector>
4  #include <algorithm>
5  #include <numeric>
6  #include "matrix.hpp"
7  #include "jacobi_eigsolver.hpp"
8
9  // Full SVD via eigendecomposition
10
11 // Helpers
12
13 Matrix getColumn(const Matrix& A, int k) {
14     int n = A.rows();
15     Matrix col(n, 1);
16     for (int i = 0; i < n; ++i) {
17         col(i, 0) = A(i, k);
18     }
19     return col;
20 }
21
22 void setColumn(Matrix& M, int k, const Matrix& v) {
23     int n = M.rows();
24     for (int i = 0; i < n; ++i) {
25         M(i, k) = v(i, 0);
26     }
27 }
28
29 // Singular Value Decomposition
30
31 Matrix computeAtA(const Matrix& A, Matrix& eigenvalues) {
32

```

```

33     int n = A.cols();
34     Matrix AtA = A.transpose() * A;
35     Matrix V(n, n);
36
37     jacobiEigenDecomposition(AtA, V, eigenvalues);
38
39     return V;
40 }
41
42 void sortEigenDecomposition(const Matrix& eigenvalues, const Matrix
    & Q, Matrix& sortedEigenvalues, Matrix& sortedQ) {
43     int n = eigenvalues.rows(); // assumption: eigenvalues is n x
    1, Q is n x n
44
45     std::vector<int> idx(n);
46     std::iota(idx.begin(), idx.end(), 0);
47
48     std::sort(idx.begin(), idx.end(), [&](int a, int b) {
49         return eigenvalues(a, 0) > eigenvalues(b, 0); // descending
50     });
51
52     for (int newPos = 0; newPos < n; ++newPos) {
53         int oldPos = idx[newPos];
54         sortedEigenvalues(newPos, 0) = eigenvalues(oldPos, 0);
55         for (int row = 0; row < n; ++row) {
56             sortedQ(row, newPos) = Q(row, oldPos);
57         }
58     }
59 }
60
61 void computeSigmaAndU(const Matrix& A, const Matrix&
    sortedEigenvalues, const Matrix& sortedV, Matrix& sigma, Matrix&
    U) {
62     int m = A.rows();
63     int n = A.cols();
64     const double tol = 1e-10;
65
66     for (int i = 0; i < n; ++i) {
67         double s = std::sqrt(std::max(sortedEigenvalues(i, 0), 0.0)
68 );
69         sigma(i, 0) = s;
70
71         if (s > tol) {
72             Matrix vi = getColumn(sortedV, i);
73             Matrix Avi = A * vi; // m x 1
74             Avi = scale(Avi, 1.0 / s);
75             setColumn(U, i, Avi);
76         }
77         // else: leave column i of U as zero, which is already true
        by construction
    }

```



```

78 }
79
80 int main() {
81
82     Matrix A(4, 6);
83     A(0,0) = 2;  A(0,1) = -1; A(0,2) = 3;  A(0,3) = 0;  A(0,4) = 5;
84     A(0,5) = 1;
85     A(1,0) = 1;  A(1,1) = 4;  A(1,2) = -2; A(1,3) = 3;  A(1,4) = 0;
86     A(1,5) = 2;
87     A(2,0) = 0;  A(2,1) = 2;  A(2,2) = 1;  A(2,3) = -1; A(2,4) = 3;
88     A(2,5) = 4;
89     A(3,0) = 3;  A(3,1) = 3;  A(3,2) = 1;  A(3,3) = 3;  A(3,4) = 5;
90     A(3,5) = 3;
91
92     int n = A.cols();
93     int m = A.rows();
94
95     Matrix eigenvalues(n, 1);
96     Matrix V = computeAtA(A, eigenvalues);
97
98     Matrix sortedEigenvalues(n, 1);
99     Matrix sortedV(n, n);
100     sortEigenDecomposition(eigenvalues, V, sortedEigenvalues,
101                             sortedV);
102
103     Matrix sigma(n, 1);
104     Matrix U(m, n);
105     computeSigmaAndU(A, sortedEigenvalues, sortedV, sigma, U);
106
107     // --- Verifier ---
108
109     // Build Sigma (n x n, zero except diagonal)
110     Matrix Sigma(n, n);
111     for (int i = 0; i < n; ++i) {
112         Sigma(i, i) = sigma(i, 0);
113     }
114
115     // Reconstruct A from U, Sigma, V
116     Matrix reconstructed = U * Sigma * sortedV.transpose();
117
118     // Max absolute reconstruction error
119     double maxDiff = 0.0;
120     for (int i = 0; i < m; ++i) {
121         for (int j = 0; j < n; ++j) {
122             double diff = std::abs(A(i, j) - reconstructed(i, j));
123             if (diff > maxDiff) {
124                 maxDiff = diff;
125             }
126         }
127     }
128 }

```

```

124     std::cout << "max reconstruction error: " << maxDiff << std::
endl;
125
126     return 0;
127 }

```

svd_jacobi.cpp

A.6 Numerical Gradient Checker

```

1  #include <iostream>
2  #include <cmath>
3  #include "../linear_algebra/matrix.hpp"
4
5  // Gradient Checker
6
7  int dim = 4;
8  Matrix A(dim,dim);
9
10 Matrix numericalGradient(double (*f)(const Matrix&), const Matrix&
    x, double h) {
11     int n = x.rows();
12
13     Matrix grad(n, 1);
14     for (int i = 0; i < n; ++i) {
15         Matrix xPlus = x;
16         Matrix xMinus = x;
17
18         xPlus(i,0) += h;
19         xMinus(i,0) -= h;
20
21         grad(i, 0) = (f(xPlus) - f(xMinus)) / (2 * h); //
Calculates the finite difference Gradient
22     }
23     return grad;
24 }
25
26 double testFunction(const Matrix& x) {
27     Matrix xtAx = x.transpose() * A * x;
28     double result = xtAx(0,0);
29     return result;
30 }
31
32 Matrix analyticGradient(const Matrix& x) {
33     Matrix sum = A + A.transpose();
34     Matrix sumx = sum * x;
35     return sumx;
36 }
37
38

```

```

39 int main() {
40
41     Matrix x(dim, 1);
42     double h = 10e-5;
43
44     A(0,0) = 4; A(0,1) = 1; A(0,2) = 0; A(0,3) = 3;
45     A(1,0) = 1; A(1,1) = 3; A(1,2) = 1; A(1,3) = 6;
46     A(2,0) = 8; A(2,1) = 1; A(2,2) = 2; A(2,3) = 1;
47     A(3,0) = 7; A(3,1) = 9; A(3,2) = 1; A(3,3) = 5;
48
49     x(0,0) = 6; x(1,0) = 3; x(2,0) = 7; x(3,0) = 9;
50
51     Matrix numerical = numericalGradient(testFunction, x, h);
52     Matrix analytic = analyticGradient(x);
53
54     double maxError = 0.0;
55     for (int i = 0; i < dim; ++i) {
56         std::cout << numerical(i, 0) << " ";
57         std::cout << analytic(i, 0) << " " << std::endl;
58         double diff = std::fabs(numerical(i,0) - analytic(i,0));
59         if (diff > maxError) {
60             maxError = diff;
61         }
62     }
63     std::cout << "Max abs error: " << maxError << std::endl;
64 }

```

gradient_checker.cpp

A.7 Gradient Descent

```

1  #include <cmath>
2  #include <iostream>
3  #include "../linear_algebra/matrix.hpp"
4  #include "../linear_algebra/jacobi_eigensolver.hpp"
5
6  // Implements the Gradient Descent algorithm
7
8  int dim = 6;
9  Matrix A(dim, dim);
10
11 double testFunction(const Matrix& x) {
12     Matrix xtAx = x.transpose() * A * x;
13     double result = xtAx(0,0);
14     return result;
15 }
16
17 Matrix analyticGradient(const Matrix& x) { // Same gradient
18     function in 'gradient_checker.cpp'
19     Matrix sum = A + A.transpose();

```

```

19     Matrix sumx = sum * x;
20     return sumx;
21 }
22
23 double computeL(const Matrix& A) { // Computes Lipschitz Constant
24     int n = A.rows();
25     Matrix Q(n, n);
26     Matrix eigenvalues(n, 1);
27     jacobiEigenDecomposition(A, Q, eigenvalues);
28
29     double lambdaMax = eigenvalues(0,0);
30     for (int i = 1; i < n; ++i) {
31         double lambda = eigenvalues(i,0);
32         if (lambda > lambdaMax) {
33             lambdaMax = lambda;
34         }
35     }
36     double L = 2 * lambdaMax;
37
38     return L;
39 }
40
41 Matrix gradientDescent(const Matrix& x0, double eta, double epsilon
42 , int& iterations) { // Main algorithm: gradient descent
43     sequence
44     Matrix x = x0;
45     iterations = 0;
46
47     while (true) {
48         Matrix grad = analyticGradient(x);
49         if (norm(grad) < epsilon) {
50             break;
51         }
52         subtractInPlace(x, scale(grad, eta));
53         iterations += 1;
54     }
55     return x;
56 }
57
58 int main() { // Test case
59
60     Matrix M(5, 6);
61     M(0,0) = 4;  M(0,1) = -3; M(0,2) = 2;  M(0,3) = 1;  M(0,4) =
-5; M(0,5) = 0;
62     M(1,0) = -2; M(1,1) = 6;  M(1,2) = 0;  M(1,3) = -4; M(1,4) = 3;
M(1,5) = 5;
63     M(2,0) = 1;  M(2,1) = 1;  M(2,2) = -6; M(2,3) = 2;  M(2,4) = 0;
M(2,5) = -3;
64     M(3,0) = 5;  M(3,1) = 0;  M(3,2) = 3;  M(3,3) = -1; M(3,4) = 4;
M(3,5) = 2;

```

```

64     M(4,0) = -3; M(4,1) = 2;  M(4,2) = -1; M(4,3) = 6;  M(4,4) = 1;
        M(4,5) = -4;
65
66     A = M.transpose() * M; // Guarantee a symmetric nxn matrix
67
68     double L = computeL(A);
69     double eta = 1.0 / L;
70
71     Matrix x0(6, 1);
72     x0(0,0) = 2;
73     x0(1,0) = -3;
74     x0(2,0) = 6;
75     x0(3,0) = -8;
76     x0(4,0) = 1;
77     x0(5,0) = 4;
78
79     const double epsilon = 10e-7;
80     int iterations;
81     Matrix descent = gradientDescent(x0, eta, epsilon, iterations);
82     double fxk = testFunction(descent);
83
84     double x0squared = dot(x0, x0); // norm squared
85     double twoEtaK = 2 * eta * iterations;
86     double ratio = x0squared / twoEtaK;
87
88     std::cout << "f(x_K): " << fxk << std::endl;
89     std::cout << "ratio: " << ratio << std::endl;
90     std::cout << "iterations: " << iterations << std::endl;
91
92     return 0;
93 }

```

gradient_descent.cpp

A.8 KKT Solver

```

1  #include "../linear_algebra/matrix.hpp"
2  #include "../linear_algebra/lu_solver.hpp"
3
4
5  void assembleKKT(
6      const Matrix& Q, const Matrix& A, const Matrix& c,
7      const Matrix& b, Matrix& K, Matrix& d) {
8
9      int n = Q.rows();
10     int m = A.rows();
11
12     for (int i = 0; i < n; ++i) {
13         for (int j = 0; j < n; ++j) {
14             K(i, j) = Q(i, j);

```

```

15     }
16 }
17 for (int i = 0; i < n; ++i) {
18     for (int j = 0; j < m; ++j) {
19         K(i, n+j) = -A(j, i);
20     }
21 }
22 for (int i = 0; i < m; ++i) {
23     for (int j = 0; j < n; ++j) {
24         K(n+i, j) = A(i, j);
25     }
26 }
27 for (int i = 0; i < n; ++i) {
28     d(i, 0) = -c(i, 0);
29 }
30 for (int i = 0; i < m; ++i) {
31     d(n+i, 0) = b(i, 0);
32 }
33 }
34
35 void solveKKT(
36     const Matrix& Q, const Matrix& A, const Matrix& c,
37     const Matrix& b, Matrix& x, Matrix& lambda) {
38
39     int n = Q.rows();
40     int m = A.rows();
41
42     Matrix K(n+m, n+m);
43     Matrix d(n+m, 1);
44
45     assembleKKT(Q, A, c, b, K, d);
46
47     std::vector<int> perm = factoriseLU(K);
48     Matrix z = solveLU(K, perm, d);
49
50     for (int i = 0; i < n; ++i) {
51         x(i, 0) = z(i, 0);
52     }
53     for (int i = 0; i < m; ++i) {
54         lambda(i, 0) = z(n+i, 0);
55     }
56 }
57
58
59 int main() {
60     // Test case:  $\min (1/2)x^T Q x + c^T x$  s.t.  $Ax = b$ 
61     // Q built as  $M^T M$  for a non-symmetric  $M = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$ ,
62     // giving a
63     // non-identity SPD Q rather than the maximally symmetric  $I_2$ 
64     // case.
65     //  $M^T M = \begin{bmatrix} 4 & 2 \\ 2 & 10 \end{bmatrix}$ 

```

```

64 // c = (1,-2) (nonzero, breaking the earlier c=0 triviality)
65 // A = (1 1), b = (1)
66
67 int n = 2;
68 int m = 1;
69
70 Matrix Q(n, n);
71 Q(0,0) = 4; Q(0,1) = 2;
72 Q(1,0) = 2; Q(1,1) = 10;
73
74 Matrix A(m, n);
75 A(0,0) = 1; A(0,1) = 1;
76
77 Matrix c(n, 1);
78 c(0,0) = 1;
79 c(1,0) = -2;
80
81 Matrix b(m, 1);
82 b(0,0) = 1;
83
84 Matrix x(n, 1);
85 Matrix lambda(m, 1);
86
87 solveKKT(Q, A, c, b, x, lambda);
88
89 std::cout << "x:" << std::endl;
90 for (int i = 0; i < n; ++i) {
91     std::cout << "  x[" << i << "] = " << x(i, 0) << std::endl;
92 }
93
94 std::cout << "lambda:" << std::endl;
95 for (int i = 0; i < m; ++i) {
96     std::cout << "  lambda[" << i << "] = " << lambda(i, 0) <<
std::endl;
97 }
98
99 // Verification: residuals of the two KKT equations
100 Matrix Atlambda(n, 1);
101 for (int i = 0; i < n; ++i) {
102     double sum = 0.0;
103     for (int j = 0; j < m; ++j) {
104         sum += A(j, i) * lambda(j, 0);
105     }
106     Atlambda(i, 0) = sum;
107 }
108
109 Matrix Qx(n, 1);
110 for (int i = 0; i < n; ++i) {
111     double sum = 0.0;
112     for (int j = 0; j < n; ++j) {
113         sum += Q(i, j) * x(j, 0);

```

```

114     }
115     Qx(i, 0) = sum;
116 }
117
118 std::cout << "residual Qx - A^T lambda + c:" << std::endl;
119 for (int i = 0; i < n; ++i) {
120     double r1 = Qx(i, 0) - Atlambda(i, 0) + c(i, 0);
121     std::cout << " [" << i << "] = " << r1 << std::endl;
122 }
123
124 std::cout << "residual Ax - b:" << std::endl;
125 for (int i = 0; i < m; ++i) {
126     double sum = 0.0;
127     for (int j = 0; j < n; ++j) {
128         sum += A(i, j) * x(j, 0);
129     }
130     double r2 = sum - b(i, 0);
131     std::cout << " [" << i << "] = " << r2 << std::endl;
132 }
133
134 return 0;
135 }

```

kkt_solver.cpp

A.9 Categorical MLE

```

1 #include <iostream>
2 #include <vector>
3 #include <random>
4 #include <cmath>
5
6 std::vector<int> sampleCategorical(const std::vector<double>& p,
7     int n, std::mt19937& rng) {
8
9     std::vector<double> cdf(p.size());
10    double runningTotal = 0.0;
11    for (size_t k = 0; k < p.size(); ++k) {
12        runningTotal += p[k];
13        cdf[k] = runningTotal;
14    }
15    std::vector<int> samples(n);
16    std::uniform_real_distribution<double> dist(0.0, 1.0);
17    for (int i = 0; i < n; ++i) {
18        double u = dist(rng);
19
20        samples[i] = static_cast<int>(cdf.size()) - 1;
21        for (size_t k = 0; k < cdf.size(); ++k) {
22            if (cdf[k] >= u) {

```



```

23         break;
24     }
25 }
26 }
27 return samples;
28 }
29
30 std::vector<int> countOccurrences(const std::vector<int>& samples,
    int K) {
31     std::vector<int> counts(K, 0);
32
33     for (int s : samples) {
34         counts[s]++;
35     }
36     return counts;
37 }
38
39 std::vector<double> computeMLE(const std::vector<int>& counts, int
    n) {
40     std::vector<double> pHat(counts.size());
41
42     for (size_t k = 0; k < counts.size(); ++k) {
43         pHat[k] = static_cast<double>(counts[k]) / n;
44     }
45     return pHat;
46 }
47
48 double logLikelihood(const std::vector<int>& counts, const std::
    vector<double>& p) {
49     double logLik = 0.0;
50
51     for (size_t k = 0; k < counts.size(); ++k) {
52         logLik += counts[k] * std::log(p[k]);
53     }
54
55     return logLik;
56 }
57
58 std::vector<double> perturb(const std::vector<double>& p, int i,
    int j, double epsilon) {
59     std::vector<double> result = p;    // makes a copy of p
60
61     result[i] = result[i] - epsilon;
62     result[j] = result[j] + epsilon;
63
64     return result;
65 }
66
67 int main() {
68     // Main test function written by Claude
69     std::mt19937 rng(42);

```

```

70
71 // True distribution and sample size
72 std::vector<double> pTrue = {0.5, 0.3, 0.2};
73 int n = 1000;
74 int K = static_cast<int>(pTrue.size());
75
76 // Step 1: draw samples
77 std::vector<int> samples = sampleCategorical(pTrue, n, rng);
78
79 // Step 2: compute counts and the MLE
80 std::vector<int> counts = countOccurrences(samples, K);
81 std::vector<double> pHat = computeMLE(counts, n);
82
83 // Piece 1: print the MLE against the true distribution
84 for (size_t k = 0; k < pHat.size(); ++k) {
85     std::cout << "p_hat[" << k << "] = " << pHat[k]
86         << " (true p[" << k << "] = " << pTrue[k] << ")
87     " << std::endl;
88 }
89
90 // Compute log-likelihood at the MLE
91 double llAtMLE = logLikelihood(counts, pHat);
92
93 // Piece 2: print it
94 std::cout << "log-likelihood at MLE: " << llAtMLE << std::endl;
95
96 // Piece 3: perturb away from the MLE in three directions and
97 // confirm
98 // the log-likelihood only ever decreases (or stays equal),
99 // consistent
100 // with prop:categorical-mle claiming pHat is the maximiser.
101 double epsilon = 0.01;
102
103 std::vector<double> pPerturbed01 = perturb(pHat, 0, 1, epsilon)
104 ;
105 double llPerturbed01 = logLikelihood(counts, pPerturbed01);
106 std::cout << "perturbed (0->1) log-likelihood: " <<
107 llPerturbed01
108     << " (vs MLE: " << llAtMLE << ")" << std::endl;
109
110 std::vector<double> pPerturbed12 = perturb(pHat, 1, 2, epsilon)
111 ;
112 double llPerturbed12 = logLikelihood(counts, pPerturbed12);
113 std::cout << "perturbed (1->2) log-likelihood: " <<
114 llPerturbed12
115     << " (vs MLE: " << llAtMLE << ")" << std::endl;
116
117 std::vector<double> pPerturbed20 = perturb(pHat, 2, 0, epsilon)
118 ;
119 double llPerturbed20 = logLikelihood(counts, pPerturbed20);

```

```

12     std::cout << "perturbed (2->0) log-likelihood: " <<
    llPerturbed20
13         << " (vs MLE: " << llAtMLE << ")" << std::endl;
14
15     return 0;
16 }

```

categorical_mle.cpp

A.10 Forward Pass

```

1  #pragma once
2
3  #include <algorithm>
4  #include "../linear_algebra/matrix.hpp"
5
6
7  struct LayerGradients {
8      Matrix gradLA;
9      Matrix gradLb;
10
11      LayerGradients(Matrix gradA, Matrix gradB): gradLA(gradA),
    gradLb(gradB) {}
12 };
13
14
15 double reluScalar(double t) {
16     return std::max(t, 0.0);
17 }
18 double reluScalarDerivative(double t) {
19     double tPrime;
20     // ReLU is not differentiable at t=0; ReLU'(0):=0 is a
convention
21     if (t <= 0) {
22         tPrime = 0;
23     }
24     else {
25         tPrime = 1;
26     }
27
28     return tPrime;
29 }
30
31 double identityScalar(double t) {
32     return t;
33 }
34 double identityScalarDerivative() {
35     return 1;
36 }
37

```

```

38 Matrix applyActivation(const Matrix& v, double (*f)(double)) {
39     Matrix result(v.rows(), 1);
40     for (int i = 0; i < v.rows(); ++i) {
41         result(i, 0) = f(v(i, 0));
42     }
43     return result;
44 }
45
46 Matrix softmax(const Matrix& z) {
47
48     double m = z(0, 0);
49     for (int i = 1; i < z.rows(); ++i) {
50         m = std::max(m, z(i, 0));
51     }
52
53     Matrix result(z.rows(), 1);
54     double sum = 0.0;
55     for (int i = 0; i < z.rows(); ++i) {
56         result(i, 0) = std::exp(z(i, 0) - m);
57         sum += result(i, 0);
58     }
59
60     for (int i = 0; i < z.rows(); ++i) {
61         result(i, 0) = result(i, 0) / sum;
62     }
63
64     return result;
65 }
66
67 Matrix hadamard(const Matrix& u, const Matrix& v) {
68
69     Matrix hadprod(u.rows(), 1);
70     for (int j = 0; j < u.rows(); ++j) {
71         hadprod(j, 0) = u(j, 0) * v(j, 0);
72     }
73     return hadprod;
74 }
75
76 Matrix outputLayerError(const Matrix& zL, int y) {
77
78     int K = zL.rows();
79     Matrix pHat(K, 1);
80     pHat(y, 0) = 1;
81
82     Matrix qThetaX = softmax(zL);
83     subtractInPlace(qThetaX, pHat);
84
85     Matrix deltaL = qThetaX;
86     return deltaL;
87 }
88

```

```

89 Matrix backwardStep(const Matrix& aNext, const Matrix& deltaNext,
    const Matrix& z, double (*f)(double)) {
90     Matrix sigmaPrime = applyActivation(z, f);
91     Matrix delta = hadamard(aNext.transpose() * deltaNext,
        sigmaPrime);
92
93     return delta;
94 }
95
96 LayerGradients parameterGradients(const Matrix& delta, const Matrix
    & xMinusOne) {
97     Matrix gradLA = delta * xMinusOne.transpose();
98     Matrix gradLb = delta;
99
100    return LayerGradients(gradLA, gradLb);
101 }
102
103 struct Layer {
104     Matrix A;
105     Matrix b;
106     bool hasActivation;
107
108     Layer(int n_in, int n_out, bool hasActivation): A(n_out, n_in),
        b(n_out, 1), hasActivation(hasActivation) {}
109 };
110
111 struct ForwardCache {
112     std::vector<Matrix> preActivations;
113     std::vector<Matrix> postActivations;
114 };
115
116 struct Network {
117     std::vector<Layer> layers;
118     double (*activation)(double);
119     double (*activationDerivative)(double);
120
121     Matrix forward(const Matrix& x0) {
122         ForwardCache cache = forwardTrace(x0);
123         Matrix current = cache.postActivations.back();
124
125         return current;
126     }
127
128     ForwardCache forwardTrace(const Matrix& x0) {
129         ForwardCache cache;
130         cache.postActivations.push_back(x0);
131
132         Matrix current = x0;
133         for (size_t i = 0; i < layers.size(); ++i) {
134             Matrix z = layers[i].A * current + layers[i].b;
135             cache.preActivations.push_back(z);

```

```

136         if (layers[i].hasActivation) {
137             current = applyActivation(z, activation);
138         }
139         else {
140             current = z;
141         }
142         cache.postActivations.push_back(current);
143     }
144     return cache;
145 }
146
147 std::vector<LayerGradients> backward(const Matrix& x0, int y) {
148     ForwardCache cache = forwardTrace(x0);
149     int N = layers.size();
150
151     Matrix delta = outputLayerError(cache.preActivations.back()
152 , y);
153
154     std::vector<LayerGradients> grads;
155     grads.push_back(parameterGradients(delta, cache.
156 postActivations[N-1]));
157     // loop index i (0-indexed) corresponds to math layer i+1;
158     // the weight matrix needed here is A_{i+2}, i.e. layers[i+1].A.
159     for (int i = N - 2; i >= 0; --i) {
160         delta = backwardStep(layers[i+1].A, delta, cache.
161 preActivations[i], activationDerivative);
162         grads.push_back(parameterGradients(delta, cache.
163 postActivations[i]));
164     }
165     std::reverse(grads.begin(), grads.end());
166     return grads;
167 }
168 };
169
170 void applyGradients(Network& net, const std::vector<LayerGradients
171 >& grads, double learningRate) {
172     for (size_t i = 0; i < net.layers.size(); ++i) {
173         subtractInPlace(net.layers[i].A, scale(grads[i].gradLA,
174 learningRate));
175         subtractInPlace(net.layers[i].b, scale(grads[i].gradLb,
176 learningRate));
177     }
178 }
179
180 double evaluateAccuracy(Network& net, const std::vector<Matrix>&
181 images, const std::vector<int>& labels) {
182     int correct = 0;
183     int total = static_cast<int>(images.size());
184

```

```

178     for (int i = 0; i < total; ++i) {
179         Matrix output = net.forward(images[i]);
180
181         int predicted = 0;
182         double maxVal = output(0, 0);
183         for (int r = 1; r < output.rows(); ++r) {
184             if (output(r, 0) > maxVal) {
185                 maxVal = output(r, 0);
186                 predicted = static_cast<int>(r);
187             }
188         }
189
190         if (predicted == labels[i]) {
191             ++correct;
192         }
193     }
194
195     return static_cast<double>(correct) / static_cast<double>(total
196 );
197 }
198
199 double crossEntropyLoss(const Matrix& zL, int y) {
200     Matrix probs = softmax(zL);
201     return -std::log(probs(y, 0));
202 }
203
204 double evaluateLoss(Network& net, const std::vector<Matrix>& images
205 , const std::vector<int>& labels) {
206     double totalLoss = 0.0;
207     int total = static_cast<int>(images.size());
208
209     for (int i = 0; i < total; ++i) {
210         Matrix output = net.forward(images[i]);
211         totalLoss += crossEntropyLoss(output, labels[i]);
212     }
213
214     return totalLoss / static_cast<double>(total);
215 }

```

network.hpp

```

1  #include "../neural_networks/network.hpp"
2
3
4  int main() {
5      // Main test function written by Claude
6
7      Network net;
8      net.layers.push_back(Layer(2, 2, true));
9      net.layers.push_back(Layer(2, 2, false));
10     net.activation = identityScalar;
11

```

```

12 // Layer 0: A (2x2), b (2x1)
13 net.layers[0].A(0,0) = 1; net.layers[0].A(0,1) = 2;
14 net.layers[0].A(1,0) = 3; net.layers[0].A(1,1) = 4;
15 net.layers[0].b(0,0) = 1;
16 net.layers[0].b(1,0) = -1;
17
18 // Layer 1: A (2x2), b (2x1)
19 net.layers[1].A(0,0) = 3; net.layers[1].A(0,1) = 3;
20 net.layers[1].A(1,0) = -5; net.layers[1].A(1,1) = -1;
21 net.layers[1].b(0,0) = 2;
22 net.layers[1].b(1,0) = 3;
23
24 // Input
25 Matrix x0(2, 1);
26 x0(0,0) = 1;
27 x0(1,0) = 1;
28
29 // Expected results
30 Matrix expected = net.layers[1].A * (net.layers[0].A * x0 + net
31 .layers[0].b) + net.layers[1].b;
32
33 Matrix output = net.forward(x0);
34
35 std::cout << "net.forward(x0):" << std::endl;
36 for (int i = 0; i < output.rows(); ++i) {
37     std::cout << " [" << i << "] = " << output(i, 0) << std::
38     endl;
39 }
40
41 std::cout << "expected:" << std::endl;
42 for (int i = 0; i < expected.rows(); ++i) {
43     std::cout << " [" << i << "] = " << expected(i, 0) << std
44     ::endl;
45 }
46
47 // Softmax check
48 Matrix probs = softmax(output);
49
50 std::cout << "softmax(net.forward(x0)):" << std::endl;
51 double sum = 0.0;
52 for (int i = 0; i < probs.rows(); ++i) {
53     std::cout << " [" << i << "] = " << probs(i, 0) << std::
54     endl;
55     sum += probs(i, 0);
56 }
57
58 std::cout << "sum of softmax entries: " << sum << std::endl;
59
60 return 0;
61 }

```


A.11 Backward Pass

The shared network definitions used by both passes (Layer, Network, and every function called from backward()) are listed as `network.hpp` in Appendix A.10.

```

1  #include <iostream>
2  #include <cmath>
3  #include "../linear_algebra/matrix.hpp"
4  #include "../neural_networks/network.hpp"
5
6
7  int main() {
8      // Main test function written by Claude
9      // Step 1: build the network
10     Network net;
11     net.layers.push_back(Layer(2, 3, true)); // hidden layer,
ReLU
12     net.layers.push_back(Layer(3, 2, false)); // output layer, no
activation
13
14     net.layers[0].A(0,0) = 0.5; net.layers[0].A(0,1) = -0.3;
15     net.layers[0].A(1,0) = 0.2; net.layers[0].A(1,1) = 0.8;
16     net.layers[0].A(2,0) = -0.1; net.layers[0].A(2,1) = 0.4;
17     net.layers[0].b(0,0) = 0.1;
18     net.layers[0].b(1,0) = -0.2;
19     net.layers[0].b(2,0) = 0.05;
20
21     net.layers[1].A(0,0) = 1.0; net.layers[1].A(0,1) = -0.5; net.
layers[1].A(0,2) = 0.3;
22     net.layers[1].A(1,0) = -0.2; net.layers[1].A(1,1) = 0.7; net.
layers[1].A(1,2) = 0.1;
23     net.layers[1].b(0,0) = 0.0;
24     net.layers[1].b(1,0) = 0.0;
25
26     net.activation = reluScalar;
27     net.activationDerivative = reluScalarDerivative;
28
29     // Step 2: test case
30     Matrix x0(2, 1);
31     x0(0,0) = 1.0;
32     x0(1,0) = -0.5;
33     int y = 1; // valid class index, output layer has width 2
34
35     // Step 3: scalar loss function
36     auto totalLoss = [](Network& net, const Matrix& x0, int y) ->
double {
37         Matrix zL = net.forward(x0);

```

```

38     Matrix probs = softmax(zL);
39     return -std::log(probs(y, 0));
40 };
41
42 // Step 4: analytic gradients
43 std::vector<LayerGradients> grads = net.backward(x0, y);
44
45 double h = 1e-5;
46
47 // Steps 5-7: spot-check net.layers[0].A(0,0)
48 {
49     double original = net.layers[0].A(0,0);
50
51     net.layers[0].A(0,0) = original + h;
52     double lossPlus = totalLoss(net, x0, y);
53
54     net.layers[0].A(0,0) = original - h;
55     double lossMinus = totalLoss(net, x0, y);
56
57     net.layers[0].A(0,0) = original; // restore
58
59     double numerical = (lossPlus - lossMinus) / (2*h);
60     double analytic = grads[0].gradLA(0,0);
61
62     std::cout << "layers[0].A(0,0): numerical = " << numerical
63               << ", analytic = " << analytic
64               << ", diff = " << std::abs(numerical - analytic)
65 << std::endl;
66 }
67
68 // Step 8: additional spot-checks
69 {
70     // one entry in the other layer
71     double original = net.layers[1].A(1,2);
72
73     net.layers[1].A(1,2) = original + h;
74     double lossPlus = totalLoss(net, x0, y);
75
76     net.layers[1].A(1,2) = original - h;
77     double lossMinus = totalLoss(net, x0, y);
78
79     net.layers[1].A(1,2) = original;
80
81     double numerical = (lossPlus - lossMinus) / (2*h);
82     double analytic = grads[1].gradLA(1,2);
83
84     std::cout << "layers[1].A(1,2): numerical = " << numerical
85               << ", analytic = " << analytic
86               << ", diff = " << std::abs(numerical - analytic)
87 << std::endl;
88 }

```

```

87
88     {
89         // one bias entry
90         double original = net.layers[0].b(2,0);
91
92         net.layers[0].b(2,0) = original + h;
93         double lossPlus = totalLoss(net, x0, y);
94
95         net.layers[0].b(2,0) = original - h;
96         double lossMinus = totalLoss(net, x0, y);
97
98         net.layers[0].b(2,0) = original;
99
100        double numerical = (lossPlus - lossMinus) / (2*h);
101        double analytic = grads[0].gradLb(2,0);
102
103        std::cout << "layers[0].b(2,0): numerical = " << numerical
104                  << ", analytic = " << analytic
105                  << ", diff = " << std::abs(numerical - analytic)
106        << std::endl;
107    }
108
109    return 0;
110 }

```

backward_pass.cpp

A.12 MNIST Loader

```

1  #include <fstream>
2  #include <cstdint>
3  #include <cstring>
4  #include <cmath>
5  #include <random>
6  #include <vector>
7  #include <string>
8  #include "../linear_algebra/matrix.hpp"
9  #include "../neural_networks/network.hpp"
10
11
12  int32_t readInt32BigEndian(std::ifstream& stream) {
13      unsigned char buf[4];
14      stream.read(reinterpret_cast<char*>(buf), 4);
15      uint32_t value = (static_cast<uint32_t>(buf[0]) << 24)
16                      | (static_cast<uint32_t>(buf[1]) << 16)
17                      | (static_cast<uint32_t>(buf[2]) << 8)
18                      | static_cast<uint32_t>(buf[3]);
19      int32_t result;
20      std::memcpy(&result, &value, sizeof(result));
21

```

```

22     return result;
23 }
24
25 struct ImageHeader {
26     int32_t magic;
27     int32_t numImages;
28     int32_t rows;
29     int32_t cols;
30 };
31
32 ImageHeader readImageHeader(std::ifstream& file) {
33     int32_t magic = readInt32BigEndian(file);
34     int32_t numImages = readInt32BigEndian(file);
35     int32_t rows = readInt32BigEndian(file);
36     int32_t cols = readInt32BigEndian(file);
37
38     if (magic != 2051) {
39         throw std::runtime_error("Invalid MNIST image file: bad
40 magic number");
41     }
42
43     return ImageHeader{magic, numImages, rows, cols};
44 }
45
46 std::vector<Matrix> loadImages(const std::string& path) {
47     std::ifstream file(path, std::ios::binary);
48     if (!file.is_open()) {
49         throw std::runtime_error("Failed to open image file: " +
50 path);
51     }
52
53     ImageHeader header = readImageHeader(file);
54
55     size_t pixelCount = static_cast<size_t>(header.numImages)
56         * static_cast<size_t>(header.rows)
57         * static_cast<size_t>(header.cols);
58
59     std::vector<unsigned char> buffer(pixelCount);
60     file.read(reinterpret_cast<char*>(buffer.data()), pixelCount);
61
62     std::vector<Matrix> images;
63     images.reserve(header.numImages);
64
65     size_t imageSize = static_cast<size_t>(header.rows) *
66 static_cast<size_t>(header.cols);
67
68     for (int32_t i = 0; i < header.numImages; ++i) {
69         Matrix img(imageSize, 1);
70         size_t offset = static_cast<size_t>(i) * imageSize;
71         for (size_t p = 0; p < imageSize; ++p) {

```

```

69         img(p, 0) = static_cast<double>(buffer[offset + p]) /
255.0;
70     }
71     images.push_back(img);
72 }
73
74     return images;
75 }
76
77 struct LabelHeader {
78     int32_t magic;
79     int32_t numLabels;
80 };
81
82 LabelHeader readLabelHeader(std::ifstream& file) {
83     int32_t magic = readInt32BigEndian(file);
84     int32_t numLabels = readInt32BigEndian(file);
85
86     if (magic != 2049) {
87         throw std::runtime_error("Invalid MNIST label file: bad
magic number");
88     }
89
90     return LabelHeader{magic, numLabels};
91 }
92
93 std::vector<int> loadLabels(const std::string& path) {
94     std::ifstream file(path, std::ios::binary);
95     if (!file.is_open()) {
96         throw std::runtime_error("Failed to open label file: " +
path);
97     }
98
99     LabelHeader header = readLabelHeader(file);
100
101     size_t labelCount = static_cast<size_t>(header.numLabels);
102
103     std::vector<unsigned char> buffer(labelCount);
104     file.read(reinterpret_cast<char*>(buffer.data()), labelCount);
105
106     std::vector<int> labels(labelCount);
107     for (size_t i = 0; i < labelCount; ++i) {
108         labels[i] = static_cast<int>(buffer[i]);
109     }
110
111     return labels;
112 }
113
114 void initializeWeights(Network& net) {
115     std::mt19937 engine(std::random_device{}());
116

```

```

117     for (Layer& layer : net.layers) {
118         size_t n_out = layer.A.rows();
119         size_t n_in = layer.A.cols();
120
121         double limit = 1.0 / std::sqrt(static_cast<double>(n_in));
122         std::uniform_real_distribution<double> dist(-limit, limit);
123
124         for (size_t r = 0; r < n_out; ++r) {
125             for (size_t c = 0; c < n_in; ++c) {
126                 layer.A(r, c) = dist(engine);
127             }
128         }
129         // b stays at zero per convention.
130     }
131 }

```

mnist_loader.hpp

A.13 MNIST Training

```

1  #include <iostream>
2  #include <vector>
3  #include "../linear_algebra/matrix.hpp"
4  #include "../neural_networks/network.hpp"
5  #include "../mnist/mnist_loader.hpp"
6
7  int main() {
8      std::vector<Matrix> trainImages = loadImages("impl/mnist/data/
mnist/train-images-idx3-ubyte");
9      std::vector<int> trainLabels = loadLabels("impl/mnist/data/
mnist/train-labels-idx1-ubyte");
10     std::vector<Matrix> testImages = loadImages("impl/mnist/data/
mnist/t10k-images-idx3-ubyte");
11     std::vector<int> testLabels = loadLabels("impl/mnist/data/mnist
/t10k-labels-idx1-ubyte");
12
13     Network net;
14     net.layers.push_back(Layer(784, 128, true));
15     net.layers.push_back(Layer(128, 10, false));
16
17     initializeWeights(net);
18
19     net.activation = reluScalar;
20     net.activationDerivative = reluScalarDerivative;
21
22     int numEpochs = 100;
23     size_t batchSize = 32;
24     double learningRate = 0.1;
25     int loggingFrequency = 2;
26

```

```

27     size_t N = trainImages.size();
28     std::vector<size_t> indices(N);
29     std::iota(indices.begin(), indices.end(), 0);
30     std::mt19937 rng(std::random_device{}());
31
32     std::ofstream logFile("impl/mnist/data/training_log.csv");
33     logFile << "epoch,accuracy,loss\n";
34
35     for (int epoch = 0; epoch < numEpochs; ++epoch) {
36         std::shuffle(indices.begin(), indices.end(), rng);
37
38         for (size_t batchSize = 0; batchSize < N; batchSize +=
batchSize) {
39             size_t batchEnd = std::min(batchStart + batchSize, N);
40             size_t actualBatchSize = batchEnd - batchSize;
41
42             std::vector<LayerGradients> accum;
43             for (size_t i = 0; i < net.layers.size(); ++i) {
44                 accum.push_back(LayerGradients(
45                     Matrix(net.layers[i].A.rows(), net.layers[i].A.
cols()),
46                     Matrix(net.layers[i].b.rows(), net.layers[i].b.
cols())
47                 ));
48             }
49
50             for (size_t k = batchSize; k < batchEnd; ++k) {
51                 size_t idx = indices[k];
52                 std::vector<LayerGradients> exampleGrads = net.
backward(trainImages[idx], trainLabels[idx]);
53
54                 for (size_t i = 0; i < net.layers.size(); ++i) {
55                     accum[i].gradLA = accum[i].gradLA +
exampleGrads[i].gradLA;
56                     accum[i].gradLb = accum[i].gradLb +
exampleGrads[i].gradLb;
57                 }
58             }
59
60             for (size_t i = 0; i < net.layers.size(); ++i) {
61                 accum[i].gradLA = scale(accum[i].gradLA, 1.0 /
static_cast<double>(actualBatchSize));
62                 accum[i].gradLb = scale(accum[i].gradLb, 1.0 /
static_cast<double>(actualBatchSize));
63             }
64
65             applyGradients(net, accum, learningRate);
66         }
67
68         if (epoch % loggingFrequency == 0 || epoch == numEpochs -
1) {

```

```

69         std::cout << "epoch " << epoch << " complete\n";
70
71         double accuracy = evaluateAccuracy(net, testImages,
72 testLabels);
73         double trainLoss = evaluateLoss(net, trainImages,
74 trainLabels);
75
76         std::cout << "epoch " << epoch << " test accuracy: " <<
77 accuracy
78 << ", training loss: " << trainLoss << "\n";
79
80         logFile << epoch << "," << accuracy << "," << trainLoss
81 << "\n";
82     }
83 }
84
85 return 0;
86 }

```

train_mnist.cpp

Bibliography

- Boyd, S. & Vandenberghe, L. (2004), *Convex Optimization*, Cambridge University Press, Cambridge, UK.
- Bubeck, S. (2015), ‘Convex optimization: Algorithms and complexity’, *Foundations and Trends in Machine Learning* **8**(3–4), 231–357.
- Cybenko, G. (1989), ‘Approximation by superpositions of a sigmoidal function’, *Mathematics of Control, Signals and Systems* **2**(4), 303–314.
- Golub, G. H. & Van Loan, C. F. (2013), *Matrix Computations*, 4th edn, Johns Hopkins University Press, Baltimore, MD.
- Goodfellow, Ian & Ilya, A. (2016), *Deep Learning*, MIT Press, Cambridge, MA.
- Horn, R. A. & Johnson, C. R. (2013), *Matrix Analysis*, 2nd edn, Cambridge University Press, Cambridge.
- Hornik, K. (1991), ‘Approximation capabilities of multilayer feedforward networks’, *Neural Networks* **4**(2), 251–257.
- Nair, V. & Hinton, G. E. (2010), Rectified linear units improve restricted boltzmann machines, in ‘Proceedings of the 27th International Conference on Machine Learning (ICML)’, pp. 807–814.
- Rudin, W. (1976), *Principles of Mathematical Analysis*, 3rd edn, McGraw-Hill, New York, NY.
- Rumelhart, D. E., Hinton, G. E. & Williams, R. J. (1986), ‘Learning representations by back-propagating errors’, *Nature* **323**(6088), 533–536.